

DÉDICACE

“

*À mes parents, pour leur soutien et leur confiance tout
au long de mes études.*

*À mes frères et sœurs, pour leur encouragement
constant.*

À mes amis, pour leur présence et leur soutien.

*À mes collègues de classe, pour leur collaboration et
leur aide précieuse.*

*À toutes les personnes qui m'ont aidé et apporté leur
expertise et leur soutien pour la réalisation de ce projet.*

À tous ceux qui me sont chers, à vous tous

Merci.

”

- Amine

REMERCIEMENTS

Je tiens tout d'abord à exprimer ma gratitude envers **M. M'hamed AIT KBIR**, mon encadrant académique, pour ses conseils, son soutien et sa patience tout au long de ce projet. Sa disponibilité m'a permis de mener à bien ce travail.

Je remercie également **M. Mohamed ACHARKI**, mon encadrant professionnel, pour son expertise et ses conseils avisés. Grâce à lui, j'ai pu acquérir de nouvelles compétences et approfondir mes connaissances.

Je suis reconnaissant envers **M. Yassine CHAKOR**, le CEO de SMART AUTOMATION TECHNOLOGIES, pour m'avoir donné l'opportunité de réaliser ce stage et de développer ce projet.

Je remercie tout le corps administratif et professoral de la FSTT, en particulier le chef de filière **M. Mohamed BEN AHMED**, ainsi que tous les professeurs pour m'avoir assuré une formation solide.

Je tiens à remercier **M. Haitam Belainoussi**, membre de mon équipe, pour son soutien et sa collaboration durant ce projet.

Je suis profondément reconnaissant à l'ensemble de l'équipe de SAT pour leur accueil et leur soutien tout au long de mon stage. Leur aide a été essentielle à la réussite de ce projet. Merci particulièrement à **Jaber El AARAJ**, **Rania OUS- SALEH TAOUFIK**, **Hafida Khalfaoui** et **Ibtihal MAYOUCHE**, ainsi qu'à tous les membres de l'entreprise.

Je remercie mes parents pour leur soutien inconditionnel et leur encouragement constant, qui m'ont permis de persévérer.

Merci à mes amis pour leur amitié et leur soutien, qui ont été une source de motivation constante.

Enfin, merci à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet. Sans vous, je n'aurais jamais pu atteindre mes objectifs. Je vous en suis profondément reconnaissant.

RÉSUMÉ

La gestion et l'analyse de volumes massifs de données provenant de multiples sources représentent un défi majeur dans le domaine du Big Data. Bien que les technologies modernes offrent des opportunités pour traiter ces données, leur intégration et leur gestion demeurent complexes. Ce projet de fin d'études se concentre sur la conception et la mise en œuvre d'un écosystème Big Data capable de collecter, ingérer, stocker, traiter et visualiser des données en temps réel et par lots.

Notre solution intègre des technologies clés telles que Kafka, Flink, Spark, MongoDB, PostgreSQL, HDFS, Elasticsearch et Kibana. Ces composants assurent une gestion efficace des flux de données, une analyse rapide et une visualisation détaillée des informations. Le développement de cet écosystème permet de répondre aux besoins de traitement des données massives et de faciliter la prise de décisions éclairées.

Pour illustrer la capacité de l'écosystème à intégrer l'apprentissage automatique, nous avons développé un modèle prédictif pour la congestion du trafic à Paris, démontrant l'efficacité de l'infrastructure pour des applications de machine learning.

Les résultats obtenus soulignent la robustesse, l'efficacité et l'évolutivité de l'écosystème conçu, capable de répondre aux exigences complexes des environnements de données actuels et futurs. Ce projet ouvre la voie à des innovations continues dans les domaines du Big Data, avec des applications potentielles pour améliorer les processus opérationnels et les prises de décision.

Mots clés : Big Data, écosystème, traitement en temps réel, traitement par lots, gestion des données massives, apprentissage automatique, prédiction du trafic.

ABSTRACT

The management and analysis of large volumes of data from multiple sources represent a major challenge in the field of Big Data. Although modern technologies offer opportunities to process these data, their integration and management remain complex. This final year project focuses on the design and implementation of a Big Data ecosystem capable of collecting, ingesting, storing, processing, and visualizing data in real-time and batch modes.

Our solution integrates key technologies such as Kafka, Flink, Spark, MongoDB, PostgreSQL, HDFS, Elasticsearch, and Kibana. These components ensure efficient data flow management, rapid analysis, and detailed information visualization. The development of this ecosystem addresses the needs of massive data processing and facilitates informed decision-making.

To illustrate the ecosystem's capability to integrate machine learning, we developed a predictive model for traffic congestion in Paris, demonstrating the infrastructure's effectiveness for machine learning applications.

The results highlight the robustness, efficiency, and scalability of the designed ecosystem, capable of meeting the complex requirements of current and future data environments. This project paves the way for continuous innovation in the fields of Big Data, with potential applications to improve operational processes and decision-making.

Keywords : Big Data, écosystème, traitement en temps réel, traitement par lots, gestion des données massives, apprentissage automatique, prédiction du trafic.

ملخص

يمثل إدارة وتحليل كميات ضخمة من البيانات من مصادر متعددة تحديًا كبيرًا في مجال البيانات الضخمة. وعلى الرغم من أن التقنيات الحديثة توفر فرصًا لمعالجة هذه البيانات، إلا أن تكاملها وإدارتها يظل معقدًا. يركز هذا المشروع النهائي على تصميم وتنفيذ نظام بيئي للبيانات الضخمة قادر على جمع واستيعاب وتخزين ومعالجة وعرض البيانات في الوقت الفعلي والدفعات.

تدمج الحلول التي نقدمها تقنيات رئيسية مثل Kafka و Flink و Spark و MongoDB و PostgreSQL و HDFS و Elasticsearch و Kibana. تضمن هذه المكونات إدارة فعالة لتدفق البيانات وتحليل سريع وعرض تفصيلي للمعلومات. يتيح تطوير هذا النظام البيئي تلبية احتياجات معالجة البيانات الضخمة ويسهل اتخاذ القرارات المستنيرة.

لإظهار قدرة النظام البيئي على دمج التعلم الآلي، قمنا بتطوير نموذج تنبؤي للازدحام المروري في باريس، مما يوضح فعالية البنية التحتية لتطبيقات التعلم الآلي.

تسلط النتائج الضوء على قوة وكفاءة وقدرة النظام البيئي المصمم على التوسع، بما يمكنه من تلبية المتطلبات المعقدة لبيئات البيانات الحالية والمستقبلية. يفتح هذا المشروع الطريق للابتكارات المستمرة في مجالات البيانات الضخمة، مع تطبيقات محتملة لتحسين العمليات التشغيلية واتخاذ القرارات.

كلمات مفتاحية :

البيانات الضخمة، النظام البيئي، المعالجة في الوقت الفعلي، معالجة الدفعات، إدارة البيانات الضخمة، التعلم الآلي، التنبؤ بحركة المرور.

TABLE DES MATIÈRES

DÉDICACE	I
REMERCIEMENTS	II
RÉSUMÉ	III
ABSTRACT	IV
V ملخص	
LISTE DES SIGLES ET ACRONYMES	XII
INTRODUCTION GÉNÉRALE	1
I CONTEXTE GÉNÉRAL DU PROJET	2
I.1 INTRODUCTION	3
I.2 PRÉSENTATION DE L'ENTREPRISE	3
I.2.1 Approches et valeurs	3
I.2.2 Métiers et services	4
I.2.3 Produits et Solutions	5
I.3 PRÉSENTATION DU PROJET CIBLE	6
I.3.1 Le principe d'EcoDID	6
I.3.2 problématique	6
I.3.3 Objectif du projet	8
I.3.4 Approche du projet	9
I.3.5 Finalité du projet	10
I.3.6 Organisation et planification du projet	11
I.4 CONCLUSION	16
II ETAT DE L'ART	17
II.1 INTRODUCTION	18
II.2 ÉTUDE DE CONCEPTS	18
II.2.1 l'émergence du BIG DATA	18
II.2.2 Technologies Big Data émergentes	23
II.3 APPROCHE ARCHITECTURAL DES ÉCOSYSTÈMES BIG DATA	25
II.3.1 Architecture LAMBDA	25

Table des matières

II.3.2	Architecture KAPPA	26
II.3.3	Lambda VS Kappa	27
II.4	VEILLE CONCURRENTIELLE	28
II.5	VEILLE TECHNOLOGIQUE	30
II.6	CONCLUSION	32
III	ANALYSE ET CONCEPTION	33
	INTRODUCTION	34
III.1	ANALYSE DES BESOINS	34
III.1.1	L'importance de l'écosystème pour l'entreprise	34
III.1.2	Fonctionnement de notre écosystème avec EcoDID	35
III.1.3	Besoins fonctionnels	36
III.1.4	Besoins non fonctionnels	37
III.2	ÉTUDE CONCEPTUELLE	39
III.2.1	Diagramme de cas d'utilisation	39
III.2.2	Conception de l'architecture	40
III.3	ARCHITECTURE GLOBALE	47
III.3.1	Hadoop HDFS Cluster	50
III.3.2	Interactions entre les Composants	51
III.3.3	Flux de Données	52
III.3.4	Stratégie de mise en œuvre	53
III.3.5	Mode de déploiement	56
IV	IMPLEMENTATION	58
IV.1	INTRODUCTION	59
IV.2	PRÉPARATION DE L'ENVIRONNEMENT	60
IV.2.1	Configuration des outils de développement	60
IV.3	GESTION DES DÉPENDANCES	61
IV.4	DÉPLOIEMENT DES COMPOSANTS	63
IV.4.1	Mise en place de Docker et des conteneurs	63
IV.4.2	Déploiement de Kafka et Zookeeper	63
IV.4.3	Configuration des topics et des connecteurs Kafka	64
IV.4.4	Déploiement de Flink	65
IV.5	CONFIGURATION DES JOBS DE TRAITEMENT EN TEMPS RÉEL	66
IV.6	DÉPLOIEMENT DES BASES DE DONNÉES	67
IV.7	DÉPLOIEMENT DE ELASTICSEARCH ET KIBANA	67
IV.8	TESTS ET VALIDATION	69
IV.8.1	Tests unitaires et d'intégration	69
IV.8.2	Tests de performance	69
IV.9	CONCLUSION	71
V	ETUDE DE CAS DE LA CONGESTION DU TRAFIC EN TEMPS RÉEL	72
V.1	ETUDE DE CAS DE LA CONGESTION DU TRAFIC EN TEMPS RÉEL	73

Table des matières

V.1.1	Introduction	73
V.1.2	Capteurs et logistique mise en place	73
V.1.3	Source et prétraitement des données	74
V.1.4	Exploration et Visualisation des Données	77
V.2	NETTOYAGE ET PRÉPARATION DES DONNÉES	79
V.3	CHOIX ET ENTRAÎNEMENT DES MODÈLES D'APPRENTISSAGE AU- TOMATIQUE	80
V.3.1	Machines à Vecteurs de Support	80
V.3.2	K-Nearest Neighbors (KNN)	82
V.3.3	Gated Recurrent Unit (GRU)	84
V.4	INTERPRÉTATION DES MÉTRIQUES DE PERFORMANCE	87
V.4.1	Métriques d'évaluation	87
V.4.2	Comparaison entre les modèles	88
V.5	INTÉGRATION DU MODÈLE DANS L'ÉCOSYSTÈME EXISTANT	88
V.6	CONCLUSION	89
	CONCLUSION ET PERSPECTIVES	90
	ANNEXES	96
A	DÉFINITIONS	97
B	PERFORMANCE BENCHMARKS	101

TABLE DES FIGURES

I.1	Logo de l'entreprise	3
I.2	Valeurs de l'entreprise SAT	4
I.3	Tableau Kanban du projet	12
I.4	diagramme des tâches	14
I.5	Diagramme de Gantt du projet	14
II.1	Les Trois piliers du Big Data	19
II.2	Les deux nouveaux piliers du Big Data	19
II.3	Mode de fonction de traitement en flux	24
II.4	traitement par lot	24
II.5	Architecture Lambda	26
II.6	Architecture KAPPA	26
III.1	Diagramme de cas d'utilisation de l'écosystème Big Data	39
III.2	Architecture globale de l'écosystème solution	40
III.3	Les sources des données	41
III.4	Couche d'ingestion des données	42
III.5	Couche de stockage des données	42
III.6	Couche de traitement des données	43
III.7	Couche de consommation de données	43
III.8	Représentation visuelle de l'architecture Technologique	44
III.9	Phases de mise en oeuvre	54
IV.1	Architecture Docker	61
IV.2	Brokers Kafka	64
IV.3	Partitions et répartition de topic entre les brokers	65
IV.4	Connecteur MongoDB consommant des données du topic Kafka	65
IV.5	tableau de bord du JobManager Flink	66
IV.6	Gestion du taskManager Flink	66
IV.7	Gestion des indices Elasticsearch	68
IV.8	Les conteneurs en cours d'exécution	69
IV.9	Évaluation des performances de l'écosystème	70
V.1	1/3 Tableau représentant les données utilisées	76
V.2	2/3 Tableau représentant les données utilisées	76
V.3	3/3 Tableau représentant les données utilisées	76

Table des figures

V.4	Description cartographique des données de Paris	77
V.5	Distribution du risque de congestion	78
V.6	Taux d'occupation par état de trafic	79
V.7	Architecture de Model SVM	81
V.8	Resultat de model SVM	82
V.9	l'architecture de model KNN	83
V.10	Resultat de KNN	84
V.11	l'architecture de model GRU	85
V.12	Fonction de coût régularisée avec L2	85
V.13	Evolution de la fonction d'erreur en fonction d'epochs (GRU)	87
B.1	Structure du répertoire contenant les Dockerfiles et les scripts de dé- ploiement	104
B.2	QR Code vers le dépôt GitHub	106

LISTE DES TABLEAUX

II.1	Big Data storage models	22
II.2	Comparaison de LAMBDA et KAPPA	27
II.3	Fournisseurs de services Big Data	29
V.1	Taux d'occupation des routes	75
V.2	Sélection des attributs et transformations appliquées	80
V.3	Hyperparamètres du modèle SVM	81
V.4	Prédiction du modèle SVM	82
V.5	Hyperparamètres du modèle KNN	83
V.6	Prédiction du modèle KNN	84
V.7	Hyperparamètres du modèle GRU	86
V.8	Prédiction du modèle GRU	86
V.9	Métriques d'évaluation	87
V.10	Comparaison entre les modèles	88
B.1	Performances de Référence des Composants de l'Écosystème Big Data	103

LISTE DES SIGLES ET ACRONYMES

SAT	SMART AUTOMATION TECHNOLOGIES
EcoDID	Application Programming Interface
IoT	Internet des objets
NoSQL	Not Only SQL
NLP	Traitement du Langage Naturel
HDFS	Hadoop Distributed File System
S3	Amazon Simple Storage Service
EMR	Elastic MapReduce
HADInsight	Hadoop on Azure
VM	Machine Virtuelle
SQL	Structured Query Language
AWS	Amazon Web Services
GCP	Google Cloud Platform
CIOMag	Chief Information Officer Magazine
CSV	Comma Separated Values
REST	Representational State Transfer
API	Application Programming Interface
CIA	Confidentialité, Intégrité et Disponibilité
ETL	Extract, Transform, Load
IDE	Integrated Development Environment
RAM	Random Access Memory
CPU	Central Processing Unit
SVM	Support Vector Machine
KNN	K-Nearest Neighbors
GRU	Gated Recurrent Unit
SMOTE	Synthetic Minority Over-sampling Technique
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
Adam	Adaptive Moment Estimation
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation

INTRODUCTION GÉNÉRALE

LE monde moderne est caractérisé par une explosion exponentielle des données, alimentée par des facteurs tels que la numérisation croissante, l'Internet des objets (IoT) et les systèmes intelligents. Cette profusion de données a transformé la manière dont les entreprises opèrent, offrant des opportunités sans précédent mais posant également des défis majeurs en matière de gestion et d'analyse des données à grande échelle. Dans ce contexte, le Big Data est devenu un pilier essentiel pour les organisations cherchant à exploiter pleinement le potentiel de leurs données et à prendre des décisions stratégiques basées sur des insights pertinents.

Ce projet s'inscrit dans le cadre d'une collaboration avec l'entreprise **Smart Automation Technologies**, une entreprise de pointe dans le domaine de l'automatisation et des technologies appliquées à l'intelligence artificielle. L'objectif principal est de concevoir et de mettre en œuvre un écosystème Big Data capable de gérer efficacement les données massives générées par des projets clés, notamment l'“Edge box pour les camions” ou l'“optimisation des itinéraires pour les camions de marchandises”.

Pour atteindre cet objectif ambitieux, une approche combinant des concepts essentiels du Big Data tels que l'ingestion de données, le stockage distribué, le traitement en temps réel (stream processing), l'analyse avancée et la visualisation des données a été adoptée.

Cette introduction pose ainsi les bases pour explorer en détail les différentes phases du projet, allant de l'analyse des besoins à la réalisation de l'écosystème Big Data, en passant par l'intégration des technologies, les résultats obtenus et les perspectives d'avenir. Ce rapport détaille une approche stratégique et technologique visant à transformer la gestion du transport et de la logistique à travers l'utilisation intelligente et proactive des données massives.

I Contexte Général du Projet

SOMMAIRE

I.1	INTRODUCTION	3
I.2	PRESENTATION DE L'ENTREPRISE	3
I.2.1	Approches et valeurs	3
I.2.2	Métiers et services	4
I.2.3	Produits et Solutions	5
I.3	PRÉSENTATION DU PROJET CIBLE	6
I.3.1	Le principe d'EcoDID	6
I.3.2	problématique	6
I.3.3	Objectif du projet	8
I.3.4	Approche du projet	9
I.3.5	Finalité du projet	10
I.3.6	Organisation et planification du projet	11
I.4	CONCLUSION	16

CE chapitre offre un aperçu complet de l'environnement de l'entreprise d'accueil, ainsi que les objectifs et les défis du projet. Il détaille les approches adoptées, l'organisation et la planification et met en lumière l'importance stratégique de cette initiative pour l'entreprise.

I.1 Introduction

Ce chapitre présente le contexte global du projet, en mettant en lumière l'entreprise Smart Automation Technologies et en introduisant ses objectifs et défis. Nous examinerons la problématique, les objectifs, les approches adoptées et les résultats attendus. En outre, nous détaillerons l'organisation du projet, incluant la méthodologie de travail et la planification envisagée.

I.2 Présentation de l'entreprise

SMART AUTOMATION TECHNOLOGIES est une entreprise fondée par un groupe de consultants seniors spécialisés dans les nouvelles technologies de l'information, la science des données et l'intelligence artificielle. L'entreprise se distingue par son engagement envers l'innovation et la performance, offrant son expertise aux entreprises et administrations souhaitant des solutions adaptées à leurs besoins et contraintes. L'approche de **SMART AUTOMATION TECHNOLOGIES** repose sur une recherche continue et un développement constant pour rester à la pointe de la technologie de l'information et de l'intelligence artificielle. En tant que partenaire de confiance, l'entreprise accompagne ses clients depuis la conception jusqu'à l'évolution de leurs systèmes d'information, ainsi que dans leurs besoins en études, formations et progiciels intégrés. L'objectif de **SMART AUTOMATION TECHNOLOGIES** est d'établir des relations durables avec ses clients en tant que maillon essentiel de leur chaîne de valeur. L'entreprise met à disposition les compétences en informatique et en apprentissage automatique nécessaires à la réussite de ses clients, leur permettant ainsi de se concentrer sur leur cœur de métier.



FIG. I.1 : Logo de l'entreprise

I.2.1 Approches et valeurs

SMART AUTOMATION TECHNOLOGIES adopte une approche axée sur la différenciation en accordant une grande importance à l'éthique et à la déontologie. Cette approche garantit une relation transparente et engagée avec ses clients,

basée sur la confiance et la compréhension mutuelle.

L'entreprise cultive une relation de proximité avec ses clients afin de mieux comprendre leurs besoins et d'anticiper leurs attentes. Cette relation repose sur la confiance et une communication ouverte, favorisant ainsi une collaboration fructueuse et durable.

De plus, **SMART AUTOMATION TECHNOLOGIES** reconnaît la valeur de son capital humain et investit dans le développement de ses collaborateurs. Ces derniers bénéficient d'un encadrement, d'un suivi, d'une évaluation et d'une formation permanente, assurant ainsi un haut niveau de compétence et d'engagement.

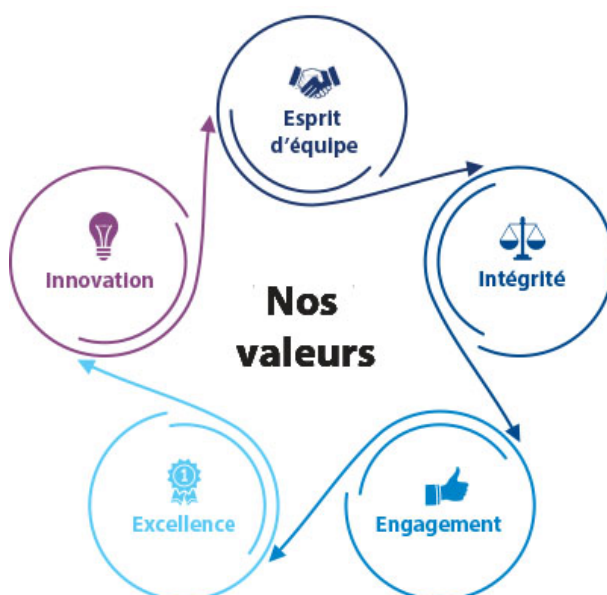


FIG. I.2 : Valeurs de l'entreprise SAT

I.2.2 Métiers et services

SMART AUTOMATION TECHNOLOGIES offre une gamme complète de services et de solutions dédiées, mettant en avant son expérience et son expertise pour répondre aux besoins variés de ses clients. Voici un aperçu des métiers et services proposés :

- **Services et Solutions Dédiées** : **SMART AUTOMATION TECHNOLOGIES** met en œuvre son savoir-faire dans la conception et la mise en place de systèmes d'information adaptés à chaque client. Les solutions dédiées sont conçues pour apporter une réelle valeur ajoutée dans divers secteurs d'activités.

- **Support et Formations** : En plus de ses services de conception, SMART AUTOMATION TECHNOLOGIES assure un support continu à ses clients, ainsi que des formations pour garantir une utilisation optimale des solutions mises en place.

La stratégie conseil de **SMART AUTOMATION TECHNOLOGIES** repose sur une approche centrée sur le client, le projet et la méthode. L'entreprise accompagne ses clients dans leurs projets d'organisation et d'évolution, conduisant leur stratégie d'évolution vers un plan d'orientation et de transformation du système d'information. Voici quelques opérations spécifiques réalisées par **SMART AUTOMATION TECHNOLOGIES** :

- **Gouvernance IT et Management de projets**
- **Conduite du Changement**
- **Élaboration d'un Schéma Directeur Informatique**
- **Urbanisation du système d'information**
- **Assistance à Maîtrise d'Ouvrage**
- **Architecture logicielle**
- **Solutions clé en main**
- **Benchmarking et audit logiciel**

Ces métiers et services illustrent l'engagement de **SMART AUTOMATION TECHNOLOGIES** à fournir des solutions personnalisées et innovantes, tout en accompagnant ses clients dans leur évolution numérique et stratégique.

I.2.3 Produits et Solutions

SMART AUTOMATION TECHNOLOGIES s'est engagée dans la création de produits répondant à des besoins réels, en mettant l'accent sur l'intégration de l'intelligence artificielle, de la science des données et du Big Data, notamment dans les domaines de l'industrie, de la logistique, du marketing et de la finance. L'entreprise explore également la mise en œuvre du processus de l'e-Transformation autour du principe d'écosystème digital, intelligent et décisionnel (EcoDID). Actuellement, **SMART AUTOMATION TECHNOLOGIES** développe plusieurs projets de recherche et développement en partenariat avec le milieu universitaire, notamment :

- **Technologie Edge Box pour la maintenance prédictive**

- Technologie Edge Data Center
- Plateforme logistique basée sur le concept EcoDID
- Plateforme marketing basée sur le concept EcoDID
- Plateforme éducative basée sur le concept EcoDID

I.3 Présentation du projet cible

I.3.1 Le principe d'EcoDID

— **EcoDID**, ou écosystème numérique, intelligent et décisionnel, est un concept développé par SMART AUTOMATION TECHNOLOGIES pour répondre aux besoins croissants de gestion de données dans divers domaines tels que l'industrie, la logistique, le marketing et la finance. Le principe fondamental d'EcoDID consiste à intégrer les technologies de pointe telles que l'intelligence artificielle, la science des données et le Big Data pour créer un environnement numérique agile et réactif.

— L'objectif d'EcoDID est de fournir une plateforme holistique permettant la collecte, le stockage, le traitement et l'analyse des données provenant de différentes sources, y compris les dispositifs Edge, les appareils finaux et le Cloud. En mettant l'accent sur l'intégration transparente des données et des technologies, EcoDID vise à fournir des solutions intelligentes et décisionnelles pour soutenir la prise de décision stratégique et l'optimisation des processus métier.

— Grâce à EcoDID, les organisations peuvent bénéficier d'une gestion efficace des données, d'une prise de décision plus rapide et plus éclairée, ainsi que d'une meilleure compréhension des tendances et des modèles grâce à l'analyse avancée des données. En intégrant les principes d'EcoDID dans notre projet, nous visons à créer un environnement numérique innovant et performant, capable de répondre aux défis complexes de la gestion de données dans un monde en constante évolution.

I.3.2 problématique

Dans le contexte du Big Data, l'entreprise est confrontée à plusieurs défis liés à ses projets axés sur les données (data-driven projects). Le déploiement du "Edge box" pour la maintenance prédictive, tant dans le domaine de la logistique que dans celui des écosystèmes informatiques, génère continuellement des données volumineuses, notamment des journaux (logs) et des métriques provenant des capteurs intégrés aux

serveurs ou aux camions. De plus, les plateformes marketing et éducatives produisent également d'importants volumes de données.

Pour améliorer la qualité de ses services et l'expérience utilisateur, toutes ces données sont cruciales pour l'entreprise. Cependant, l'augmentation exponentielle des sources de données dépasse les capacités des solutions traditionnelles qui ne sont pas conçues pour gérer efficacement ce type de projet.

Ainsi, les défis spécifiques auxquels l'entreprise est confrontée dans ce contexte incluent :

1. **Gestion de données massives** : La quantité importante de données générées par les différents projets data-driven nécessite un écosystème capable de stocker, traiter et analyser ces données de manière efficace et évolutive.
2. **Intégration de données hétérogènes** : Les données proviennent de sources diverses et variées, ce qui rend leur intégration et leur harmonisation complexes. Il est essentiel de pouvoir agréger et corrélérer ces données pour en tirer des insights pertinents.
3. **Traitement en temps réel** : La nécessité de traiter les données en temps réel pour des applications critiques telles que la maintenance prédictive et les décisions opérationnelles demande des capacités de traitement rapides et réactives.
4. **Sécurité et confidentialité des données** : Avec la sensibilité des données collectées, la sécurité et la confidentialité des données sont des préoccupations majeures. Il est crucial de mettre en place des mesures de protection des données tout au long de leur cycle de vie.
5. **Optimisation des ressources Edge** : Pour les projets impliquant l'utilisation de l'Edge Computing, il est essentiel d'optimiser l'utilisation des ressources Edge pour garantir des performances optimales et une gestion efficace des données au niveau local.

La problématique du Big Data au sein de l'entreprise se focalise sur la nécessité de gérer efficacement un volume massif de données, de traiter ces données à grande vitesse, en temps réel et par lots, ainsi que sur la diversité des sources de données hétérogènes. Il est également crucial d'assurer la véracité et la sécurité des données. L'objectif est de surmonter ces défis pour exploiter efficacement les données, en extraire des insights précieux, prendre des décisions stratégiques pertinentes et améliorer la qualité des services offerts.

I.3.3 Objectif du projet

L'objectif principal de ce projet est de concevoir et de mettre en œuvre un écosystème Big Data robuste et évolutive, capable de répondre aux besoins spécifiques de l'entreprise. Les objectifs fixés pour ce projet sont les suivants :

1. **Améliorer la gestion des données :** Mettre en place une architecture Big Data qui permet de gérer efficacement le volume massif, la variété et la vitesse des données générées par les différents projets data-driven de l'entreprise.
2. **Optimiser le traitement en temps réel :** Développer des capacités de traitement en temps réel pour permettre des analyses et des décisions instantanées basées sur les données en continu.
3. **Intégrer et harmoniser les données hétérogènes :** Assurer l'intégration et la cohérence des données provenant de sources diverses telles que les capteurs IoT, les systèmes de surveillance et les plateformes de gestion.
4. **Garantir la véracité et la sécurité des données :** Mettre en place des mécanismes de contrôle de qualité des données pour garantir leur exactitude, leur fiabilité et leur confidentialité tout au long de leur cycle de vie.
5. **Faciliter l'analyse et l'extraction d'insights :** Développer des capacités avancées d'analyse de données pour extraire des insights précieux, identifier des tendances, détecter des anomalies et prendre des décisions stratégiques pertinentes.
6. **Optimiser le traitement par lot (batch processing) :** Mettre en place des processus efficaces de traitement par lot pour les tâches qui ne nécessitent pas de traitement en temps réel, tout en garantissant des performances et une fiabilité élevées.

En atteignant ces objectifs, l'entreprise pourra exploiter pleinement le potentiel de ses données, prendre des décisions stratégiques éclairées, améliorer la qualité de ses services et rester compétitive sur le marché en pleine évolution.

Les bénéfices escomptés pour les clients incluent une meilleure expérience utilisateur, des services personnalisés et innovants basés sur les données, une prise de décision plus rapide et plus précise, ainsi qu'une optimisation des opérations et des coûts. Pour l'entreprise, les bénéfices comprennent une meilleure compétitivité, une croissance des revenus, une optimisation des ressources et une capacité accrue à répondre aux besoins évolutifs du marché.

I.3.4 Approche du projet

Pour résoudre la problématique du Big Data, une approche méthodologique rigoureuse a été adoptée, mettant en avant l'utilisation de technologies et de méthodes innovantes pour concevoir et mettre en œuvre la solution. Les principales étapes de l'approche du projet sont les suivantes :

1. **Analyse des besoins et des contraintes** : La première étape consiste à analyser en profondeur les besoins spécifiques de l'entreprise en matière de gestion des données, de traitement en temps réel et par lot (batch processing), d'intégration des sources de données et de sécurité. Cette analyse permet d'identifier les contraintes techniques, les exigences fonctionnelles et les objectifs à atteindre.
2. **Conception de l'architecture Big Data** : Sur la base de l'analyse des besoins, une architecture Big Data est conçue pour répondre aux exigences de stockage distribué, de traitement en temps réel, traitement par lot distribuer, d'analyse avancée et de sécurité des données. Cette architecture inclut des composants tels que les systèmes de gestion de données distribués, les moteurs de traitement de flux en temps réel, des technologie de traitement distribuer, les outils d'analyse et de visualisation, ainsi que les mécanismes de sécurité.
3. **Sélection des technologies** : Un processus rigoureux de sélection des technologies est effectué pour choisir les outils et les plateformes les mieux adaptés aux besoins de l'entreprise. Cela inclut l'utilisation de technologies telles que Hadoop HDFS, Apache Kafka, Apache Flink, Apache Spark, Apache Hive, MongoDB, Grafana, ELK, et Docker, entre autres.
4. **Implémentation et intégration** : Une fois les technologies sélectionnées, l'implémentation de l'architecture Big Data est réalisée en suivant des bonnes pratiques de développement et d'intégration. Les différents composants sont configurés et intégrés pour assurer leur interopérabilité et leur fonctionnement harmonieux.
5. **Tests et validation** : Des tests approfondis sont effectués pour valider le bon fonctionnement de l'écosystème Big Data . Cela comprend des tests de performance, de fiabilité, de sécurité et de conformité aux exigences métier.
6. **Déploiement et optimisation** : Une fois validée, l'écosystème est déployé dans l'environnement de production. Des processus d'optimisation continue sont mis en place pour garantir des performances optimales, une évolutivité adéquate et une adaptation aux évolutions technologiques et aux besoins changeants de l'entreprise.

Cette approche méthodologique permet de concevoir et de mettre en œuvre une solution Big Data robuste, évolutive et adaptée aux besoins spécifiques de l'entreprise, tout en garantissant des performances optimales, une sécurité renforcée et une gestion efficace des données.

I.3.5 Finalité du projet

La finalité principale de ce projet est de fournir à l'entreprise un écosystème Big Data efficace et performant, permettant de tirer pleinement parti de ses données pour obtenir des bénéfices concrets et des résultats finaux significatifs. Les bénéfices et les résultats visés par le projet sont les suivants :

1. **Optimisation des opérations** : Grâce à une gestion efficace des données et à des capacités de traitement en temps réel, les opérations de l'entreprise seront optimisées, réduisant les temps de traitement et améliorant la réactivité aux événements en temps réel.
2. **Amélioration de la prise de décision basée sur l'analyse batch** : Le traitement par lot permettra d'effectuer des analyses approfondies sur des volumes importants de données historiques, permettant ainsi une prise de décision plus éclairée et stratégique, basée sur des tendances et des modèles statistiques fiables.
3. **Stockage distribué (Distributed Storage)** : Mettre en place des solutions de stockage distribué pour gérer efficacement le volume massif de données générées par les projets.
4. **Disponibilité des données (Data Availability)** : Garantir la disponibilité et l'accessibilité des données à tout moment pour les utilisateurs autorisés.
5. **Sécurité des données (Data Security)** : Mettre en œuvre des mesures de sécurité robustes pour assurer la confidentialité, l'intégrité et la disponibilité des données sensibles.
6. **Gestion efficace du cycle de vie des données** : Le traitement par lot permettra également une gestion efficace du cycle de vie des données, incluant la collecte, le nettoyage, la transformation et le stockage des données, assurant ainsi leur intégrité et leur qualité tout au long de leur utilisation.
7. **Évolutivité (Scalability)** : Concevoir un écosystème évolutif qui peut s'adapter à la croissance des données et des besoins de l'entreprise sans compromettre les performances.

I.3.6 Organisation et planification du projet

I.3.6.1 Méthodologie Kanban

Pour assurer une gestion efficace du projet, nous avons opté pour une méthodologie de gestion de projet Kanban, qui favorise la flexibilité, la collaboration et l'adaptabilité tout au long du processus de développement de l'écosystème Big Data.

Kanban se concentre sur la visualisation du flux de travail et la gestion du travail en cours (WIP).

Voici comment Kanban nous a aidés :

- **Adaptabilité** : Kanban permet une adaptation continue aux changements et aux nouveaux besoins au fur et à mesure qu'ils surviennent, essentielle dans un projet innovant comme le nôtre.
- **Visibilité** : Le tableau Kanban transparent facilite la visibilité sur l'état d'avancement des tâches et la gestion des priorités, permettant une allocation efficace des ressources.
- **Amélioration continue** : Les réunions régulières de revue et d'amélioration du flux de travail assurent une optimisation continue du processus de développement.

En appliquant ces principes Kanban, nous avons pu gérer efficacement le projet de mise en place de l'écosystème Big Data, en mettant l'accent sur le flux de travail continu et l'amélioration constante.

Chapitre I. Contexte Général du Projet

Kanban board

Kanban board

Detailed board

Table view

Table

Table view (1)

+

Big Data Engineer

23

...

+

Name	Status	Assign	Deadline	Team	AI keywords
Recherche	Reviewing	Haitam B Amine Aissi	23 février 2024 → 4 m	Big Data Engineer	Documentation
Analyse des besoins	Done	Amine Aissi Haitam	9 février 2024 → 23 fé	Big Data Engineer	Documentation
Hdfs Security	Not started	Haitam B		Big Data Engineer	Documentation Deployment Integrat
Security Configuration Flink	Not started	Amine Aissi		Big Data Engineer	Documentation
Encryption	In development	Haitam B		Big Data Engineer	Testing Deployment Documentat
SSL/TLS kafka	In development	Amine Aissi		Big Data Engineer	Integration Deployment Documenta
Authentication and Authorization	Not started	Amine Aissi Haitam		Big Data Engineer	Integration
Fault Tolerance and Data Recovery Testing	Testing	Haitam B		Big Data Engineer	Testing
Fault Tolerance and High Availability Testing	Testing	Amine Aissi		Big Data Engineer	Testing
Aggregation in Apache flink	Testing	Amine Aissi		Big Data Engineer	Testing
Connecteur PostgreSQL	Reviewing	Amine Aissi		Big Data Engineer	Integration Deployment Documenta
Mode de déploiement	Reviewing	Amine Aissi Haitam		Big Data Engineer	Deployment
Hive Cluster	Reviewing	Haitam B		Big Data Engineer	Deployment Testing
MongoDB Cluster	Done	Amine Aissi		Big Data Engineer	Documentation Deployment Perform
Grafana	Done	Haitam B		Big Data Engineer	Documentation Deployment Testing
Elasticsearch et Kibana	Reviewing	Amine Aissi		Big Data Engineer	Documentation Deployment
Flink Cluster	Done	Amine Aissi		Big Data Engineer	Documentation Deployment Perform
Kafka Cluster	Done	Amine Aissi		Big Data Engineer	Documentation Deployment Perform
Kafka Connect Cluster	Done	Amine Aissi		Big Data Engineer	Integration Deployment Documenta
Redation de rapport	Done	Amine Aissi	1 avril 2024 → 21 juin	Big Data Engineer	Documentation
Documentation	Reviewing	Amine Aissi	11 mars 2024	Big Data Engineer	Documentation
FlinkML	Reviewing	Amine Aissi	15 juin 2024	Big Data Engineer	Integration Performance Deploymen
Conception	Done	Amine Aissi Haitam	4 avril 2024 → 1 mai 2	Big Data Engineer	Documentation

FIG. I.3 : Tableau Kanban du projet

I.3.6.2 Revues de Flux de Travail (Kanban Meetings)

Pour assurer un suivi continu de l’avancement du projet, des revues régulières du flux de travail avec mon encadrant au sein de l’entreprise ont été organisées. Ces revues avaient pour but de :

- Discuter de l’avancement des tâches en cours et à venir.
- Identifier et résoudre les problèmes rencontrés.
- Ajuster les objectifs et les tâches en fonction des résultats obtenus et des défis rencontrés.
- Planifier les prochaines étapes et les livraisons.

Ces réunions étaient cruciales pour maintenir une communication fluide et pour s’assurer que le projet avançait conformément aux délais et aux objectifs fixés.

I.3.6.3 Environnement de travail

Travail en équipe Les différentes parties de l'écosystème ont été prises en charge par :

- Amine Aissi : élève ingénieur BIG DATA en tant que "Big Data Engineer".
- Haitam Belainoussi : élève ingénieur BIG DATA en tant que "Big Data Engineer".

Répartition des tâches

La répartition des tâches a été effectuée après la conception pour que chacun puisse construire sa partie de l'écosystème. Après la conception, la responsabilité de chaque partie de l'écosystème choisie est assignée, et le développeur de cette partie de l'écosystème prend en charge tous les aspects de la construction, des tests, du déploiement ainsi que des cas d'utilisation.

Avantages

La construction d'un tel écosystème, qui couvre différents aspects, nécessite un travail approfondi pour s'assurer que chaque partie de l'écosystème est couverte, de la première phase de conception jusqu'à la dernière.

I.3.6.4 Tâches

Les tâches spécifiques à réaliser pour atteindre les objectifs du projet sont les suivantes :

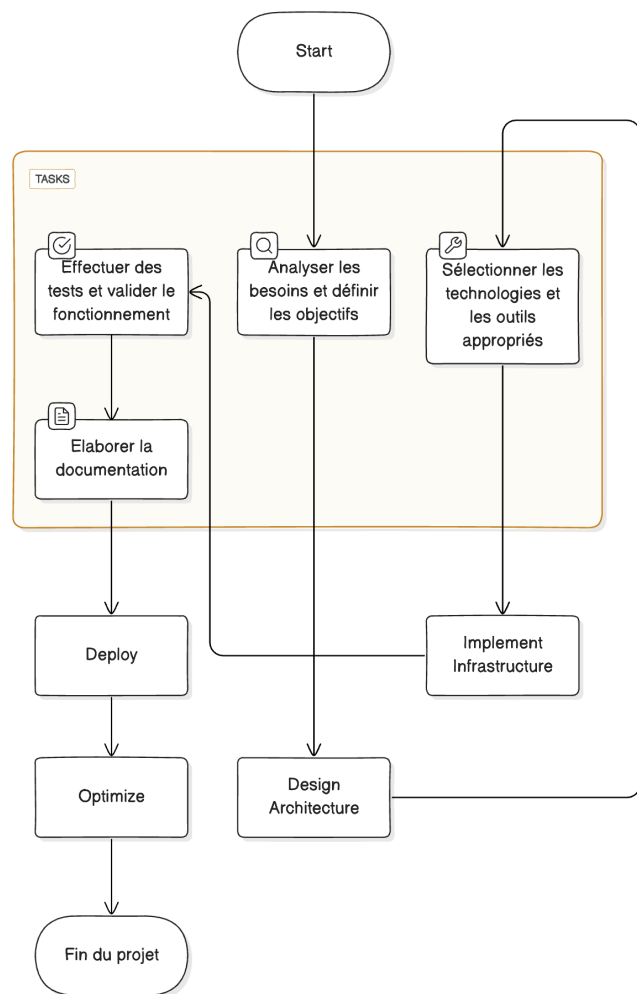


FIG. I.4 : diagramme des tâches

I.3.6.5 Diagramme de Gantt

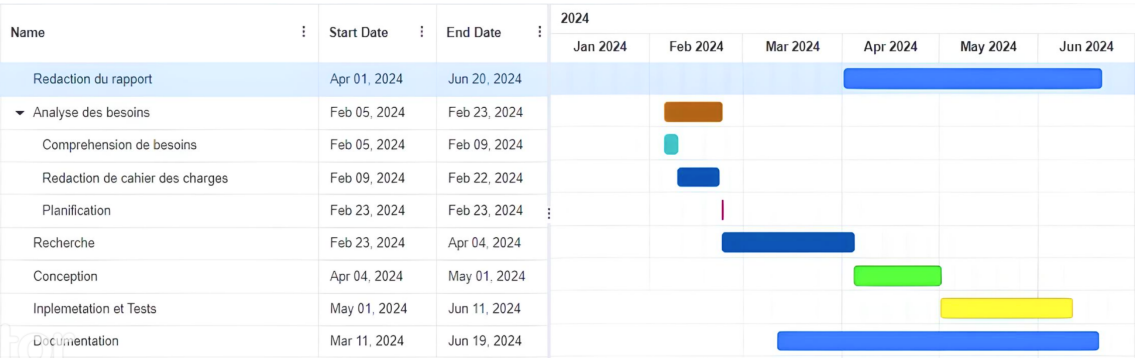


FIG. I.5 : Diagramme de Gantt du projet

I.3.6.6 Difficulté du projet

Le projet de mise en place d'un écosystème Big Data au sein de l'entreprise Smart Automation Technologies est confronté à plusieurs défis majeurs, notamment :

1. **Complexité technologique :** L'un des principaux défis réside dans la complexité technologique inhérente à ce type de projet. L'utilisation d'un éventail de technologies Big Data telles que Hadoop, Kafka, Flink, Spark, Hive, entre autres, nécessite une expertise approfondie pour assurer leur intégration harmonieuse et leur fonctionnement optimal. La diversité des technologies et leur évolution constante exigent une veille technologique continue et une capacité d'adaptation rapide aux nouvelles tendances et aux mises à jour.
2. **Intégration des systèmes existants :** L'intégration de l'écosystème Big Data avec les systèmes existants de l'entreprise présente un défi majeur. Les écosystèmes legacy, les bases de données traditionnelles et les applications préexistantes doivent être compatibles et interopérables avec les nouveaux outils et technologies mis en place. Cela nécessite une analyse approfondie des architectures existantes, une conception d'interfaces d'intégration efficaces et des tests rigoureux pour assurer la cohérence et la performance globale du système.
3. **Manque de contenu éducatif spécifique :** Un autre défi important est le manque de contenu éducatif spécifique concernant ce type de projet. Les ressources et les formations dédiées à l'intégration des pipelines de données massives et à la gestion des environnements Big Data peuvent être limitées. Cela rend crucial le développement de programmes de formation internes ou l'engagement d'experts externes pour combler ce manque de connaissances et renforcer les compétences nécessaires au sein de l'équipe.
4. **Ambiguïté de la documentation des technologies :** La documentation des différentes technologies utilisées dans le cadre du projet peut parfois être ambiguë ou incomplète. Cela peut entraîner des difficultés lors de la mise en œuvre, de la configuration et de l'optimisation des outils Big Data. Une approche proactive consiste à rechercher des ressources fiables, à participer à des communautés en ligne et à établir des partenariats avec des fournisseurs de technologie pour bénéficier d'un support et de conseils spécialisés.

En surmontant ces défis avec une approche méthodique, une collaboration étroite entre les équipes techniques et métiers, une formation adéquate et une gestion efficace du changement, le projet peut atteindre ses objectifs et contribuer de manière significative à l'amélioration des services et des performances de l'entreprise dans le domaine du Big Data.

I.4 Conclusion

Le chapitre sur le contexte général du projet a permis de dresser un tableau complet de l'entreprise Smart Automation Technologies, de définir les objectifs et les défis du projet, d'explorer les approches adoptées, ainsi que de détailler l'organisation et la planification du projet. À travers cette analyse, il ressort clairement que la mise en place d'un écosystème Big Data est cruciale pour répondre aux besoins croissants de l'entreprise en matière de gestion des données et d'optimisation des processus. En conclusion, ce chapitre a posé les bases essentielles pour la compréhension globale du projet, mettant en évidence l'importance stratégique de cette initiative pour Smart Automation Technologies. La prochaine étape consistera à plonger plus en profondeur dans chaque aspect du projet, en examinant notamment l'approche méthodologique adoptée, les technologies et outils sélectionnés, ainsi que les défis spécifiques à surmonter.

II Etat de l'art

SOMMAIRE

II.1	INTRODUCTION	18
II.2	ÉTUDE DE CONCEPTS	18
II.2.1	l'émergence du BIG DATA	18
II.2.2	Technologies Big Data émergentes	23
II.3	APPROCHE ARCHITECTURAL DES ÉCOSYSTÈMES BIG DATA	25
II.3.1	Architecture LAMBDA	25
II.3.2	Architecture KAPPA	26
II.3.3	Lambda VS Kappa	27
II.4	VEILLE CONCURRENTIELLE	28
II.5	VEILLE TECHNOLOGIQUE	30
II.6	CONCLUSION	32

DANS ce chapitre, nous examinerons l'état de l'art du big data en commençant par une étude conceptuelle où nous aborderons l'émergence du big data, les divers types de données, les modèles de stockage, la sécurité, et les technologies émergentes. De plus, nous analyserons l'approche architecturale des écosystèmes big data et examinerons la veille concurrentielle et technologique.

II.1 Introduction

“

Chaque jour, nous générons 2,5 trillions d'octets de données. A tel point que 90% des données dans le monde ont été créées au cours des deux dernières années seulement. Ces données proviennent de partout : de capteurs utilisés pour collecter les informations climatiques, de messages sur les sites de médias sociaux, d'images numériques et de vidéos publiées en ligne, d'enregistrements transactionnels d'achats en ligne et de signaux GPS de téléphones mobiles, pour ne citer que quelques sources.

”

— **IBM**

À l'ère numérique actuelle, les entreprises de tous les secteurs s'efforcent de rester à la pointe pour maximiser les insights tirés des vastes volumes de données générées par diverses sources, telles que les transactions commerciales, les interactions sur les réseaux sociaux et les capteurs IoT. Ce chapitre vise à comprendre comment les solutions actuelles traitent le Big Data et à examiner les systèmes existants qui le gèrent efficacement. En mettant en lumière les innovations, les outils et les techniques permettant de créer des écosystèmes robustes et évolutifs, ainsi que les défis et opportunités associés.

L'importance de cet état de l'art réside dans sa capacité à offrir une vue d'ensemble des technologies existantes, des architectures recommandées et des défis à surmonter pour la mise en place d'un écosystème Big Data performant et évolutif. De plus, il permettra d'identifier les tendances émergentes et les innovations susceptibles d'influencer les décisions stratégiques de l'entreprise.

II.2 Étude de Concepts

II.2.1 l'émergence du BIG DATA

Avec la croissance exponentielle des données, le concept de Big Data a émergé, entraînant l'adoption de nouvelles techniques et technologies pour gérer et analyser ces volumes colossaux de données. Les trois caractéristiques principales, connues sous le nom de "3 V" illustré ci dessous, définissent les caractéristiques du Big Data :

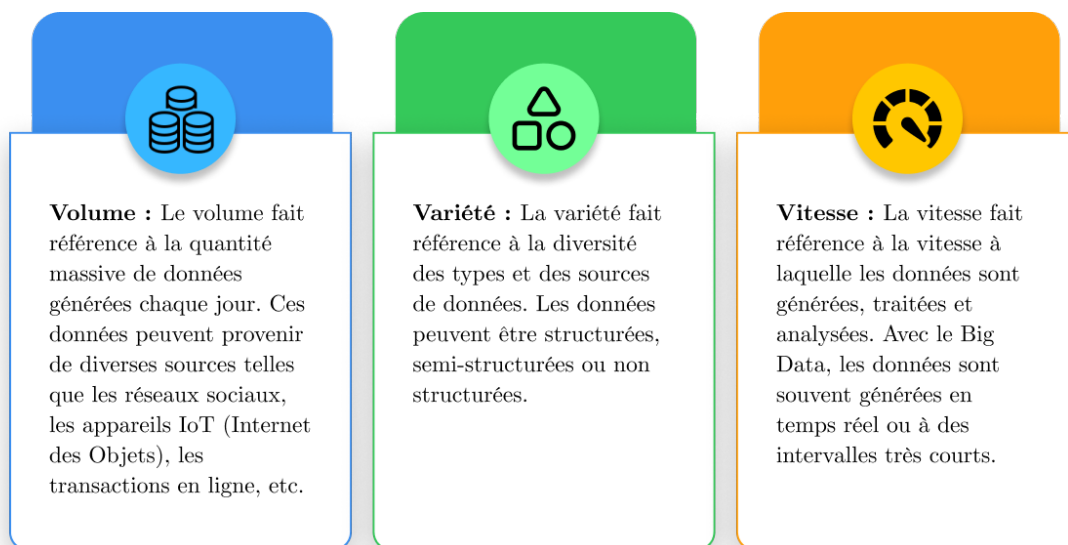


FIG. II.1 : Les Trois piliers du Big Data

En plus des 3 V traditionnels, deux nouveaux V ont émergé pour capturer la complexité accrue du Big Data :

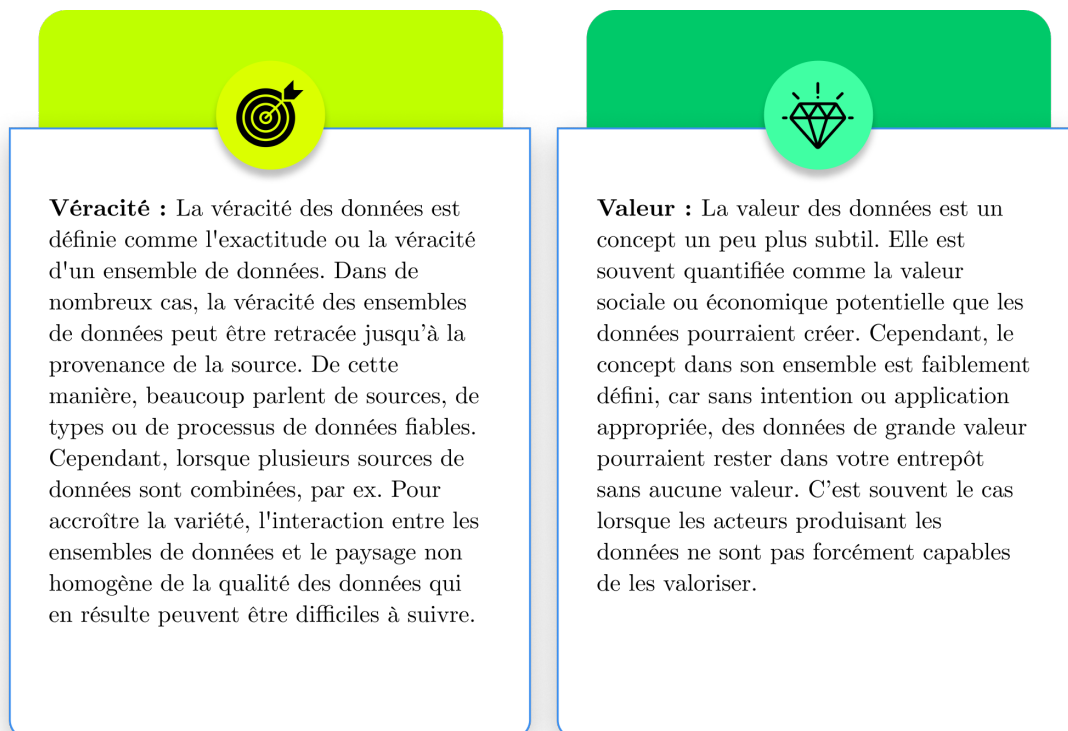


FIG. II.2 : Les deux nouveaux piliers du Big Data

L'intégration de ces cinq dimensions dans la gestion des données aide les entreprises à mieux comprendre l'ère du Big Data, ce qui leur permet de l'exploiter

efficacement pour favoriser leur croissance et renforcer leur position sur le marché. Avec cette transformation numérique, une vague de technologies innovantes a révolutionné la manière dont les données massives sont gérées et analysées, ouvrant ainsi la voie à de nouvelles possibilités et opportunités.

II.2.1.1 Types de données massives

L'ère du Big Data a produit une variété de jeux de données provenant de différentes sources et dans différents domaines. Ces jeux de données sont composés de multiples modalités, chacune ayant une représentation, une distribution, une échelle et une densité différentes. Déverrouiller la puissance de la connaissance à partir de ces jeux de données hétérogènes est d'une importance capitale dans la recherche sur le Big Data, distinguant essentiellement le Big Data des tâches traditionnelles de fouille de données [1].

Données des réseaux en ligne L'un des principaux axes du Big Data est constitué par les données des réseaux sociaux en ligne (OSN). Ces données offrent des insights précieux sur les interactions sociales et les comportements des utilisateurs. Une étude propose une méthode pour évaluer l'intensité et la catégorie des relations sociales en se basant sur leurs interrelations spatio-temporelles [2]. Les résultats montrent que cette méthode peut reconnaître avec succès différentes relations sociales. Avec le développement des réseaux sociaux, de plus en plus de sociologues s'impliquent dans l'étude du Big Data.

Données Mobiles et IoT Une autre tendance dans le Big Data est l'analyse des données mobiles et IoT. Avec le développement de la 5G, les réseaux mobiles ont amélioré les communications machine-à-machine et généré des volumes de données bien plus importants que les anciens réseaux [3]. L'IoT contribue également à cette augmentation massive de données.

Données Géographiques Les données géographiques sont des informations qui incluent des détails sur les emplacements physiques et les caractéristiques géospatiales. Avec le développement rapide des villes numériques 3D, la recherche se concentre désormais sur les services et la maintenance des bases de données géographiques 3D. Un article propose un modèle de base de données spatio-temporelle basé sur des événements, connectant les modèles de villes 3D passés et présents [4]. Il présente également une méthode de renouvellement dynamique utilisant un algorithme de correspondance adaptatif pour mettre à jour les modèles 3D complexes.

Streaming and Real-time data Accompagnée par l'essor des services de streaming en ligne, le Big Data réseau a évolué vers des données spatio-temporelles en

temps réel. Les enquêtes réseau nécessitent une analyse continue des données en raison du renouvellement constant des rapports et des statistiques. Les capteurs sans fil intérieurs (WSN) collectent des données massives dans des environnements industriels complexes, facilitant l'analyse des risques industriels [5]. La circulation urbaine, en constante évolution, bénéficie d'un cadre efficace pour les données en temps réel dans les réseaux de véhicules [6]. Les réseaux sociaux et les capteurs associés permettent une analyse distribuée en cloud, essentielle pour les corrélations sociales dynamiques.

Bien que l'émergence du Big Data offre des opportunités prometteuses, il est évident que les techniques traditionnelles de gestion des données ne seront pas capables de gérer ni l'aspect non structuré des données ni leur flux volumineux. Cela soulève la question cruciale de savoir comment ces données seront traitées efficacement.

II.2.1.2 Modèles de stockage BIG DATA

À l'époque du Big Data, les problèmes les plus difficiles à résoudre concernent la gestion efficace de grandes quantités et variétés de données. Cette section offre un aperçu des technologies de stockage de Big Data qui traitent spécifiquement les défis liés au volume, à la vitesse ou à la variété des données et ne relèvent pas des systèmes de bases de données relationnelles. Les systèmes de stockage Big Data répondent généralement au défi du volume en utilisant des architectures distribuées. Cela permet de répondre aux besoins croissants de stockage en ajoutant de nouveaux nœuds pour fournir une puissance de calcul et stockage. De nouvelles machines peuvent être ajoutées sans interruption à un cluster de stockage, et le système se charge de distribuer les données entre les nœuds de manière transparente. Les solutions de stockage doivent également gérer la vitesse et la variété des données. La vitesse est cruciale en termes de latence des requêtes, c'est-à-dire le temps nécessaire pour obtenir une réponse. Cela est particulièrement important face à des taux élevés de données entrantes. En revanche, la variété concerne l'effort nécessaire pour intégrer et manipuler des données provenant de nombreuses sources différentes. Le tableau ci-dessous présente un aperçu de l'état de l'art des solutions de stockage Big Data :

Catégorie	Description
Systèmes de Fichiers Distribués	Stockage de grandes quantités de données non structurées sur du matériel standard, adapté pour l'ingestion rapide et le traitement en masse.
Bases de Données NoSQL	Technologies cruciales utilisant des modèles de données non relationnels, ne respectant pas nécessairement les propriétés transactionnelles ACID.

Bases de Données NewSQL	Combinaison de la scalabilité des NoSQL avec les garanties transactionnelles ACID des bases de données relationnelles, utilisant SQL pour l'interaction.
Plateformes de Requêtes Big Data	Fournissent des interfaces SQL-like pour simplifier l'accès aux magasins de données sous-jacents, offrant des requêtes rapides et une flexibilité de schéma.
Stockage en Cloud	Offre flexibilité, échelle rapide, coûts réduits, mais introduit des considérations de performance et de sécurité, incluant le stockage d'objets et le stockage en blocs.
Edge Computing	Traite les données près de la source pour réduire la latence et la bande passante, permettant des analyses en temps réel et des réponses plus rapides.
Maillage de Données	Approche décentralisée de la gestion des données, permettant l'accès et le traitement des données de manière uniforme et sécurisée à travers divers environnements distribués.

TAB. II.1 : Big Data storage models

Bien que ces techniques offrent des solutions de stockage prometteuses, il est également crucial que ces systèmes gèrent efficacement l'aspect de la sécurité et de la confidentialité, garantissant ainsi la protection des données sensibles.

II.2.1.3 Sécurité et confidentialité dans le Big Data

En raison de l'ampleur du Big Data, la sécurité et la confidentialité sont des enjeux cruciaux [7]. La CSA Big Data Working Group a identifié les principaux défis de sécurité et de confidentialité pour le stockage du Big Data [8]. Voici un résumé des principaux points :

Meilleures Pratiques de Sécurité pour les Bases de Données NoSQL Les bases de données NoSQL doivent avoir les mêmes pratiques de sécurité que les SGBDR traditionnels, incluant le cryptage des données, l'isolation des processus, la validation des entrées et une authentification forte [9, 10].

Stockage Sécurisé des Données et Journaux de Transactions La distribution des données pose des défis de sécurité, notamment avec l'auto-tiering. Des stratégies doivent être mises en place pour surveiller et enregistrer les mouvements des données afin de prévenir les risques de sécurité [11]. Les schémas de recryptage par proxy peuvent assurer la confidentialité et l'authenticité des données dans un environnement de stockage multi-niveaux [12, 13].

Contrôle d'Accès Cryptographique et Communication Sécurisée Les données doivent être protégées par des contrôles d'accès cryptographiques à la fois en stockage et en transmission. Les clés cryptographiques doivent être générées et stockées par le client. Le cryptage basé sur les attributs est une technologie prometteuse pour intégrer la cryptographie au contrôle d'accès [14, 15, 16, 17].

Confidentialité pour le Contrôle d'Accès Granulaire La diversité des données nécessite des mécanismes de contrôle d'accès granulaire pour assurer la conformité avec les politiques de sécurité et de confidentialité. Les principaux composants du Big Data utilisent Kerberos [18] en conjonction avec une authentification basée sur des jetons et des listes de contrôle d'accès (ACL).

Provenance des Données L'intégrité et l'historique des données sont cruciaux. La provenance des données doit garantir l'intégrité et la confidentialité des métadonnées [19].

Avec des moyens désormais en place pour stocker le Big Data de manière sécurisée, une multitude de nouvelles techniques de traitement révolutionnaires deviennent envisageables.

II.2.2 Technologies Big Data émergentes

Les technologies Big Data émergentes transforment radicalement l'analyse des données et l'intelligence artificielle. Voici une vue d'ensemble des technologies clés qui redéfinissent le traitement de données :

Traitement en Temps Réel Cette technologie permet l'analyse instantanée des données, offrant ainsi la capacité de prendre des décisions éclairées basées sur des informations actualisées. Cette technologie est essentielle pour les applications nécessitant une réactivité immédiate, comme la surveillance en temps réel ou la détection de fraude. Par exemple, des expériences ont été menées pour analyser les données en temps réel avec des performances à faible latence [20].

Les architectures de traitement de flux contribuent à simplifier les tâches de gestion des données nécessaires pour consommer, traiter et publier les données de manière sécurisée et fiable. Le traitement de flux commence par l'ingestion de données à

partir d'un service de publication-abonnement, effectue une action sur ces données, puis publie les résultats de retour vers le service de publication-abonnement ou vers un autre stockage de données. Ces actions peuvent inclure des processus tels que l'analyse, le filtrage, la transformation, la combinaison ou le nettoyage des données.

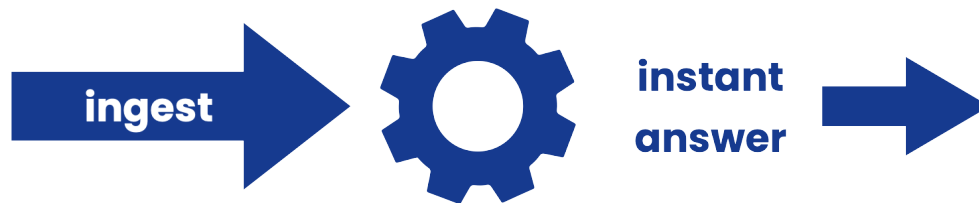


FIG. II.3 : Mode de fonction de traitement en flux

Traitement par Lot Distribué Il s'agit d'une méthode de traitement des données qui divise les tâches en plusieurs lots exécutés sur différents systèmes. Cette approche est idéale pour les opérations complexes et volumineuses qui ne nécessitent pas de traitement en temps réel. Des expériences ont montré son efficacité pour gérer de grandes quantités de données de manière organisée et structurée (Dean & Ghemawat, 2008).

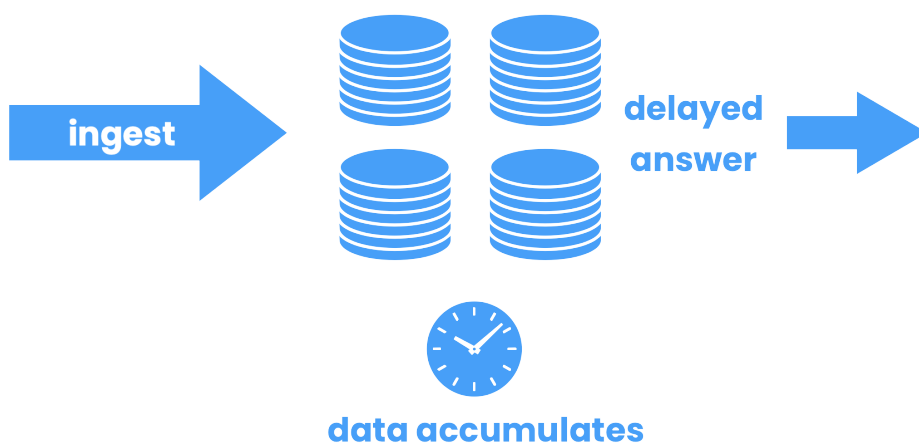


FIG. II.4 : traitement par lot

Apprentissage Automatique Distribué L'apprentissage automatique distribué permet de traiter de grands volumes de données sur plusieurs machines, améliorant ainsi la vitesse et l'efficacité des algorithmes d'apprentissage automatique. Des recherches ont démontré que cette approche optimise les processus d'apprentissage et de prédiction en utilisant des ressources distribuées (Machine Learning (cs.LG) ; Distributed, Parallel, and Cluster Computing (cs.DC) ; Machine Learning (stat.ML)).

IA Générative L'IA générative crée de nouveaux contenus à partir de vastes ensembles de données. Elle a le potentiel de transformer le développement de produits, l'automatisation du travail et l'amélioration de l'expérience client. Des études et expérimentations ont mis en lumière son impact significatif sur l'innovation et l'efficacité opérationnelle (Fiona Fui-Hoon Nah, Ruilin Zheng, Jingyuan Cai, Keng Siau, & Langtao Chen)..

II.3 Approche architectural des écosystèmes BIG Data

II.3.1 Architecture LAMBDA

L'architecture Lambda est une approche innovante pour le traitement des données de grande taille, souvent utilisée dans les écosystèmes de Big Data. Elle se compose de trois couches distinctes : la couche de lots, la couche de vitesse et la couche de service. La couche de lots stocke et traite les données historiques par lots, permettant une analyse approfondie et précise des tendances sur de longues périodes. La couche de vitesse, en revanche, traite les flux de données en temps réel de manière distribuée, offrant des insights en temps réel et une réactivité rapide. Enfin, la couche de service combine les sorties des couches de lots et de vitesse pour fournir des vues interrogeables des données. Cette architecture permet une flexibilité et une adaptabilité considérables, rendant possible le traitement simultané de grandes quantités de données en temps réel et en lots. Cependant, elle peut être complexe à mettre en œuvre et à maintenir, nécessitant une expertise en systèmes distribués et en traitement de flux de données.

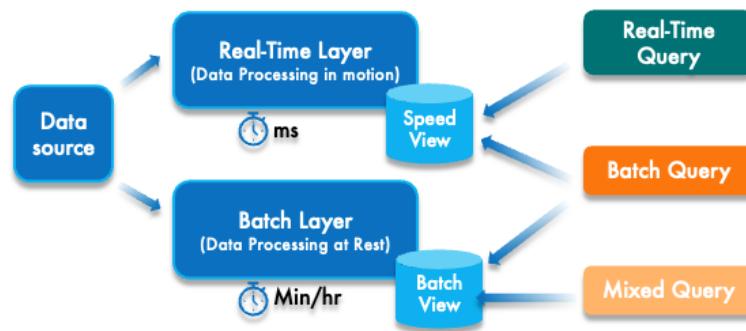


FIG. II.5 : Architecture Lambda

II.3.2 Architecture KAPPA

L'architecture Kappa est une approche simplifiée du traitement des données de grande taille, souvent utilisée dans les écosystèmes de Big Data. Elle se distingue par son utilisation d'un seul système de traitement de flux, traitant toutes les données comme des flux en continu. Cette approche élimine la nécessité d'une couche de lots distincte, comme on en trouve dans l'architecture Lambda, ce qui simplifie le pipeline de données et facilite la maintenance. L'architecture Kappa est particulièrement efficace pour les pipelines de données continus, le traitement des données en temps réel et les systèmes IoT. Cependant, elle nécessite une expertise dans le traitement de flux et les systèmes distribués. En fin de compte, l'architecture Kappa offre une alternative flexible et adaptable pour le traitement des données de grande taille, bien qu'elle puisse ne pas convenir à toutes les applications.

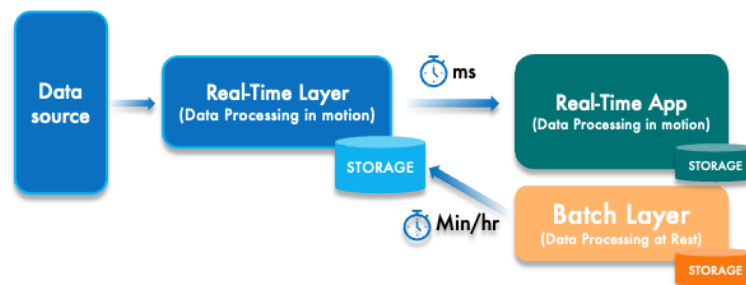


FIG. II.6 : Architecture KAPPA

II.3.3 Lambda VS Kappa

	LAMBDA	KAPPA
Traitement des données	Utilise une approche en couches avec une couche de lots pour les données historiques et une couche de vitesse pour les données en temps réel	traite toutes les données comme des flux en continu, éliminant la nécessité d'une couche de lots distincte
Complexité	peut être plus complexe à mettre en œuvre et à maintenir en raison de la nécessité de gérer deux systèmes de traitement de données distincts (lots et flux)	avec son unique système de traitement de flux, est généralement plus simple à mettre en œuvre et à maintenir
Flexibilité	offre une grande flexibilité en permettant le traitement simultané des données en temps réel et en lots	plus flexible en termes de maintenance et de mise à jour du système de traitement des données
Utilisation	idéale pour les situations où une analyse approfondie des données historiques est nécessaire en plus des insights en temps réel	particulièrement efficace pour les pipelines de données continus et les systèmes IoT

TAB. II.2 : Comparaison de LAMBDA et KAPPA

II.4 Veille Concurrentielle

Entreprise	Type	Présentation générale	Prix de service
Amazon Web Services (AWS)	Cloud	Plateforme cloud complète offrant une large gamme de services Big Data, notamment EC2, S3, Redshift et EMR.	Prix à la consommation en fonction de l'utilisation des ressources. (Très couteuse)
Google Cloud Platform (GCP)	Cloud	Plateforme cloud offrant des services Big Data tels que Dataproc, Cloud Storage et BigQuery.	Prix à la consommation en fonction de l'utilisation des ressources. (Très couteuse)
Microsoft Azure	Cloud	Plateforme cloud offrant des services Big Data tels que HDInsight, Blob Storage et Data Lake Storage Gen2.	Prix à la consommation en fonction de l'utilisation des ressources. (Très couteuse)
Cloudera	On-premises et hybride	Plateforme Big Data open source populaire offrant une suite complète de produits pour le déploiement et la gestion de Big Data sur site ou dans le cloud.	Modèles de licence flexibles, y compris les abonnements et les licences perpétuelles. (Très couteuse)
Databricks	Cloud	Plateforme d'analyse de données basée sur le cloud qui fournit un environnement unifié pour le développement, le déploiement et l'exécution d'applications Big Data.	Modèles de licence basés sur le nombre d'utilisateurs et de workloads. (Très couteuse)
Hortonworks	On-premises et hybride	Plateforme Big Data open source populaire offrant une suite complète de produits pour le déploiement et la gestion de Big Data sur site ou dans le cloud.	Modèles de licence flexibles, y compris les abonnements et les licences perpétuelles. (Très couteuse)

HPE	On-premises	Solutions matérielles et logicielles intégrées pour le déploiement et la gestion de Big Data sur site.	Prix sur devis en fonction de la configuration du système. (Très coûteuse)
Dell	On-premises	Solutions matérielles et logicielles intégrées pour le déploiement et la gestion de Big Data sur site.	Prix sur devis en fonction de la configuration du système. (Très coûteuse)

TAB. II.3 : Fournisseurs de services Big Data

II.5 Veille Technologique

Cas d'utilisation	Technologie	Avantages	Inconvénients
Virtualization	Docker	Composition, création, déploiement, et échelonnage de conteneurs sur les hôtes docker.	La gestion des conteneurs.
	VSphere	Gestion de plusieurs serveurs, Déplacement à chaud des VM	Payant, Virtualisation des ressources et de logiciel
Traitement de données	Apache Spark	Vitesse de traitement élevée, Prise en charge de divers langages de programmation, Large éventail de bibliothèques d'analyse de données	Nécessite une certaine expertise en programmation, Plus gourmand en ressources que les frameworks de traitement de données traditionnels
	Apache Flink	Faible latence et traitement en temps réel, Prise en charge de divers langages de programmation, Large éventail de connecteurs et d'intégrations	Plus complexe à configurer et à gérer que les frameworks de traitement par lots, Nécessite une expertise en Big Data
Bases de données	MongoDB	Flexibilité du schéma, Requêtes puissantes basées sur des documents, Facilité d'utilisation	Moins performant pour les analyses complexes, Nécessite une expertise en NoSQL
	Cassandra	Distribution sur plusieurs nœuds, Accès en lecture et en écriture à partir de n'importe quel nœud, Évolutivité horizontale facile	Complexité de configuration et de gestion, Nécessite une expertise en modélisation de données
	Redis	Performances de lecture et d'écriture ultra-rapides, Idéal pour les caches, les files d'attente de messages et les compteurs	Données volatiles en mémoire (perte de données au redémarrage), Capacité de stockage limitée

Ingestion de données	Apache Kafka	Messagerie de flux de données hautement performante, Prise en charge de divers cas d'utilisation de traitement de données, Intégration facile avec d'autres technologies Big Data	Nécessite une certaine expertise en configuration et en gestion, Plus complexe que les systèmes de messagerie traditionnels
	Apache Spark Streaming, Apache Flink	Faible latence et traitement en temps réel, Prise en charge de divers langages de programmation, Large éventail de connecteurs et d'intégrations	Plus complexes à configurer et à gérer que les systèmes de messagerie de streaming (Kafka), Nécessitent une expertise en Big Data
Moteurs de recherche	Elasticsearch, Solr	Moteurs de recherche open source performants et scalables, Prise en charge de la recherche en texte intégral et facettée, Communauté active et support important	Nécessite une certaine expertise pour la configuration et la gestion, Peut être coûteux en ressources pour de grandes volumétries de données
Monitoring	Kibana	Interface utilisateur intuitive et personnalisable, Large éventail de visualisations et de tableaux de bord, Intégration native avec Elasticsearch	Nécessite Elasticsearch pour fonctionner, Moins flexible pour les analyses complexes que Grafana
	Grafana	Prise en charge de diverses sources de données (bases de données, API, etc.), Large éventail de panneaux de contrôle et de graphiques personnalisables, Fonctionnalités d'alerte et de notification	Courbe d'apprentissage plus élevée que Kibana, Moins intuitif pour les utilisateurs débutants

II.6 Conclusion

Bien que l'émergence du Big Data ait favorisé l'apparition de systèmes distribués et de traitements en temps réel, il est essentiel d'assurer une cohérence entre les différentes plateformes de stockage et de traitement des données. Les différents types de données étant stockés et traités de manière distincte en fonction des tâches, un écosystème Big Data capable de gérer efficacement ce flux est indispensable. Cependant, les écosystèmes préexistants sont souvent coûteux, ce qui ajoute une couche supplémentaire de complexité et de dépenses pour les organisations cherchant à évoluer dans ce domaine. Ces systèmes, générant de grandes quantités de données, bénéficieraient grandement de l'adoption d'un tel écosystème pour optimiser leur fonctionnement et assurer une gestion fluide et cohérente des données.

III Analyse et Conception

SOMMAIRE

INTRODUCTION	34
III.1 ANALYSE DES BESOINS	34
III.1.1 L'importance de l'écosystème pour l'entreprise	34
III.1.2 Fonctionnement de notre écosystème avec EcoDID	35
III.1.3 Besoins fonctionnels	36
III.1.4 Besoins non fonctionnels	37
III.2 ÉTUDE CONCEPTUELLE	39
III.2.1 Diagramme de cas d'utilisation	39
III.2.2 Conception de l'architecture	40
III.3 ARCHITECTURE GLOBALE	47
III.3.1 Hadoop HDFS Cluster	50
III.3.2 Interactions entre les Composants	51
III.3.3 Flux de Données	52
III.3.4 Stratégie de mise en œuvre	53
III.3.5 Mode de déploiement	56

DANS ce chapitre, nous présentons l'analyse et la conception d'un écosystème Big Data. Nous décrirons l'architecture proposée, les technologies utilisées, et les stratégies de mise en œuvre. L'objectif est de créer une infrastructure capable de collecter, traiter, stocker et analyser des volumes importants de données provenant de diverses sources, tout en assurant performance, sécurité et évolutivité.

Introduction

Dans ce chapitre, nous aborderons l'étude conceptuelle de la conception et du déploiement d'un écosystème Big Data adapté à des usages multiples. L'objectif est de présenter une architecture détaillée qui répond aux besoins variés de traitement et d'analyse des données provenant de différentes sources. Nous décrirons les composants essentiels de cette architecture, les technologies utilisées, ainsi que les stratégies de mise en œuvre et d'intégration.

La conception conceptuelle est une étape cruciale dans le déploiement d'un écosystème Big Data. Elle permet de définir une architecture robuste et évolutive, capable de gérer efficacement les volumes massifs de données et de répondre aux exigences de performance, de sécurité et de fiabilité. Une bonne conception conceptuelle assure également une intégration harmonieuse des différents composants et facilite la maintenance et l'évolution de l'écosystème à long terme.

III.1 Analyse des besoins

III.1.1 L'importance de l'écosystème pour l'entreprise

L'entreprise a développé l'écosystème EcoDID pour répondre à plusieurs besoins stratégiques et opérationnels :

Gestion des données : Avec la croissance exponentielle des données provenant de diverses sources, l'écosystème EcoDID offre une solution centralisée pour collecter, stocker, traiter et analyser ces données de manière efficace.

Prise de décision éclairée : En utilisant les technologies avancées telles que l'intelligence artificielle et le Big Data, EcoDID permet à l'entreprise d'extraire des insights précieux à partir des données en temps réel, facilitant ainsi une prise de décision rapide et éclairée.

Optimisation des processus métier : En automatisant les tâches répétitives et en identifiant les inefficacités grâce à une analyse approfondie des données, EcoDID contribue à l'optimisation des processus métier, améliorant ainsi l'efficacité opérationnelle globale.

Adaptabilité et innovation : En restant à l'avant-garde des avancées technologiques, l'entreprise peut utiliser EcoDID pour s'adapter rapidement aux évolutions du marché et développer de nouvelles solutions innovantes pour répondre aux besoins évolutifs des clients.

l'écosystème EcoDID est un pilier essentiel pour soutenir la croissance et la compétitivité de l'entreprise, en lui permettant de maximiser la valeur de ses données, de prendre des décisions éclairées et d'innover dans un environnement numérique en

constante évolution.

III.1.2 Fonctionnement de notre écosystème avec EcoDID

L'intégration de l'écosystème avec EcoDID s'opérera de manière fluide et complémentaire, s'appuyant sur les fonctionnalités spécifiques de chaque composant pour assurer un environnement de traitement, de stockage et de sécurisation des données performant et cohérent.

Transfert de journal sécurisé : Mettre en place un mécanisme de transfert de journal sécurisé, utilisant des protocoles de chiffrement avancés pour garantir l'intégrité et la confidentialité des données tout au long du processus de transfert. Cela permettra de sécuriser efficacement les journaux d'événements essentiels pour l'analyse et la prise de décision.

Intégration et déploiement des algorithmes d'apprentissage automatique : Permettre l'intégration et le déploiement des algorithmes d'apprentissage automatique, permettant une intégration harmonieuse dans l'écosystème existant. Cette fonctionnalité garantira une utilisation optimale des ressources informatiques et une mise en œuvre efficace des modèles d'apprentissage.

Traitement en temps réel et par lots : L'écosystème prend en charge le traitement des données en temps réel et par lots pour répondre aux différents besoins de la plateforme. De plus, il permet l'inférence d'algorithmes d'apprentissage automatique en temps réel pour obtenir des informations opportunes, tout en offrant également la possibilité de traiter de grands ensembles de données par lots pour une analyse approfondie.

Stockage efficace des données : mettre en œuvre des mécanismes de stockage de données efficaces, incluant la partition, la compression et l'indexation des données. Ces fonctionnalités garantiront une gestion efficace des données massives, tout en optimisant les performances et la rentabilité du stockage.

Accès rapide et sécurisé aux données : assurer un accès rapide et sécurisé aux données, en mettant en place des mécanismes de sécurité avancés conformes au modèle CIA (Confidentialité, Intégrité et Disponibilité). Cette approche garantira une disponibilité continue des données tout en protégeant leur confidentialité et leur intégrité.

Interface conviviale : Développer d'une interface utilisateur conviviale, conçue pour faciliter l'interaction entre les équipes de développement et d'apprentissage automatique. Une conception intuitive favorisera une adoption rapide par les utilisateurs et une collaboration efficace entre les différentes parties prenantes.

L'écosystème de Big Data sera intégré à la plateforme EcoDID et interagira étroitement avec ces fonctionnalités clés pour offrir un environnement de traitement des données performant et sécurisé, permettant ainsi à l'entreprise de maximiser

l'utilisation de ses ressources et de ses capacités d'analyse pour prendre des décisions stratégiques éclairées.

III.1.3 Besoins fonctionnels

L'écosystème Big Data doit répondre à plusieurs besoins fonctionnels afin de prendre en charge efficacement les différents cas d'utilisation. Ces besoins incluent, sans s'y limiter :

- **Collecte et ingestion des données** : Capacité à intégrer des données provenant de multiples sources, y compris des flux en temps réel, des bases de données traditionnelles, des fichiers journaux, et des API externes.
- **Stockage des données** : Gestion de vastes volumes de données structurées, semi-structurées et non structurées, avec des mécanismes de stockage évolutifs comme les data lakes et les entrepôts de données.
- **Traitement des données** : Support pour le traitement des données en temps réel et par lots, permettant des analyses rapides et des transformations de données complexes.
- **Analyse et exploration des données** : Outils et plateformes pour l'analyse des données, y compris les tableaux de bord interactifs, les outils de visualisation et les bibliothèques de machine learning.
- **Sécurité et conformité** : Mise en place de mécanismes de sécurité robustes pour protéger les données sensibles, assurer la conformité avec les réglementations (comme le RGPD), et gérer les accès aux données.
- **Scalabilité et performance** : Capacité à évoluer horizontalement pour gérer des charges de travail croissantes et assurer des performances élevées même avec des volumes de données massifs.
- **Analyse en temps réel** : L'ingestion rapide des données en continu depuis des sources variées, telles que des dispositifs IoT, des systèmes de gestion, et des applications web. Il est crucial de pouvoir traiter ces flux de données en temps réel pour permettre des actions immédiates basées sur les analyses effectuées. Cela nécessite une architecture capable de gérer efficacement des volumes de données importants tout en assurant une faible latence et une haute disponibilité, afin de fournir des résultats pertinents et exploitables instantanément.
- **Traitement par lots** : La capacité à gérer et traiter de grandes quantités de données historiques de manière efficace. Cela inclut la planification et l'orchestration des tâches de traitement par lots, souvent effectuées en dehors des

heures de pointe pour optimiser l'utilisation des ressources. Ces tâches nécessitent des systèmes robustes et performants pour assurer que les données historiques soient accessibles et analysables rapidement, permettant ainsi de tirer des insights précieux pour l'entreprise.

- **Apprentissage automatique** : Prendre en charge les tâches de machine learning en incluant des processus automatisés pour la préparation et le nettoyage des données, rendant ainsi les données aptes à l'entraînement des modèles. De plus, faciliter l'entraînement, l'évaluation et le déploiement de ces modèles sur de larges jeux de données. Avec des ressources informatiques suffisantes pour gérer les lourdes charges de travail associées à l'apprentissage automatique, cela garantira des modèles performants et fiables, tout en simplifiant l'utilisation et le déploiement des tâches de machine learning.

III.1.4 Besoins non fonctionnels

Pour garantir le bon fonctionnement de l'écosystème Big Data, il est essentiel de prendre en compte plusieurs besoins non fonctionnels :

- **Scalabilité** : La scalabilité horizontale et la scalabilité verticale, la première implique l'ajout de nœuds au cluster pour gérer des volumes de données ou des charges de travail croissants sans perte de performance, tandis que la seconde consiste à améliorer les ressources des nœuds existants (comme la mémoire ou le CPU) pour optimiser les performances. Ces approches combinées permettent de concevoir des systèmes capables de s'adapter efficacement à des environnements évolutifs tout en maintenant des niveaux de performance élevés.
- **Fiabilité (la haute disponibilité et la tolérance aux pannes)** : La haute disponibilité vise à garantir que les services demeurent accessibles en cas de défaillance d'un ou plusieurs composants. Cela implique la mise en place de clusters redondants et de stratégies de basculement (failover) pour assurer une continuité de service sans interruption notable. D'autre part, la tolérance aux pannes se concentre sur la capacité de l'écosystème à maintenir un fonctionnement normal malgré des défaillances partielles.
- **Sécurité** : Se base sur le contrôle d'accès strict pour prévenir les accès non autorisés, l'utilisation du chiffrement pour protéger les données en transit et au repos, ainsi que l'audit et la surveillance pour suivre les accès et les modifications, garantissant ainsi la confidentialité, l'intégrité et la conformité aux normes de sécurité.
- **Performances** : Réduire la latence pour les applications en temps réel, optimiser le débit pour les traitements par lots, accélérer les analyses avec des

requêtes optimisées, et gérer efficacement les ressources pour éviter les goulots d'étranglement et garantir une utilisation optimale des écosystèmes.

En définissant clairement ces besoins fonctionnels et non fonctionnels, nous posons les bases pour une conception solide et performante de l'écosystème Big Data, capable de répondre aux exigences des différents cas d'utilisation. De plus, il sera aussi sécurisée et conforme aux régulations en vigueur, garantissant ainsi la fiabilité et la légitimité de l'exploitation des données.

III.2 Étude conceptuelle

III.2.1 Diagramme de cas d'utilisation

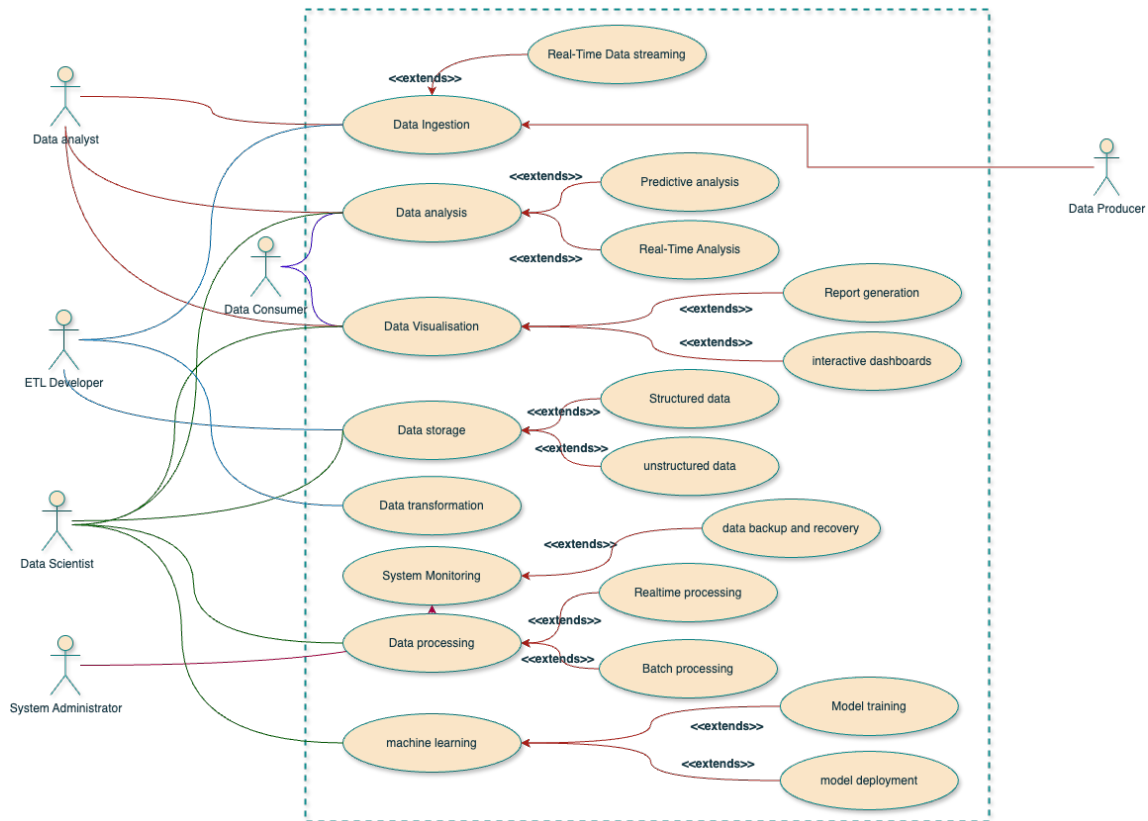


FIG. III.1 : Diagramme de cas d'utilisation de l'écosystème Big Data

Le diagramme de cas d'utilisation présente les interactions entre les différents acteurs et les fonctions principales de l'écosystème Big Data. Ce diagramme aide à comprendre comment chaque rôle utilise le système pour accomplir des tâches spécifiques, et met en lumière les relations entre les fonctions critiques de l'architecture.

III.2.2 Conception de l'architecture

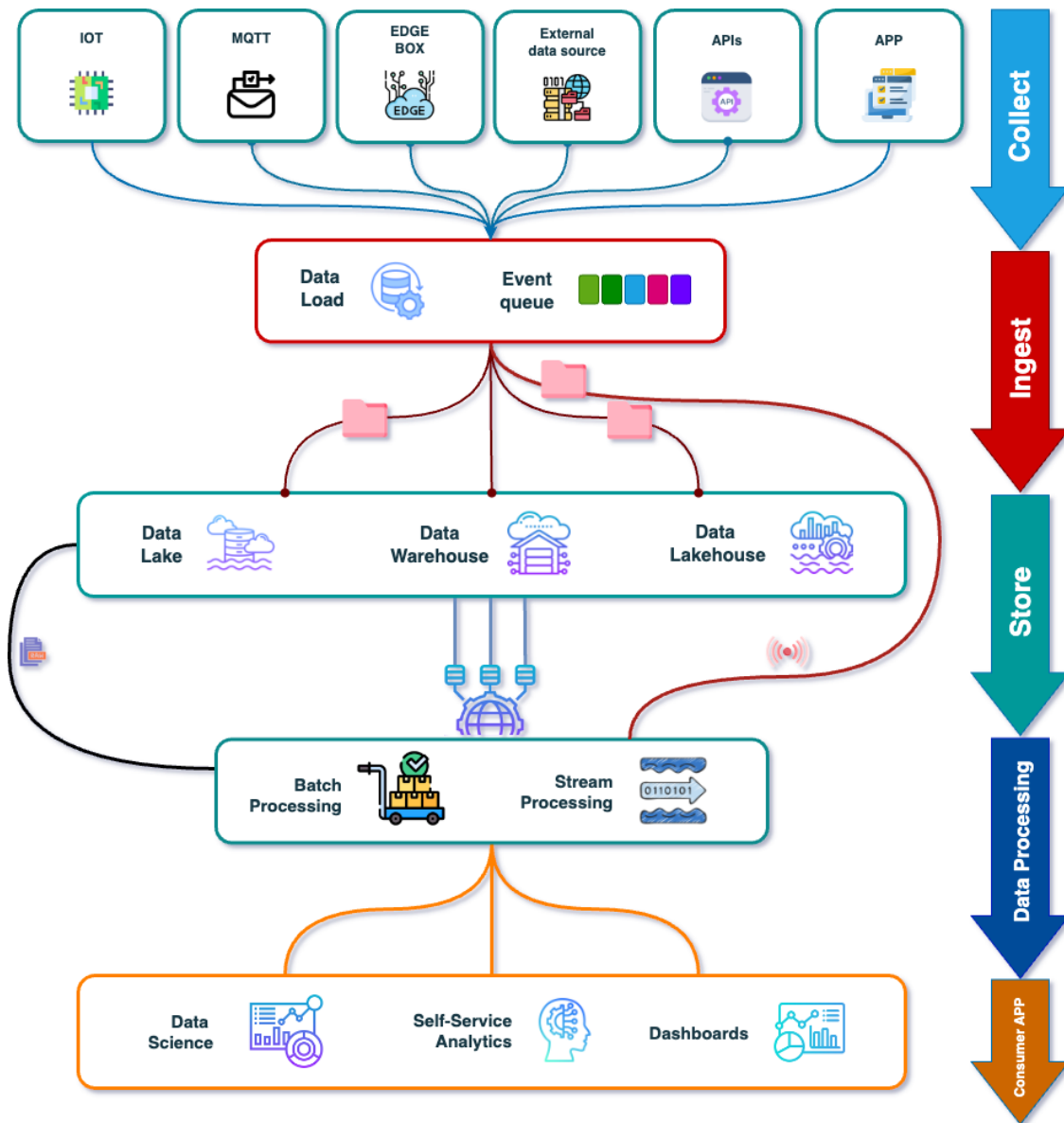


FIG. III.2 : Architecture globale de l'écosystème solution

III.2.2.1 Vue d'ensemble de l'architecture

L'architecture big data présentée est conçue pour gérer efficacement de grands volumes de données provenant de diverses sources. Elle est structurée en plusieurs couches distinctes, chacune ayant des responsabilités spécifiques allant de l'ingestion des données à leur traitement et stockage, jusqu'à la consommation finale par les applications utilisateur. Les principaux composants de cette architecture incluent les sources de données, la couche d'ingestion, le data pipeline, le traitement des, le

stockage et l'accès aux données. Chaque composant est interconnecté de manière à assurer un flux de données fluide et cohérent du début à la fin. Les données sont d'abord collectées à partir des sources, puis ingérées par des outils spécialisés, transférées et transformées dans le pipeline de données, traitées en temps réel ou par lots, stockées dans des systèmes adaptés, et finalement consommées par des applications analytiques ou d'apprentissage automatique.

III.2.2.2 Les différents couches de l'architecture et leurs rôles

Sources de données

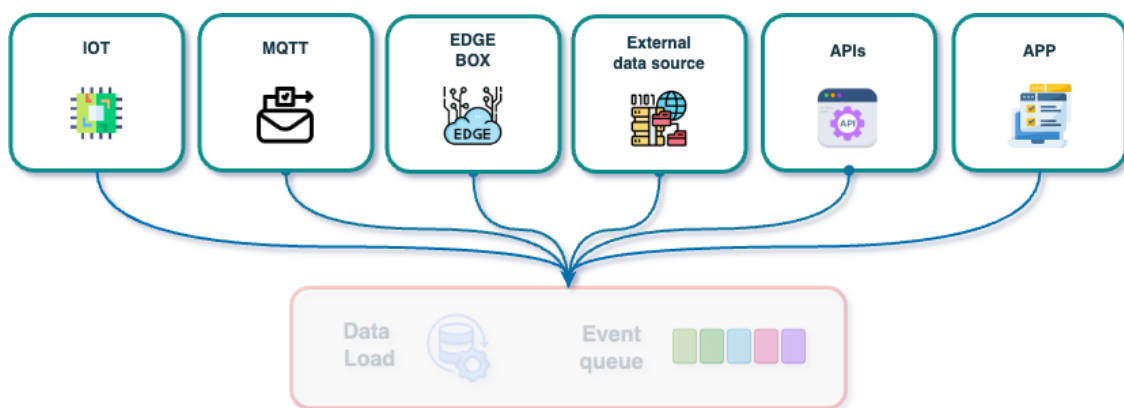


FIG. III.3 : Les sources des données

Les sources de données sont à la fois internes et externes. Les sources internes comprennent les dispositifs IoT qui collectent des données en temps réel à partir de capteurs et d'équipements, les applications qui génèrent des données d'utilisation et de transaction, et les systèmes internes contenant des informations critiques pour l'entreprise. Les sources externes incluent des API fournissant des données de services tiers et des bases de données externes offrant des données complémentaires importantes pour les analyses.

L'ingestion de données

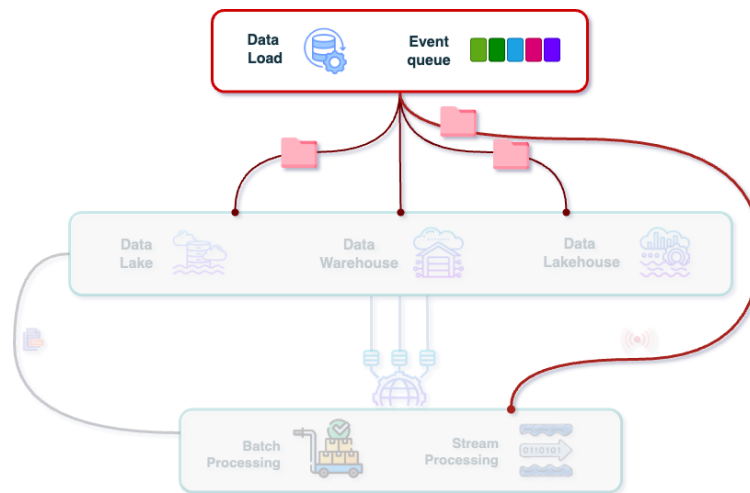


FIG. III.4 : Couche d'ingestion des données

La couche d'ingestion de données joue un rôle crucial en collectant les données provenant des différentes sources de manière fiable et efficace. Elle utilise des technologies comme Apache Kafka pour la gestion des files d'attente, la gestion des données en temps réel, et leur chargement dans les systèmes de stockage. Cette couche garantit la gestion des pics de données, la livraison fiable des données, et peut effectuer des transformations légères si nécessaire.

Stockage des données

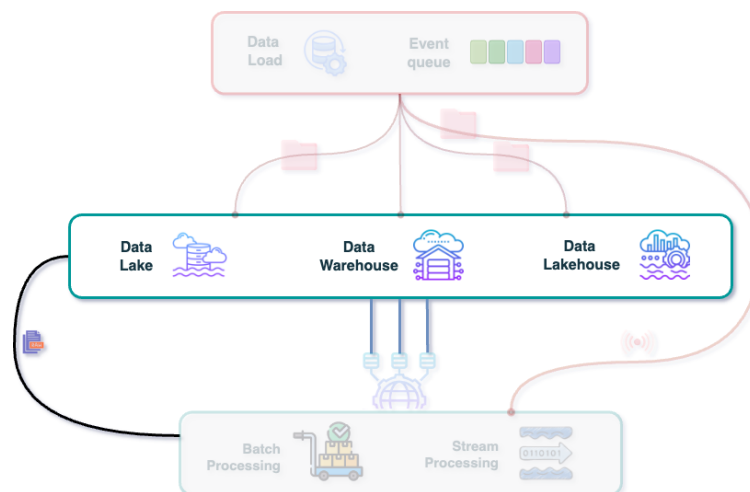


FIG. III.5 : Couche de stockage des données

Le stockage des données est organisé en deux types principaux : le data lake et le data warehouse. Le data lake permet de stocker les données brutes dans leur format natif de manière flexible, en utilisant des outils comme Hadoop HDFS pour

le stockage distribué . Le data warehouse, en revanche, est conçu pour stocker les données de manière structurée, optimisée pour des requêtes rapides et efficaces, avec des outils comme Apache Hive.

Traitement des données

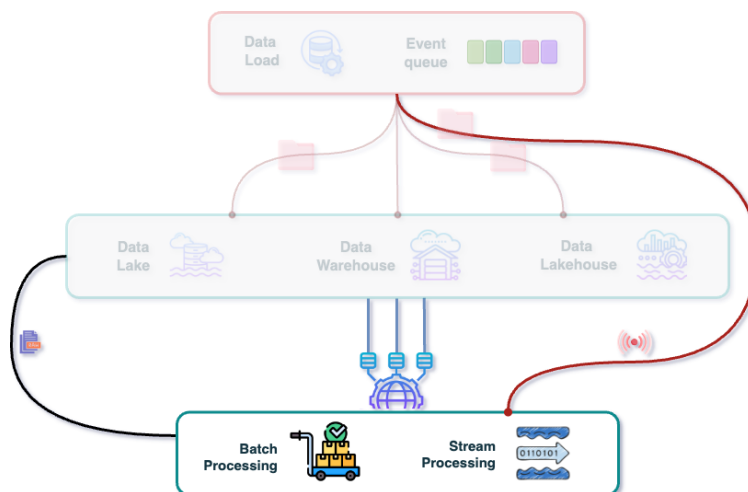


FIG. III.6 : Couche de traitement des données

La couche de traitement des données se divise en deux catégories principales : le traitement en temps réel et le traitement par lots. Le traitement en temps réel utilise des outils comme Apache Flink et Spark streaming pour l'analyse continue des flux de données, permettant des prises de décision instantanées. Le traitement par lots, quant à lui, utilise des outils comme Apache Hadoop et Apache Spark pour traiter de grandes quantités de données accumulées sur une période, souvent pour des analyses complexes nécessitant un contexte historique.

L'accès et la consommation des données

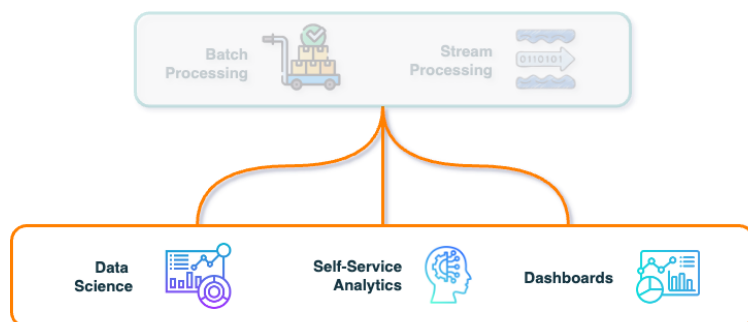


FIG. III.7 : Couche de consommation de données

l'accès et la consommation des données sont facilités par différentes méthodes et applications. Les données peuvent être accessibles via des API RESTful ou GraphQL pour un accès programmatique, ou par des requêtes SQL pour extraire et analyser les données. Les applications consommatrices incluent des outils de business intelligence (BI) comme Tableau et Power BI pour la visualisation et les analyses interactives, ainsi que des modèles d'apprentissage automatique utilisant des frameworks comme TensorFlow et PyTorch pour la création de modèles prédictifs. La sécurité et le contrôle d'accès sont assurés par des mécanismes d'authentification, d'autorisation, et de gestion des rôles, garantissant que seules les personnes autorisées peuvent accéder aux données sensibles.

III.2.2.3 Modèles conceptuels et diagrammes

Architecture Technologique

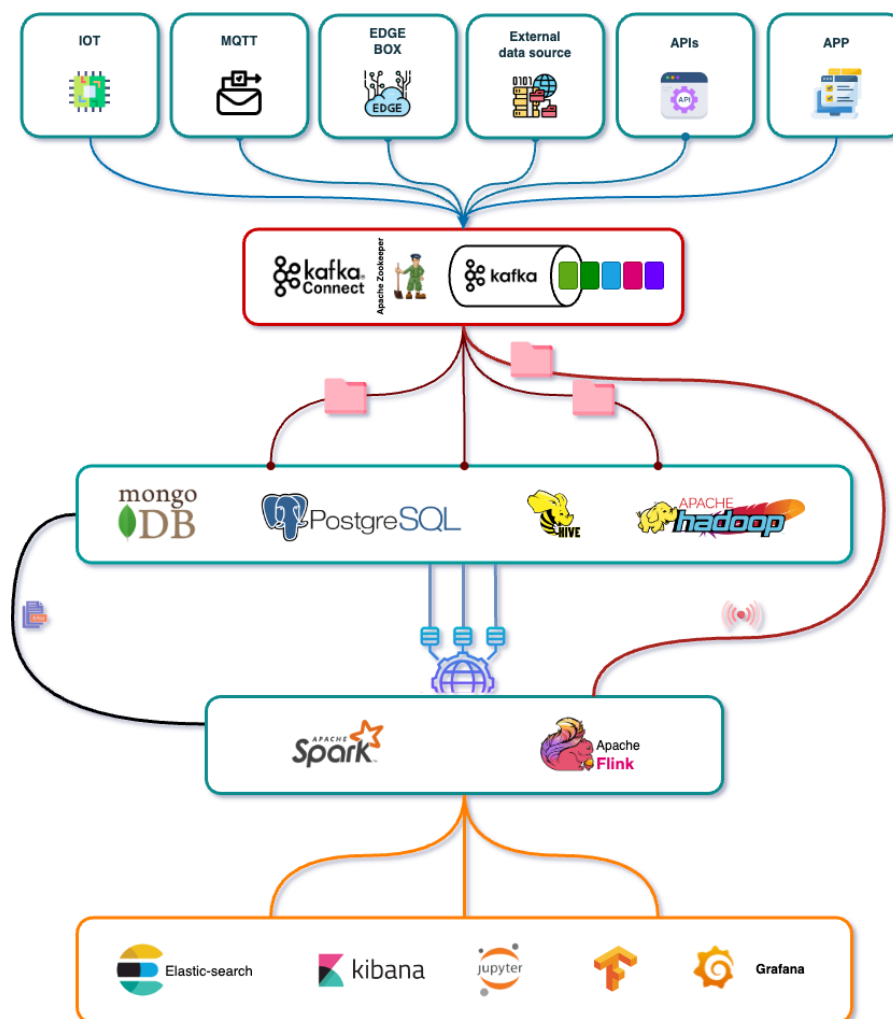


FIG. III.8 : Représentation visuelle de l'architecture Technologique

III.2.2.4 Sélection de la pile technologique

Critères de sélection

Pour choisir les technologies appropriées pour notre écosystème Big Data, nous avons considéré plusieurs critères essentiels.

En premier lieu, la **scalabilité** est cruciale, car les technologies choisies doivent pouvoir évoluer facilement pour gérer l'augmentation du volume de données et le nombre d'utilisateurs sans compromettre les performances. Ensuite, nous avons évalué le **support communautaire et commercial** des technologies, en tenant compte de la disponibilité d'une communauté active pour l'assistance et les mises à jour régulières, ainsi que la possibilité de support commercial pour les entreprises nécessitant des garanties de service.

Le **coût** global de possession, incluant les coûts de licence, de maintenance, d'écosystème, et d'exploitation, est également un facteur déterminant, tout comme les options open-source par rapport aux solutions commerciales. Nous avons également pris en compte la **performance** des technologies, en évaluant leur efficacité pour le traitement des données en temps réel et en mode batch, ainsi que leur latence et leur débit.

L'**intégration** est un autre critère essentiel, puisqu'elle détermine la capacité des technologies à fonctionner harmonieusement avec d'autres composants de l'architecture existante ou prévue. De plus, les **fonctionnalités de sécurité** intégrées, telles que l'authentification, l'autorisation, et le chiffrement des données en transit et au repos, sont indispensables pour protéger les données sensibles.

La **fiabilité et la tolérance aux pannes** des technologies ont été évaluées pour assurer une haute disponibilité des services et la robustesse face aux pannes. La gestion et l'administration ont été prises en compte pour évaluer le déploiement, la configuration, et la surveillance des technologies.

Nous avons également examiné la **compatibilité et le support multi-plateformes**, afin de garantir que les technologies puissent fonctionner sur différentes plateformes et environnements, que ce soit en cloud, on-premise ou hybride. L'**écosystème et l'intégration avec les outils de Data Science** ont été pris en compte, en s'assurant de la compatibilité avec les outils de data science et de machine learning, ainsi que les outils de visualisation des données.

Enfin, la **flexibilité et la modélisation des données** ont été évaluées, en considérant la capacité des technologies à gérer différents types de données (structurées, semi-structurées, non structurées) et à s'adapter aux besoins changeants. La **capacité d'évolution technique** a également été jugée, pour s'assurer que les technologies puissent adopter de nouvelles fonctionnalités et évoluer avec les avancées technologiques.

Comparaison des technologies

Ce basant sur la comparaison déjà effectuée dans le chapitre de l'état de l'art et le tableau de veille technologique, nous avons choisi les technologies qui répondent le mieux à nos besoins, en prenant en compte leurs performances, leur scalabilité, et leur capacité à s'intégrer efficacement dans notre écosystème.

Choix technologiques finaux

Les technologies choisies pour notre écosystème Big Data sont Kafka Connect, Apache Kafka, Apache Flink, Apache Spark, MongoDB, PostgreSQL, HDFS, Hive, Elasticsearch, Kibana, et Grafana. Chaque choix a été soigneusement justifié par une analyse approfondie de leurs capacités à répondre à nos exigences spécifiques en matière de scalabilité, de performance, d'intégration, de sécurité, et de coût.

Kafka Connect et Apache Kafka ont été sélectionnés pour leur robustesse dans le traitement des flux de données en temps réel. Kafka, en particulier, est reconnu pour sa faible latence et sa capacité à traiter des millions de messages par seconde, ce qui est essentiel pour les applications nécessitant des mises à jour instantanées et continues. Kafka Connect simplifie l'intégration de diverses sources de données avec Kafka, assurant une ingestion fluide et efficace des données.

Apache Flink et Apache Spark sont deux piliers de notre architecture de traitement de données. Apache Flink a été choisi pour ses capacités de traitement en temps réel et par lots, offrant une flexibilité dans la gestion des flux de données complexes avec une faible latence. Apache Spark, de son côté, est une solution éprouvée pour le traitement en masse de grandes quantités de données, permettant des analyses rapides et distribuées grâce à son modèle de traitement en mémoire.

Pour le **stockage de données**, nous avons opté pour **MongoDB** et **PostgreSQL** en raison de leurs caractéristiques complémentaires. MongoDB, une base de données NoSQL, est idéale pour stocker des données semi-structurées et non structurées, offrant une grande flexibilité et une évolutivité horizontale. PostgreSQL, une base de données relationnelle robuste, assure la gestion efficace des données structurées avec des fonctionnalités avancées de requête et de transaction.

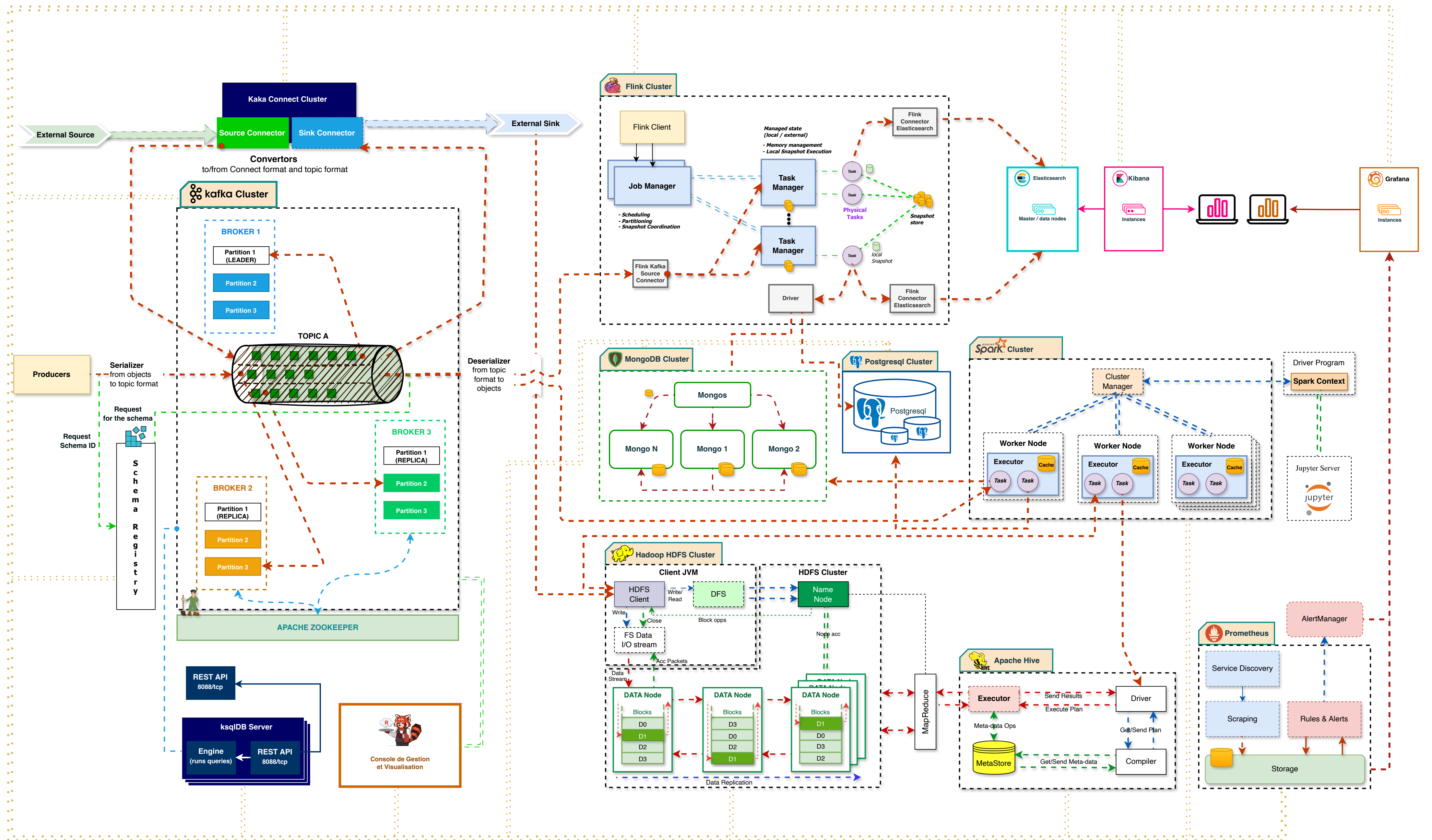
HDFS (Hadoop Distributed File System) et **Hive** ont été retenus pour leur efficacité dans la gestion des grandes quantités de données. HDFS fournit un stockage distribué fiable et économique, tandis que Hive facilite l'analyse de données en grands volumes grâce à son interface de type SQL, permettant une intégration aisée avec des outils d'analyse de données.

Pour la **visualisation et la surveillance des données**, **Elasticsearch**, **Kibana**, et **Grafana** forment un trio puissant. Elasticsearch permet une recherche et une analyse rapide des données, Kibana offre des capacités de visualisation avancées,

et Grafana fournit des tableaux de bord interactifs pour la surveillance en temps réel de l'écosystème. Ensemble, ces outils permettent de transformer les données en informations exploitables, essentielles pour la prise de décision.

En résumé, le choix de ces technologies est le fruit d'une évaluation rigoureuse basée sur leur capacité à répondre à nos besoins en termes de performance, de fiabilité, de scalabilité et de coût. Ces technologies ne se contentent pas de répondre aux exigences actuelles de notre écosystème Big Data, mais elles sont également prêtes à évoluer avec les avancées technologiques futures, garantissant ainsi un écosystème durable et performante.

III.3 Architecture Globale



Notre architecture globale démontre l'intégration de technologies avancées pour créer un pipeline de données robuste et évolutif. Elle couvre les étapes de collecte, de traitement, de stockage et d'analyse des données, assurant une gestion efficace et résiliente des flux de données. Voici une analyse détaillée des composants, de leurs interactions et du flux de données :

Kafka Connect Cluster

- **Source Connector et Sink Connector** : Les connecteurs Kafka Connect assurent l'intégration des sources de données externes et des cibles, permettant l'importation et l'exportation de données vers et depuis Kafka [21].
- **Convertisseurs** : Ces modules convertissent les formats de données entre les sources/sinks externes et le format des topics Kafka, facilitant ainsi une intégration transparente.

Kafka Cluster

- **Producers** : Les producteurs envoient des données aux topics Kafka, où elles sont partitionnées pour optimiser la gestion et la tolérance aux pannes [22].
- **Topics** : Les topics segmentent les données en partitions, réparties entre différents brokers pour garantir la scalabilité et la haute disponibilité.
- **Brokers** : Les brokers gèrent les partitions des topics, chaque partition ayant un broker leader et plusieurs réplicas pour assurer la redondance.
- **Schema Registry** : Le registre des schémas stocke les schémas des données, assurant la compatibilité des données sérialisées et la gestion des versions de schéma.

Flink Cluster

- **Flink Client** : Soumet les jobs au cluster Flink pour le traitement en temps réel -[23].
- **Job Manager** : Gère l'exécution des jobs, incluant la planification, la coordination des snapshots et la gestion des ressources.
- **Task Managers** : Exécutent les tâches assignées par le Job Manager, gérant l'état local ou externe des tâches et assurant la coordination des snapshots et la gestion de la mémoire.

Elasticsearch et Kibana

- **Elasticsearch** : Utilisé pour le stockage et l'indexation des données, permettant des recherches et analyses complexes. Les données peuvent être ingérées depuis Kafka via les connecteurs Flink [24].
- **Kibana** : Fournit une interface de visualisation pour Elasticsearch, permettant la création de tableaux de bord et de visualisations interactives des données [25].

Grafana

- **Instances** : Grafana est utilisé pour la surveillance et la visualisation des métriques de performance du système, souvent intégré avec Prometheus pour la collecte de métriques [26].

MongoDB Cluster

- **Mongos** : Distribue les requêtes aux différentes instances MongoDB [27].
- **Instances MongoDB** : Stockent les données de manière distribuée pour assurer la scalabilité et la haute disponibilité, pouvant être alimentées directement depuis Kafka ou via Flink.

PostgreSQL Cluster

- **PostgreSQL** : Base de données relationnelle utilisée pour le stockage structuré des données, alimentée par les Sink Connectors de Kafka ou les tâches de traitement de Flink [28].

III.3.1 Hadoop HDFS Cluster

- **HDFS Client** : Interface pour interagir avec HDFS [29].
- **Name Node** : Gère la métadonnée et la structure du système de fichiers HDFS.
- **Data Nodes** : Stockent les blocs de données, assurant la réplication et la tolérance aux pannes. Les données traitées peuvent être archivées dans HDFS pour des analyses ultérieures.

Apache Hive

- **MetaStore** : Gère les métadonnées des tables Hive [30].
- **Driver, Compiler, Executor** : Composants responsables de la compilation des requêtes SQL en plans d'exécution et de leur exécution sur les données stockées dans HDFS.

Prometheus

- **Service Discovery, Scraping, Rules & Alerts** : Prometheus surveille les systèmes et services, collectant des métriques et générant des alertes basées sur des règles définies [31].
- **AlertManager** : Gère les alertes générées par Prometheus.

ksqlDB

- **ksqlDB Server** : Fournit une interface SQL pour interagir avec les données Kafka en temps réel, créant des streams et des tables dérivées des topics Kafka [32].

Jupyter Server

- **Jupyter Server** : Environnement interactif pour l'analyse des données, permettant aux utilisateurs d'exécuter des notebooks Jupyter pour explorer et visualiser les données traitées.

III.3.2 Interactions entre les Composants

1. **Collecte des Données** : Les producteurs envoient des données à Kafka. Les Source Connectors de Kafka Connect importent des données de sources externes vers Kafka.
2. **Traitement des Données en Temps Réel** : Flink traite les données en temps réel provenant de Kafka. Les Task Managers exécutent les tâches de traitement, et les résultats peuvent être envoyés vers des sinks comme Elasticsearch, MongoDB ou PostgreSQL via des Sink Connectors.
3. **Stockage des Données** : Les données traitées sont stockées dans plusieurs bases de données :
 - **Elasticsearch** pour les recherches et analyses avancées.

- **MongoDB** et **PostgreSQL** pour le stockage structuré et semi-structuré.
 - **HDFS** pour l'archivage des données et les analyses de big data.
4. **Surveillance et Visualisation** : Prometheus collecte les métriques des différents composants, et Grafana visualise ces métriques pour la surveillance du système. Kibana permet de créer des tableaux de bord interactifs pour les données dans Elasticsearch.
 5. **Analyse et Exploration des Données** : Les utilisateurs utilisent Jupyter Server pour analyser et visualiser les données, se connectant aux différentes bases de données et systèmes de stockage pour extraire et traiter les informations nécessaires.

III.3.3 Flux de Données

Le flux de données (*data flow*) dans cette architecture suit un chemin bien défini et optimisé pour maximiser l'efficacité et la résilience :

1. Acquisition des Données :

- Les producteurs collectent les données de différentes sources et les envoient aux topics Kafka via des API ou des connecteurs Kafka.
- Les Source Connectors de Kafka Connect intègrent les données des sources externes et les publient dans les topics Kafka.

2. Traitement des Données en Temps Réel :

- Les données des topics Kafka sont consommées par Flink pour un traitement en temps réel.
- Flink exécute des opérations de transformation, d'agrégation et d'analyse en temps réel, puis publie les résultats traités vers des topics Kafka ou directement vers des bases de données cibles via des Sink Connectors.

3. Stockage des Données :

- Les données traitées peuvent être stockées dans Elasticsearch pour des analyses rapides et des recherches complexes.
- MongoDB et PostgreSQL sont utilisés pour le stockage structuré et semi-structuré, permettant une gestion efficace des données transactionnelles et des métadonnées.
- HDFS est utilisé pour l'archivage des données volumineuses et les analyses de big data, assurant une persistance durable et une haute disponibilité.

4. Surveillance et Visualisation :

- Prometheus surveille les performances et la santé de l'écosystème en collectant des métriques à partir des différents composants.
- Grafana et Kibana fournissent des interfaces de visualisation pour surveiller les métriques en temps réel et créer des tableaux de bord interactifs, facilitant ainsi l'analyse et la prise de décision.

5. Exploration et Analyse des Données :

- Les utilisateurs exploitent Jupyter Server pour explorer et analyser les données. Ils peuvent exécuter des notebooks interactifs, se connecter aux bases de données et systèmes de stockage pour extraire, transformer et visualiser les données.
- ksqlDB permet d'interagir avec les données Kafka en temps réel via une interface SQL, facilitant la création de streams et de tables pour des analyses dynamiques.

Cette architecture intégrée et modulaire permet une gestion optimale du cycle de vie des données, depuis leur collecte et leur traitement en temps réel jusqu'à leur stockage, leur surveillance et leur analyse approfondie. Elle assure ainsi une efficacité maximale et une résilience accrue pour répondre aux besoins complexes des systèmes Big Data modernes.

III.3.4 Stratégie de mise en œuvre

La stratégie de mise en œuvre d'un écosystème Big Data est cruciale pour assurer le succès de l'intégration et l'optimisation des performances. Cette section décrit les étapes clés de ce processus, en mettant l'accent sur un plan de mise en œuvre par phases, un plan d'intégration, ainsi que les méthodes de test et de validation.

III.3.4.1 Plan de mise en œuvre par phases

Pour assurer une transition en douceur et minimiser les risques, la mise en œuvre de l'écosystème Big Data se déroule en plusieurs phases distinctes.

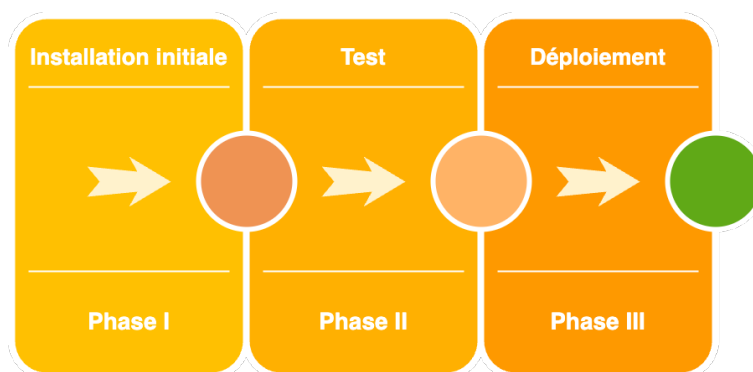


FIG. III.9 : Phases de mise en oeuvre

Phase 1: Installation initiale

Cette phase vise à préparer l'environnement, installer les outils et technologies de base, et configurer les composants essentiels. La configuration de Docker est une étape cruciale pour containeriser les applications, garantissant ainsi un déploiement cohérent à travers différents environnements. Ensuite, l'installation de Kafka et Apache Zookeeper permet de mettre en place des services de messagerie pour la gestion des flux de données. Kafka Connect sera également configuré pour assurer une intégration fluide des sources de données variées.

La mise en place de frameworks comme Apache Spark et Apache Flink est essentielle pour le traitement des données en temps réel et par lots. Hadoop Distributed File System (HDFS) sera installé et configuré pour servir de lac de données, offrant une solution de stockage scalable et fiable pour les grandes quantités de données non structurées et semi-structurées. Hive sera mis en place comme entrepôt de données pour permettre des requêtes SQL sur de grandes quantités de données stockées dans HDFS.

PostgreSQL et MongoDB seront configurés pour répondre aux besoins en bases de données relationnelles et NoSQL, respectivement. PostgreSQL offrira une gestion robuste des transactions et des capacités avancées de requête, tandis que MongoDB fournira une solution flexible pour les données non structurées et les documents. Elasticsearch sera mis en place pour l'indexation et la recherche rapide des données, permettant des requêtes complexes et des analyses en temps réel. Enfin, Kibana sera utilisé pour la visualisation des données et le monitoring en temps réel, offrant une interface utilisateur intuitive pour explorer et analyser les données indexées dans Elasticsearch.

En configurant ces technologies de manière cohérente, cette phase assure que l'écosystème est prête pour les phases de test et de déploiement ultérieures, avec une base solide pour le traitement, le stockage, et l'analyse des données à grande échelle.

Phase 2: Tests

L'objectif de cette phase est de valider la configuration initiale et de tester les composants critiques de l'écosystème. Les tests unitaires sont exécutés sur chaque composant isolé pour vérifier leur bon fonctionnement. Ensuite, des tests d'intégration sont réalisés pour s'assurer que les différents composants interagissent correctement ensemble. De plus, des scénarios de test réalistes, basés sur des cas d'utilisation spécifiques à l'entreprise, sont mis en place pour valider les performances et la fonctionnalité de l'écosystème.

Phase 3: Déploiement

Cette phase vise à déployer l'écosystème en production et à s'assurer qu'elle fonctionne correctement. Le déploiement progressif est recommandé pour surveiller et corriger les problèmes potentiels. La mise en place d'outils de surveillance et de journalisation est cruciale pour suivre les performances et diagnostiquer les problèmes. Enfin, la formation des équipes internes à l'utilisation et à la gestion du nouveau écosystème est essentielle pour garantir une adoption réussie.

III.3.4.2 Plan d'intégration

L'intégration du nouveau écosystème Big Data avec les systèmes existants est essentielle pour garantir une continuité opérationnelle et tirer parti des données disponibles.

Les stratégies d'intégration incluent l'utilisation de connecteurs Kafka (Kafka Connect) pour intégrer diverses sources de données, y compris les bases de données relationnelles comme PostgreSQL et NoSQL comme MongoDB. Le développement d'APIs RESTful permet d'accéder aux données et d'interagir avec les systèmes existants de manière fluide. La migration progressive des données des systèmes existants vers le Data Lake et le Data Warehouse est planifiée pour éviter les perturbations. En outre, des pipelines ETL (Extract, Transform, Load) et ELT (Extract, Load, Transform) sont mis en place pour assurer la qualité et la cohérence des données intégrées.

III.3.4.3 Test et validation

Les tests de performance évaluent les performances de l'écosystème sous différentes charges de travail pour identifier les goulots d'étranglement et optimiser les configurations. Les critères de test incluent la latence, le débit, la scalabilité, et la tolérance aux pannes, et des benchmarks standards sont utilisés pour comparer les performances par rapport à des attentes préétablies. Les tests de sécurité vérifient les

mécanismes de sécurité pour garantir la protection des données sensibles, comprenant des audits de sécurité pour identifier les vulnérabilités potentielles et des tests de pénétration pour simuler des attaques et évaluer la résilience de l'écosystème. Les tests de conformité assurent que l'écosystème respecte les normes et réglementations en vigueur. Enfin, la validation fonctionnelle des fonctionnalités critiques de l'écosystème est réalisée à travers des scénarios de test spécifiques aux cas d'utilisation de l'entreprise.

En suivant ce plan de mise en œuvre par phases, avec des stratégies d'intégration bien définies et des méthodes de test rigoureuses, l'écosystème Big Data sera capable de répondre aux exigences actuelles et futures de l'entreprise, tout en assurant une transition en douceur et efficace.

III.3.5 Mode de déploiement

Le déploiement de l'écosystème Big Data se déroule en plusieurs phases, chacune adaptée aux besoins spécifiques de l'étape correspondante du projet. Actuellement, nous nous trouvons dans la phase de développement, en utilisant un environnement Docker. Cette approche permet une itération rapide et une flexibilité maximale avant de passer à la phase de production sur des serveurs physiques.

III.3.5.1 Phase de développement en environnement Docker

Pendant la phase de développement, l'écosystème Big Data est déployé dans un environnement Docker. L'utilisation de Docker permet de containeriser les applications, ce qui assure une uniformité et une portabilité des environnements de développement. Chaque composant de l'écosystème, tel que Kafka, Spark, Flink, et HDFS, est déployé dans des conteneurs Docker distincts. Cette approche facilite la gestion des dépendances et des versions, et permet aux développeurs de reproduire facilement l'environnement de développement sur leurs machines locales ou sur des serveurs de développement partagés.

L'avantage principal de l'utilisation de Docker dans cette phase est la rapidité de mise en place et la flexibilité. Les développeurs peuvent rapidement itérer sur les configurations et les déploiements, tester différentes versions des composants et déboguer les problèmes dans un environnement isolé. De plus, Docker Compose est utilisé pour orchestrer les différents conteneurs, simplifiant ainsi la gestion des services interconnectés.

III.3.5.2 Phase de production sur des serveurs physiques

Lors de cette phase, l'écosystème Big Data sera déployé sur des serveurs physiques. Ce passage des conteneurs Docker aux serveurs physiques permettra d'exploiter pleinement les ressources matérielles disponibles, offrant ainsi une meilleure

performance et une plus grande fiabilité. Les serveurs physiques seront configurés pour héberger les clusters des différents composants de l'écosystème, tels que Kafka, Spark, Flink, et HDFS.

L'utilisation de serveurs physiques en production permettra également de mieux contrôler la sécurité et la conformité. Les configurations matérielles seront optimisées pour les charges de travail spécifiques de l'entreprise, et des mesures de sécurité avancées pourront être mises en place pour protéger les données. De plus, cette approche permettra de mieux gérer la scalabilité horizontale, en ajoutant ou en supprimant des nœuds au sein des clusters en fonction des besoins de l'entreprise.

En combinant ces deux approches de déploiement, nous assurons une flexibilité maximale pendant la phase de développement et une performance optimale en production. Cette stratégie permet également de tester et de valider l'écosystème dans un environnement isolé avant de la déployer dans un environnement de production plus complexe et critique.

Conclusion

L'importance d'une conception soigneusement planifiée pour le succès du déploiement d'un écosystème Big Data ne peut être sous-estimée. Une architecture soigneusement planifiée permet non seulement de répondre aux exigences fonctionnelles et non fonctionnelles, mais aussi de garantir une évolutivité et une résilience à long terme. En intégrant des technologies robustes et des stratégies de gestion efficaces, nous nous assurons que l'écosystème Big Data sera capable de soutenir les besoins croissants de l'entreprise, tout en maintenant des performances optimales et en assurant la sécurité des données.

En résumé, une approche méthodique et bien structurée est essentielle pour réussir le déploiement d'un écosystème Big Data. Cela inclut une phase de développement flexible et itérative, suivie d'un déploiement rigoureux et sécurisé en production. En suivant ces principes, nous pouvons maximiser les bénéfices de l'écosystème Big Data et soutenir efficacement les divers cas d'utilisation de l'entreprise.

IV Implementation

SOMMAIRE

IV.1	INTRODUCTION	59
IV.2	PRÉPARATION DE L'ENVIRONNEMENT	60
IV.2.1	Configuration des outils de développement	60
IV.3	GESTION DES DÉPENDANCES	61
IV.4	DÉPLOIEMENT DES COMPOSANTS	63
IV.4.1	Mise en place de Docker et des conteneurs	63
IV.4.2	Déploiement de Kafka et Zookeeper	63
IV.4.3	Configuration des topics et des connecteurs Kafka	64
IV.4.4	Déploiement de Flink	65
IV.5	CONFIGURATION DES JOBS DE TRAITEMENT EN TEMPS RÉEL . . .	66
IV.6	DÉPLOIEMENT DES BASES DE DONNÉES	67
IV.7	DÉPLOIEMENT DE ELASTICSEARCH ET KIBANA	67
IV.8	TESTS ET VALIDATION	69
IV.8.1	Tests unitaires et d'intégration	69
IV.8.2	Tests de performance	69
IV.9	CONCLUSION	71

CE chapitre détaille l'implémentation de notre écosystème Big Data, en abordant la configuration de l'environnement de développement, le déploiement des composants essentiels, et la validation du système.

IV.1 Introduction

Dans ce chapitre, nous allons détailler les étapes de mise en œuvre de l'écosystème Big Data que nous avons conçue précédemment. Après la phase de conception, nous avons réparti le travail de manière stratégique pour tirer parti de nos compétences respectives. J'ai été chargé de la partie d'ingestion et de traitement des données en temps réel, incluant les composants suivants :

- **Kafka** et **Kafka Connect** pour la gestion des flux de données et l'intégration de sources variées.
- **ksqlDB** pour l'interaction en temps réel avec les données Kafka via une interface SQL.
- **Redpanda** pour une gestion améliorée des flux de données avec des performances optimisées.
- **MongoDB** et **PostgreSQL** pour le stockage des données structurées et semi-structurées.
- **Flink** pour le traitement en temps réel des flux de données.
- **Elasticsearch** et **Kibana** pour l'indexation, la recherche et la visualisation des données.

Haitam Belainoussi, second membre de l'équipe, a pris en charge la mise en œuvre des autres composants critiques de l'écosystème, incluant :

- **Spark** pour le traitement par lots de grandes quantités de données.
- **HDFS** pour le stockage distribué et évolutif des données.
- **Apache Hive** pour l'analyse des données stockées dans HDFS via une interface SQL.
- **Jupyter** pour l'analyse interactive et la visualisation des données.
- **Prometheus** pour la surveillance et la collecte des métriques de performance

Cette répartition nous a permis de travailler efficacement et de manière spécialisée sur les différentes parties de l'écosystème, assurant ainsi une mise en œuvre fluide et cohérente. Dans les sections suivantes, nous détaillerons les étapes spécifiques de l'implémentation, les défis rencontrés et les solutions adoptées pour chaque composant de l'écosystème.

IV.2 Préparation de l'environnement

IV.2.1 Configuration des outils de développement

Avant de commencer l'implémentation de l'écosystème Big Data, il est essentiel de préparer un environnement de développement adéquat pour garantir une configuration cohérente et une gestion efficace des composants. Voici les étapes suivies pour la configuration des outils de développement :

Choix des environnements de développement (local, Docker, etc.)

Pour assurer une flexibilité et une portabilité maximales, nous avons opté pour l'utilisation de Docker. Docker permet de containeriser les applications, ce qui facilite leur déploiement et leur gestion à travers différents environnements, qu'ils soient locaux ou sur des serveurs de développement partagés. Chaque composant de l'écosystème, tel que Kafka, Kafka Connect, ksqlDB, Redpanda, MongoDB, PostgreSQL, Flink, Elasticsearch, et Kibana, a été déployé dans des conteneurs Docker distincts.

- **Environnement local** : Pour le développement et les tests initiaux, nous avons configuré un environnement local sur nos machines de développement. Cela nous a permis de développer et de tester des composants de manière isolée avant de les intégrer dans un environnement plus complexe.
- **Environnement Docker** : Docker a été choisi pour containeriser les applications et assurer une cohérence dans les configurations à travers différents environnements. Cela a également permis une scalabilité facile et une gestion simplifiée des dépendances.

Installation et configuration des outils nécessaires

Pour le développement et la gestion des conteneurs, les outils suivants ont été installés et configurés :

- **Visual Studio Code et IntelliJ** : Nous avons utilisé des environnements de développement intégrés tels que Visual Studio Code et IntelliJ IDEA pour écrire et gérer le code. Ces IDE offrent de nombreuses extensions et fonctionnalités qui facilitent le développement, le débogage et la collaboration.
- **Docker** : Docker a été installé pour containeriser les applications. La configuration de Docker inclut la création de fichiers Dockerfile pour chaque composant et l'utilisation de Docker Compose pour orchestrer les différents conteneurs.

- **Dockerfile** : Pour chaque composant, un fichier Dockerfile a été créé pour définir l'image Docker à construire. Cela inclut l'installation des dépendances nécessaires et la configuration de l'environnement d'exécution.
- **Docker Compose** : Docker Compose a été utilisé pour définir et gérer des applications multi-conteneurs. Un fichier `docker-compose.yml` a été créé pour orchestrer les différents services et assurer leur communication et coordination.

Voici l'architecture Docker utilisée pour déployer les différents composants de l'écosystème :

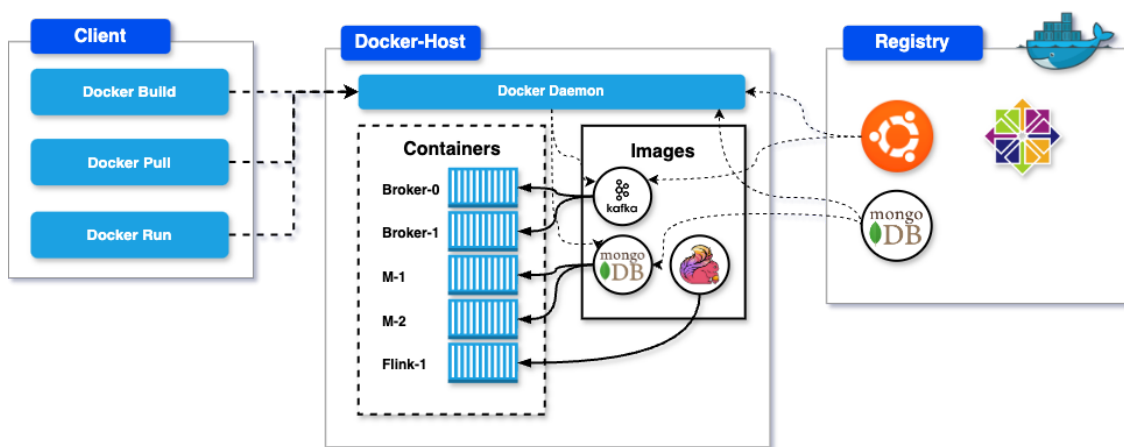


FIG. IV.1 : Architecture Docker

Cette architecture montre comment les clients utilisent des commandes Docker (build, pull, run) pour interagir avec le Docker daemon, qui gère les conteneurs et les images sur le Docker host. Les images peuvent être stockées dans un registre Docker, ce qui facilite le déploiement et la gestion des conteneurs sur différentes plateformes.

IV.3 Gestion des dépendances

Une gestion efficace des dépendances est cruciale pour assurer le bon fonctionnement et la maintenance de notre écosystème Big Data. Cette section décrit les bibliothèques et outils nécessaires, ainsi que la configuration des fichiers de dépendances utilisés dans notre projet.

Configuration des fichiers de dépendances

La gestion des dépendances a été réalisée principalement via des fichiers de configuration spécifiques à chaque technologie utilisée. Voici les principaux fichiers de configuration et leur rôle :

- **Docker Compose pour orchestrer les conteneurs**

Docker Compose a été utilisé pour orchestrer le déploiement et la gestion des conteneurs. Le fichier `docker-compose.yml` regroupe l'ensemble des configurations nécessaires pour démarrer tous les services de l'écosystème. Voici les étapes clés de cette orchestration :

- **Définition des services** : Le fichier `docker-compose.yml` définit chaque service avec ses ports, variables d'environnement et dépendances. Cela inclut la configuration des réseaux et des volumes pour assurer la communication et la persistance des données entre les conteneurs.
- **Gestion des dépendances** : Les services sont configurés pour démarrer dans un ordre spécifique, assurant que les dépendances critiques, telles que Zookeeper pour Kafka ou Elasticsearch pour Kibana, sont disponibles avant le lancement des services dépendants.
- **Isolation des réseaux** : Docker Compose permet de définir des réseaux isolés pour les différents services, assurant une communication sécurisée et contrôlée entre les conteneurs.
- **Volumes de données** : Les volumes sont utilisés pour persister les données des bases de données et autres services critiques, permettant la sauvegarde et la récupération des données en cas de redémarrage des conteneurs.

- **Dockerfile**

Pour chaque service, un fichier `Dockerfile` a été créé pour définir l'image Docker à construire. Cela inclut l'installation des dépendances nécessaires et la configuration de l'environnement d'exécution.

- **Maven pour Apache Flink (pom.xml)**

Le fichier `pom.xml` de Maven gère les dépendances pour les jobs Flink. Il spécifie les bibliothèques nécessaires pour le développement et l'exécution des applications Flink, telles que les connecteurs Kafka, les drivers PostgreSQL, et les outils de logging. Ce fichier assure que toutes les dépendances requises sont disponibles et correctement configurées pour le projet Flink.

En structurant notre gestion des dépendances de cette manière, nous avons pu établir une base solide pour le développement et le déploiement de notre écosystème Big Data. Cette approche assure que chaque composant fonctionne de manière cohérente et intégrée, facilitant ainsi le développement, la maintenance et la scalabilité de notre solution. Mon travail sur cette partie a permis de garantir une ingestion et un traitement efficaces des données en temps réel, contribuant de manière significative au succès global de notre projet.

IV.4 Déploiement des composants

IV.4.1 Mise en place de Docker et des conteneurs

Le déploiement des composants de l'écosystème Big Data a été réalisé en utilisant Docker pour containeriser chaque service et Docker Compose pour orchestrer l'ensemble des conteneurs. Cette approche garantit une isolation optimale des services, facilite la gestion des dépendances, et assure une portabilité et une reproductibilité des environnements de développement et de production.

Création de fichiers Dockerfile pour chaque composant

Pour chaque composant de l'écosystème, un fichier Dockerfile a été créé. Ces fichiers définissent l'environnement d'exécution des services, spécifiant les dépendances nécessaires, les configurations et les instructions pour le démarrage des applications. Voici un aperçu des étapes de création des Dockerfile :

- **Kafka et Kafka Connect** : Les Dockerfile pour Kafka et Kafka Connect incluent les configurations de base pour les brokers Kafka, ainsi que les plugins nécessaires pour Kafka Connect, tels que les connecteurs MongoDB et HDFS.
- **ksqlDB** : Le Dockerfile pour ksqlDB configure le service pour interagir avec les brokers Kafka et le schéma registry, permettant l'exécution de requêtes SQL sur les flux de données.
- **Redpanda** : Utilisé pour la visualisation et la gestion des topics Kafka, le Dockerfile pour Redpanda configure l'interface web et les connexions aux brokers Kafka.
- **MongoDB et PostgreSQL** : Les Dockerfile pour ces bases de données incluent les configurations nécessaires pour les paramètres de connexion, la réplication et la persistance des données.
- **Flink** : Le Dockerfile pour Flink configure les JobManager et TaskManager, définissant les paramètres de traitement des données et les connexions aux sources de données.
- **Elasticsearch et Kibana** : Les Dockerfile pour Elasticsearch et Kibana incluent les configurations pour l'indexation des données et la visualisation des tableaux de bord.

IV.4.2 Déploiement de Kafka et Zookeeper

Le déploiement de Kafka et Zookeeper constitue une étape cruciale dans la mise en place de l'écosystème de traitement des données. Kafka est utilisé pour la gestion

des flux de données en temps réel, tandis que Zookeeper assure la coordination des services distribués.

Installation et configuration de Kafka et Zookeeper

- **Kafka** : Trois brokers Kafka ont été configurés pour assurer la redondance et la haute disponibilité. Chaque broker est défini dans un conteneur Docker distinct avec les paramètres de configuration nécessaires, tels que les ports, les ID de broker, et les paramètres de réplication.
- **Zookeeper** : Un conteneur Zookeeper a été déployé pour assurer la coordination des brokers Kafka. Zookeeper gère la configuration et le statut des brokers, ainsi que les informations de synchronisation.

Voici l'état actuel de notre cluster Kafka, montrant que tous les brokers sont en cours d'exécution et que les services Kafka Connect et Schema Registry sont également opérationnels :

Running	2.15 GiB	v3.6	3 of 3	12	372
Cluster Status	Cluster Storage Size	Cluster Version	Brokers Online	Topics	Replicas

BROKER DETAILS			
ID	STATUS	SIZE	
1	✓ Running	734 MiB	View
2	✓ Running	734 MiB	View
3	✗ Running	734 MiB	View

CLUSTER DETAILS	
SERVICES	
Kafka Connect	Running
Schema Registry	Running
STORAGE	
Total Bytes	2.15 GiB
Primary	734 MiB
Replicated	1.43 GiB

FIG. IV.2 : Brokers Kafka

IV.4.3 Configuration des topics et des connecteurs Kafka

Après l'installation, les topics nécessaires ont été configurés avec des paramètres spécifiques, tels que le facteur de réplication et le nombre de partitions, pour assurer une distribution équilibrée des données et une tolérance aux pannes.

- **Topics** : Des topics ont été créés pour différents types de données. Par exemple, un topic pour les données de trafic a été configuré avec un facteur de réplication de 3 pour garantir la disponibilité des données même en cas de défaillance d'un broker. Voici les partitions et leur répartition entre les brokers

1.82 GiB

1308540

Size

Messages

delete

cleanup.policy

7 days

retention.ms

Infinite

retention.bytes

Messages	Consumers	Partitions	Configuration	ACL	Documentation
PARTITION ID ▲	LOW ⓘ ▲	HIGH ⓘ ▲	MESSAGES	BROKERS	
0	0	437 794	437 794	1 2 3	
2	0	429 853	429 853	1 2 3	
1	0	440 893	440 893	1 2 3	

20 / page ▼

< 1 >

FIG. IV.3 : Partitions et répartition de topic entre les brokers

- **Connecteurs Kafka** : Des connecteurs Kafka ont été configurés pour intégrer Kafka avec d’autres systèmes de données, tels que MongoDB et PostgreSQL. Cela permet de transférer les données de manière fluide entre les différents composants de l’écosystème. Voici le connecteur MongoDB qui consomme les données du topic configuré :

1.82 GiB

Size

1308540

Messages

delete

cleanup.policy

7 days

retention.ms

Infinite

retention.bytes

Messages

Consumers

Partitions

Configuration

ACL

Documentation

GROUP ▲

LAG ▲

connect-mongodb-sink-connector

0

20 / page ▼

< 1 >

FIG. IV.4 : Connecteur MongoDB consommant des données du topic Kafka

Cette configuration assure la résilience et la disponibilité de notre système de gestion des flux de données, permettant une ingestion et un traitement efficaces des données en temps réel.

IV.4.4 Déploiement de Flink

Le déploiement de Flink est essentiel pour le traitement en temps réel des données collectées par Kafka. Flink permet d’analyser et de traiter les flux de données en continu, offrant une plateforme robuste pour les applications de traitement de données à faible latence.

IV.4.4.1 Installation et configuration de Flink

Les composants principaux de Flink, tels que le JobManager et les TaskManagers, ont été configurés pour travailler ensemble de manière harmonieuse.

- **JobManager** : Le JobManager est responsable de la coordination des jobs Flink. Il a été configuré dans un conteneur Docker avec les paramètres nécessaires pour gérer les tâches et distribuer le travail aux TaskManagers.

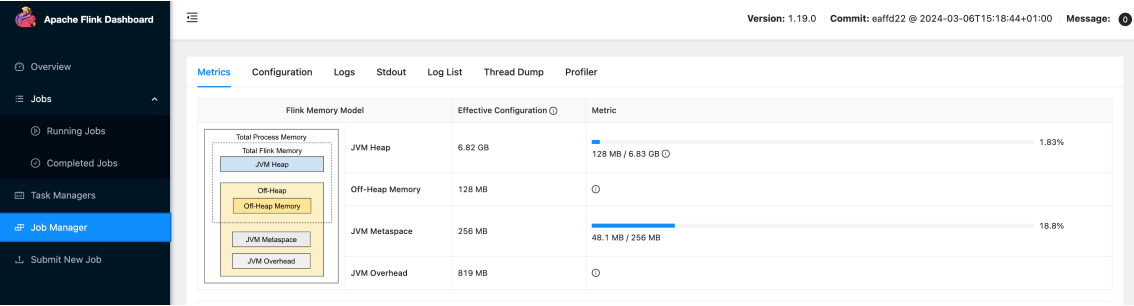


FIG. IV.5 : tableau de bord du JobManager Flink

- **TaskManagers** : Les TaskManagers exécutent les tâches assignées par le JobManager. Plusieurs TaskManagers ont été déployés pour assurer une scalabilité horizontale, permettant à Flink de traiter de grandes quantités de données en parallèle.

Task Managers

Path, ID	Data Port	Last Heartbeat	All Slots	Free Slots	CPU Cores	Physical MEM	JVM Heap Size	Flink Managed MEM
172.21.0.11:43863-ee7fb1	43411	2024-06-05 14:26:42	4	4	30	197 GB	512 MB	512 MB
pekko.tcp://flink@172.21.0.11:43863/user/rpc/t								
askmanager_0								

FIG. IV.6 : Gestion du taskManager Flink

IV.5 Configuration des jobs de traitement en temps réel

Une fois Flink installé et configuré, des jobs de traitement en temps réel ont été définis pour analyser les données en provenance de Kafka.

- **Définition des jobs** : Les jobs Flink ont été écrits pour lire les données des topics Kafka, les traiter en temps réel et écrire les résultats dans des bases de données ou d'autres systèmes de stockage. Les jobs incluent des tâches telles que l'agrégation, la transformation, et le filtrage des données.
- **Exécution des jobs** : Les jobs de traitement en temps réel sont soumis au JobManager, qui les distribue aux TaskManagers pour exécution. Les résultats des jobs sont ensuite stockés dans MongoDB et PostgreSQL, ou indexés dans Elasticsearch pour une analyse ultérieure.

En intégrant Flink dans notre écosystème, nous avons pu mettre en place un système de traitement de données en temps réel performant et évolutif, capable de gérer les flux de données massifs générés par nos sources de données.

IV.6 Déploiement des bases de données

Le déploiement de MongoDB et PostgreSQL est une étape cruciale pour assurer un stockage structuré et non structuré des données. Ces bases de données sont configurées pour gérer efficacement les grandes quantités de données collectées et traitées par notre écosystème.

configuration de MongoDB et PostgreSQL

- **MongoDB** : Pour la gestion des données non structurées, plusieurs instances de MongoDB ont été configurées en mode réplica pour assurer la haute disponibilité et la tolérance aux pannes.
- **PostgreSQL** : PostgreSQL a été installé pour gérer les données structurées. PostgreSQL a été configuré avec les paramètres nécessaires pour optimiser les performances et garantir la fiabilité des transactions.

Configuration des schémas et des index

- **Schémas MongoDB** : Les collections MongoDB ont été créées avec des schémas définis pour structurer les données. Des index ont été configurés pour améliorer la rapidité des requêtes et l'efficacité des opérations de recherche.
- **Schémas PostgreSQL** : Les tables dans PostgreSQL ont été définies avec des schémas appropriés pour les différentes entités de données. Des clés primaires et des index ont été ajoutés pour optimiser les opérations de requête et assurer l'intégrité des données.

Ces configurations permettent à MongoDB et PostgreSQL de gérer efficacement les données collectées et traitées, assurant ainsi une performance optimale et une haute disponibilité de notre écosystème de stockage des données.

IV.7 Déploiement de Elasticsearch et Kibana

Le déploiement d'Elasticsearch et Kibana est essentiel pour permettre une recherche rapide et une visualisation des données collectées. Ces outils sont configurés

pour offrir une interface intuitive permettant de surveiller et d'analyser les données en temps réel.

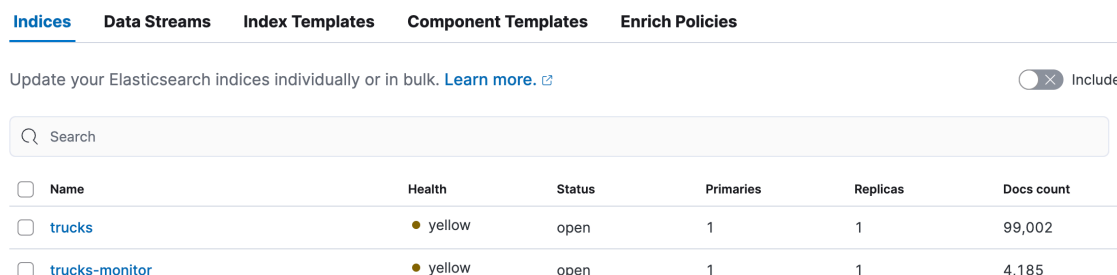
Installation et configuration de Elasticsearch et Kibana

- **Elasticsearch** : Conçue pour fournir des capacités de recherche avancées, Elasticsearch a été configuré pour fonctionner en mode cluster, ce qui permet d'assurer la scalabilité et la tolérance aux pannes.
- **Kibana** : élaboré pour permettre la visualisation des données indexées par Elasticsearch, Kibana est configuré pour se connecter à notre cluster Elasticsearch, offrant ainsi une interface utilisateur conviviale pour la création de tableaux de bord et de visualisations interactives.

Création des index et des tableaux de bord

- **Création des index** : Les index Elasticsearch ont été créés pour structurer et optimiser la recherche des données. Des mappings ont été définis pour chaque type de données, assurant ainsi une indexation efficace et une recherche rapide.

Index Management



The screenshot shows the 'Indices' tab in the Elasticsearch Index Management interface. At the top, there are navigation links: 'Indices' (active), 'Data Streams', 'Index Templates', 'Component Templates', and 'Enrich Policies'. Below the links, there is a text prompt: 'Update your Elasticsearch indices individually or in bulk. [Learn more.](#)' and a toggle switch labeled 'Include' which is currently turned off. A search bar with a magnifying glass icon and the text 'Search' is positioned above a table. The table has six columns: 'Name', 'Health', 'Status', 'Primarys', 'Replicas', and 'Docs count'. There are two rows of data, each with a checkbox in the 'Name' column. The first row has 'Name' as 'trucks', 'Health' as 'yellow' (indicated by a yellow dot), 'Status' as 'open', 'Primarys' as '1', 'Replicas' as '1', and 'Docs count' as '99,002'. The second row has 'Name' as 'trucks-monitor', 'Health' as 'yellow' (indicated by a yellow dot), 'Status' as 'open', 'Primarys' as '1', 'Replicas' as '1', and 'Docs count' as '4,185'.

☐ Name	Health	Status	Primarys	Replicas	Docs count
☐ trucks	● yellow	open	1	1	99,002
☐ trucks-monitor	● yellow	open	1	1	4,185

FIG. IV.7 : Gestion des indices Elasticsearch

- **Création des tableaux de bord** : Kibana a été utilisé pour créer des tableaux de bord interactifs qui permettent de visualiser les données en temps réel. Ces tableaux de bord facilitent la surveillance des performances du système et l'analyse des données collectées, offrant ainsi des insights précieux pour la prise de décision.

Grâce à cette configuration, Elasticsearch et Kibana fournissent une solution robuste pour la recherche et la visualisation des données, permettant de tirer pleinement parti des informations collectées et traitées par notre écosystème Big Data.

Enfin, voici tous les conteneurs actuellement en cours d'exécution dans notre environnement :

	Mean ▾	Last *
 docker: gcr.io/cadvisor/cadvisor (cadvisor)	55.92%	56.60%
 docker: gcr.io/cadvisor/cadvisor (cadvisor-2)	20.64%	21.12%
 docker: confluentinc/cp-kafka:7.6.0 (kafka-3)	7.33%	27.86%
 docker: confluentinc/cp-kafka:7.6.0 (kafka-1)	7.13%	26.81%
 docker: confluentinc/cp-kafka:7.6.0 (kafka-2)	6.81%	25.64%
 docker: confluentinc/cp-kafka-connect-base:7.6.0 (kafka-connect)	6.06%	19.65%
 docker: mongo:4.0.4 (m1)	5.39%	22.29%
 docker: prom/prometheus (prometheus)	5.25%	8.28%
 docker: docker.elastic.co/kibana/kibana:8.12.2 (kibana)	4.32%	4.25%
 docker: grafana (grafana)	3.60%	9.74%
 docker: mongo:4.0.4 (m3)	3.58%	12.68%
 docker: mongo:4.0.4 (m2)	3.57%	12.67%
 docker: jupyter (jupyter)	2.14%	0.18%
 docker: docker.elastic.co/elasticsearch/elasticsearch:8.12.2 (elasticsearch)	1.67%	1.66%
 docker: hadoop (master)	1.49%	1.35%
 docker: flink:1.19.0-scala_2.12 (taskmanager)	1.11%	1.10%
 docker: flink:1.19.0-scala_2.12 (jobmanager)	1.09%	1.07%
 docker: hadoop (slave3)	1.05%	0.89%
 docker: hadoop (slave2)	1.03%	0.89%
 docker: hadoop (slave1)	0.99%	0.93%
 docker: hadoop (slave4)	0.97%	0.91%
 docker: postgres:latest (postgres)	0.77%	0.78%
 docker: confluentinc/cp-schema-registry:7.6.0 (schema-registry)	0.54%	0.51%
 docker: hadoop (historyserver)	0.45%	0.14%
 docker: confluentinc/ksqldb-server:0.29.0 (ksqldb)	0.37%	0.37%
 docker: confluentinc/cp-zookeeper:7.6.0 (zookeeper)	0.30%	0.15%
 docker: docker.redpanda.com/redpandadata/console:latest (redpanda)	0.14%	0.05%
 docker: jark/demo-sql-client:0.2 (sqlClient)	0.00%	0.00%

FIG. IV.8 : Les conteneurs en cours d'exécution

IV.8 Tests et validation

IV.8.1 Tests unitaires et d'intégration

Pour garantir le bon fonctionnement de chaque composant de l'écosystème, des tests unitaires ont été réalisés. Chaque composant, comme Kafka, MongoDB, PostgreSQL, et Flink, a été testé individuellement pour vérifier qu'il fonctionne comme prévu. Les tests d'intégration ont ensuite été effectués pour s'assurer que tous les composants interagissent correctement ensemble. Ces tests ont permis de valider le flux complet de données, depuis leur ingestion avec Kafka, leur traitement en temps réel avec Flink, jusqu'à leur stockage et visualisation dans MongoDB, PostgreSQL, Elasticsearch et Kibana.

IV.8.2 Tests de performance

Le serveur utilisé pour cette évaluation dispose des caractéristiques CPU suivantes :

- **Architecture** : x86_64
- **Modes de fonctionnement du CPU** : 32-bit, 64-bit
- **Nombre de CPU** : 30

- **ID du fournisseur** : GenuineIntel
- **Modèle** : Intel(R) Xeon(R) CPU E5-2695 v4 @ 2.10GHz

Pour évaluer les performances de notre infrastructure, nous avons développé un script Python simulant l'envoi continu de données à partir d'une boîte de véhicule. Le script ajoute un nouveau véhicule toutes les deux secondes jusqu'à atteindre un total de 100 000 véhicules. Ces données sont ensuite envoyées vers un topic Kafka avec 3 réplicas pour assurer la redondance et la tolérance aux pannes. Flink a été configuré pour traiter ces données en temps réel et les acheminer vers MongoDB et Elasticsearch pour le stockage et l'analyse.

Après la simulation, nous avons analysé les performances de l'infrastructure en utilisant Grafana pour visualiser l'utilisation des ressources par les différents conteneurs. Par exemple, avec la simulation de 100 000 véhicules, nous avons constaté que notre serveur équipé de 30 cœurs utilisait environ 30% de sa capacité CPU, démontrant ainsi la capacité de notre infrastructure à gérer efficacement des charges élevées.

Ces tests ont confirmé la scalabilité et la flexibilité de notre infrastructure, prouvant qu'elle peut gérer efficacement des volumes massifs de données tout en maintenant des performances stables. Les résultats obtenus ont également permis d'identifier des opportunités d'optimisation supplémentaires pour améliorer encore plus les performances et l'efficacité de notre infrastructure.

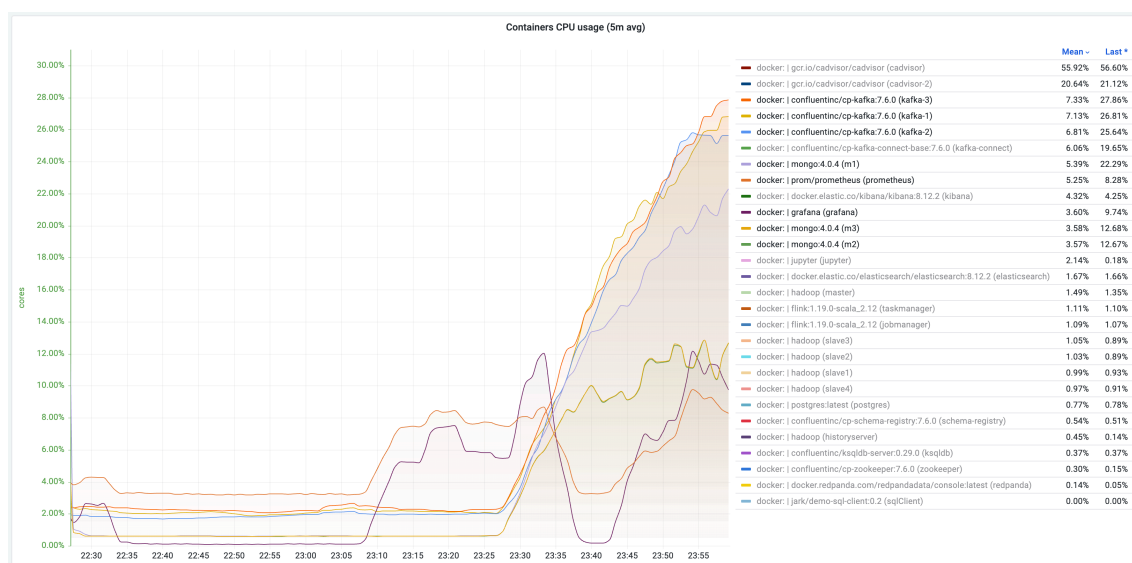


FIG. IV.9 : Évaluation des performances de l'écosystème

IV.9 Conclusion

Ce chapitre a décrit en détail les étapes clés de l'implémentation de notre écosystème Big Data, mettant en avant la préparation de l'environnement, le déploiement des composants essentiels tels que Kafka, Zookeeper, Flink, MongoDB, PostgreSQL, Elasticsearch et Kibana, ainsi que les tests et la validation de l'ensemble du système. L'utilisation de Docker pour la containerisation a assuré une flexibilité et portabilité, facilitant ainsi la gestion des dépendances et le déploiement. Les tests unitaires et d'intégration ont garanti la cohérence et la fiabilité, tandis que les tests de performance ont permis d'optimiser les configurations pour une meilleure scalabilité. Les techniques d'optimisation et les stratégies de maintenance mises en place garantiront la durabilité et la performance continue de l'écosystème. En résumé, cette implémentation pose une base solide pour les futures évolutions, avec des enseignements précieux pour améliorer et sécuriser davantage le système.

V Etude de cas de la congestion du trafic en temps réel

SOMMAIRE

V.1	ETUDE DE CAS DE LA CONGESTION DU TRAFIC EN TEMPS RÉEL .	73
V.1.1	Introduction	73
V.1.2	Capteurs et logistique mise en place	73
V.1.3	Source et prétraitement des données	74
V.1.4	Exploration et Visualisation des Données	77
V.2	NETTOYAGE ET PRÉPARATION DES DONNÉES	79
V.3	CHOIX ET ENTRAÎNEMENT DES MODÈLES D'APPRENTISSAGE AUTOMATIQUE	80
V.3.1	Machines à Vecteurs de Support	80
V.3.2	K-Nearest Neighbors (KNN)	82
V.3.3	Gated Recurrent Unit (GRU)	84
V.4	INTERPRÉTATION DES MÉTRIQUES DE PERFORMANCE	87
V.4.1	Métriques d'évaluation	87
V.4.2	Comparaison entre les modèles	88
V.5	INTÉGRATION DU MODÈLE DANS L'ÉCOSYSTÈME EXISTANT	88
V.6	CONCLUSION	89

DANS ce chapitre, nous aborderons la prédiction de la congestion du trafic à l'aide de l'apprentissage automatique. Nous discuterons de la collecte des données, du prétraitement, du choix des modèles, de l'entraînement, des tests, ainsi que de la comparaison des modèles, la sélection du modèle le plus adapté et l'intégration du modèle dans notre écosystème.

V.1 Etude de cas de la congestion du trafic en temps réel

V.1.1 Introduction

Afin d'illustrer un exemple concret où la nécessité d'un écosystème de big data est cruciale, nous avons choisi la prédiction de la congestion du trafic en temps réel. Dans cette étude de cas, nous démontrerons comment les composants concernés de notre écosystème collaborent pour collecter, traiter et analyser ces données afin de prévoir efficacement les embouteillages. Cela mettra en lumière l'importance et l'efficacité d'un écosystème de big data dans l'amélioration de la gestion du trafic urbain.

V.1.1.1 Problématique du trafic routier

La gestion du trafic routier est un défi majeur pour les zones urbaines modernes. L'augmentation du nombre de véhicules entraîne des embouteillages fréquents, des pertes de temps, une pollution accrue et un impact négatif sur la qualité de vie. La prédiction et la gestion efficaces de la congestion sont essentielles pour améliorer la fluidité des déplacements.

Avec la prolifération des dispositifs IoT (Internet des Objets), nous disposons de nouvelles opportunités pour mieux contrôler ces problèmes. Les capteurs de trafic, les GPS des véhicules et les caméras de surveillance génèrent une quantité massive de données en temps réel. L'intégration de ces données au sein d'un écosystème de big data permet de surveiller les conditions de circulation, de prédire les congestions potentielles et de prendre des mesures proactives pour les éviter.

V.1.2 Capteurs et logistique mise en place

Pour prédire la congestion du trafic, nous utilisons des données provenant de diverses sources, notamment des edgeboxes et d'autres capteurs. Les edgeboxes, placés dans les véhicules, sont équipés de capteurs qui capturent des informations variées sur les véhicules (état du véhicule, kilométrage, suivi, etc.) et les conditions routières. D'autres capteurs fournissent des données sur les accidents, les chantiers et l'état des routes.

Ces données, collectées en temps réel, offrent une vue détaillée et actuelle des conditions de circulation. En les exploitant, nous pouvons prédire les embouteillages de manière précise et proactive. L'utilisation de ces données issues des edgeboxes et des capteurs permet de mieux comprendre les dynamiques du trafic et de mettre en œuvre des stratégies efficaces pour sa gestion.

V.1.2.1 Objectifs du chapitre

Dans ce chapitre, nous avons pour objectif d'entraîner un modèle en utilisant les données historiques collectées à partir des edgeboxes et d'autres capteurs situés sur les routes de Paris. L'idée est d'utiliser ces données pour créer un modèle de prédiction de la congestion du trafic dans le but de pouvoir traiter de nouvelles données et effectuer des prédictions en temps réel sur notre infrastructure. Cela permettra d'améliorer la gestion du trafic urbain et de réduire les embouteillages de manière proactive. Pour cela , nous allons :

- **Présenter les différentes sources de données utilisées et les transformations établis** : Détailler les types de données collectées, leur prétraitement et leur exploration.
- **Décrire les modèles d'apprentissage automatique choisis** : Expliquer les modèles considérés.
- **Détailler le processus d'entraînement des modèles** : Présenter les métriques d'évaluation et l'optimisation des hyperparamètres.
- **Évaluer les performances des modèles** : Analyser les résultats obtenus, comparer les performances des différents modèles et interpréter les métriques de performance.
- **Aborder le déploiement du modèle** : Décrire l'intégration du modèle dans l'écosystème et expliquer comment le flux de données sera géré.

V.1.3 Source et prétraitement des données

V.1.3.1 Sources de données

Pour développer des modèles de prédiction de la congestion du trafic, nous avons utilisé plusieurs sources de données fournies par l'entreprise. Ces données couvrent divers aspects du réseau routier de Paris et sont essentielles pour une analyse complète de la situation du trafic.

V.1.3.2 Données des routes

Les données des routes comprennent des informations détaillées sur les différentes voies de circulation de Paris. Cela inclut des identifiants uniques pour chaque route, les noms des routes, ainsi que des données géographiques précises permettant de localiser chaque route sur une carte. Ces informations sont cruciales pour comprendre la structure du réseau routier et pour identifier les principales artères de circulation.

V.1.3.3 Données de comptage routier permanent

Les données de comptage routier permanent fournissent des informations sur le volume de trafic, la vitesse moyenne des véhicules, et l'état de la circulation à différents moments. Ces données nous ont permis de quantifier le niveau de congestion sur certaines routes et d'identifier les périodes de pointe où le trafic est le plus intense.

Les paramètres enregistrés comprennent le taux d'occupation des routes à des intervalles réguliers, qui indiquent le niveau de congestion du trafic ainsi que la date et heure du comptage.

Le taux d'occupation des routes se traduit comme suit :

$0\% \leq K < 15\%$	Fluide
$15\% \leq K < 30\%$	Pré-saturé
$30\% \leq K < 50\%$	Saturé
$50\% \leq K$	Bloqué

TAB. V.1 : Taux d'occupation des routes

V.1.3.4 Données des accidents

Les données sur les accidents de circulation incluent des informations détaillées sur chaque accident, telles que la date, l'heure et l'emplacement géographique des accidents. Ces données permettent d'identifier les zones à haut risque. Elles sont essentielles pour la planification de mesures de sécurité routière et pour l'analyse des impacts des accidents sur la congestion du trafic.

V.1.3.5 Données des chantiers perturbant la circulation

Les données sur les chantiers incluent des informations sur les travaux routiers en cours, tels que la nature des travaux, les voies affectées, et la période durant laquelle les travaux ont lieu. Ces informations permettent de repérer les zones où des travaux perturbent la circulation, ce qui peut entraîner des retards et des congestions.

En utilisant ces différentes sources de données, nous avons pu avoir une vue d'ensemble du réseau routier de Paris, de sa congestion, des accidents et des perturbations liées aux travaux. Ces données ont été essentielles pour évaluer l'impact des problèmes liés aux trajets et ont servi de base pour le développement d'un modèle de prédiction de la congestion du trafic.

Voici le tableau qui représente les données utilisées :

Chapitre V. Etude de cas de la congestion du trafic en temps réel

	jour_accident integer	mois_accident integer	annee_accident integer	heure_accident time without time zone	geom_accident geometry	route_id integer	libelle text	date_heure_comptage timestamp with time zone	taux_occupation numeric
1	0	0	0	0 00:00:00	[null]	3915	Av_Champs_Elysees	2023-05-18 20:00:00+00	29.5827800000
2	0	0	0	0 00:00:00	[null]	16569	Bd_Poniatowski	2023-05-18 07:00:00+00	2.1616700000
3	0	0	0	0 00:00:00	[null]	15042	Av_Jean_Jaures	2023-05-18 08:00:00+00	10.1172200000
4	0	0	0	0 00:00:00	[null]	2012	PE_Malliot	2023-05-17 23:00:00+00	5.7500000000
5	0	0	0	0 00:00:00	[null]	6558	Claude_Bernard	2023-05-18 05:00:00+00	0.9438900000
6	0	0	0	0 00:00:00	[null]	26032	Bd_Victor	2023-05-18 15:00:00+00	4.0133400000
7	0	0	0	0 00:00:00	[null]	12007	Bd_Vincent_Auriol	2023-05-18 20:00:00+00	1.8383300000
8	0	0	0	0 00:00:00	[null]	10541	Bd_de_Charonne	2023-05-18 06:00:00+00	0.8777800000
9	0	0	0	0 00:00:00	[null]	13785	Bd_Voltaire	2023-05-18 08:00:00+00	2.9950000000
10	0	0	0	0 00:00:00	[null]	8973	Av_Kleber	2023-05-18 09:00:00+00	3.1077800000
11	0	0	0	0 00:00:00	[null]	26063	Bd_Victor	2023-05-18 18:00:00+00	2.0288900000
12	19	5	2023	13:15:00	0101000020E6100000AA60545227A0C93F97395D1613FB4740	5852	Av_de_la_Pte_de_St...	2023-05-18 22:00:00+00	2.5416700000
13	0	0	0	0 00:00:00	[null]	7388	Qual_Panhard_Leva...	2023-05-18 20:00:00+00	4.0705600000

FIG. V.1 : 1/3 Tableau représentant les données utilisées

etat_trafic text	etat_barre text	date_debut date	date_fin date	geom_route geometry	id_chantier text	objet text	description text	voie text
Pré-saturé	Invalide	1996-10-10	2023-01-01	0105000020E61000000102000000020000009840FC97107823040925060053F4A4840D55FE172E5240340B479E0B846A4840...	CP001838	REHABILITATION_IMMUEBLE	Garage du 11/02/21 au 30/06/21	rue Vernet
Fluide	Invalide	1996-10-03	2023-01-01	0105000020E61000000100000001020000000500000086C97107823040925060053F4A4840D55FE172E5240340B479E0B846A4840...	0	0	0	0
Fluide	Invalide	1996-10-17	2023-01-01	0105000020E61000000100000001020000000200000022FDFB911220340F11C90188B7148401E59913E16200340348851C1B0714840...	0	0	0	0
Fluide	Ouvert	1996-11-05	2023-01-01	0105000020E610000001000000010200000005000000382B5A48F44002403D59850B80704840B9AD1D18F93F024037CD67778B704840...	0	0	0	0
Fluide	Invalide	1996-10-17	2023-01-01	0105000020E610000001000000010200000000085859697C4024024167938B0648401DA90B50FEC602402C97E94E7E684840...	0	0	0	0
Fluide	Invalide	1996-10-03	2023-01-01	0105000020E610000001000000010200000002000000260568588A3F024077F3AAAF16A48409C9D1A001F3A0240256F4770B684840...	0	0	0	0
Fluide	Invalide	1996-10-21	2023-01-01	0105000020E6100000010000000102000000020000008B90AF08BCB0240C8F89BC2896A4840663B8F65CF002400538C49ED26A4840...	0	0	0	0
Fluide	Invalide	1996-10-17	2023-01-01	0105000020E61000000100000001020000000200000028EC1A01842F0340EFAE846F56C48400C1BE2983C300340D0521CE8D076C4840...	0	0	0	0
Fluide	Invalide	2000-10-26	2023-01-01	0105000020E6100000010000000102000000020000002598B158A8030340A64DFD7D0A6E4840750A3E832707034038B16F7CE96D4840...	0	0	0	0
Fluide	Invalide	[null]	[null]	[null]	0	0	0	0
Fluide	Invalide	1996-10-03	2023-01-01	0105000020E61000000100000001020000000200000067565CFC0390240122D31F0016B4840A2D68C8603F0240C687C109EC6A4840...	0	0	0	0
Fluide	Invalide	1996-10-16	2023-01-01	0105000020E61000000100000001020000000200000080FB4220005024098BF49B0546B4840448BEF15540802406F351FD33506B4840...	0	0	0	0
Fluide	Invalide	1996-10-22	2023-01-01	0105000020E610000001000000010200000002000000D125F826D30C3402F76E450616A4840E19DE0D5E30903403132F477856A4840...	0	0	0	0

FIG. V.2 : 2/3 Tableau représentant les données utilisées

date_debut_chantier text	date_fin_chantier text	impact_circulation text	geom_chantier geometry	accidents integer	chantiers integer	etattrafic integer
2021-02-11	2023-03-31	0	0101000020E6100000AA4061792C6502404D2036B0906F4840	0	1	1
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	0	0	0
0	0	0	[null]	1	0	0
0	0	0	[null]	0	0	0

FIG. V.3 : 3/3 Tableau représentant les données utilisées

Voici une description cartographique des données de Paris :

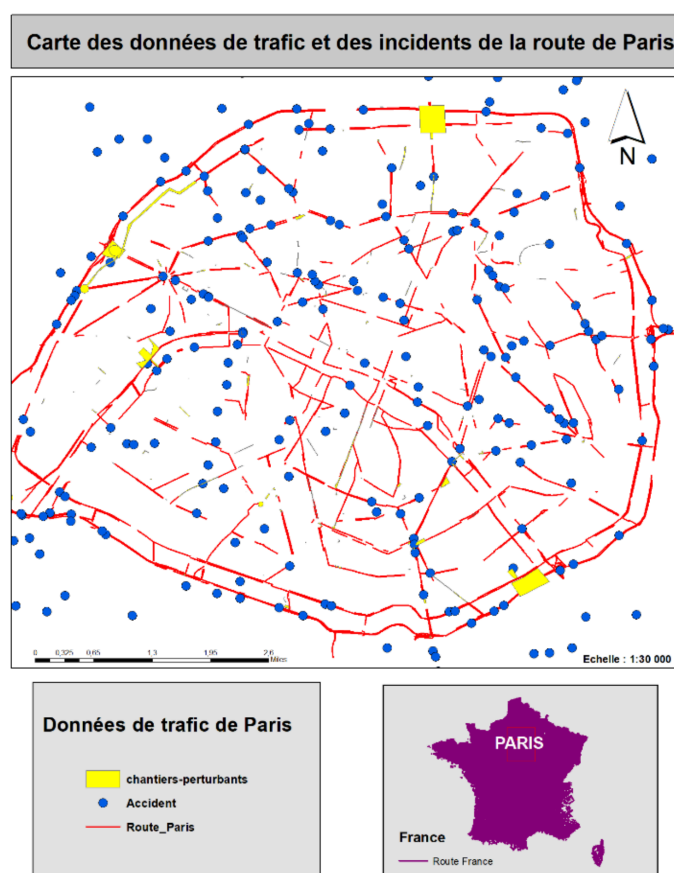


FIG. V.4 : Description cartographique des données de Paris

V.1.3.6 Données des véhicules (Edgebox)

Les données des véhicules sont collectées à partir d'appareils Edgebox installés sur les routes et les intersections. Ces dispositifs captent des informations en temps réel sur les véhicules, telles que la vitesse, le nombre de véhicules, les types de véhicules et leur direction. Les Edgeboxes utilisent diverses technologies, y compris des capteurs infrarouges, des caméras et des dispositifs de comptage de véhicules, pour recueillir des données précises et fiables.

V.1.4 Exploration et Visualisation des Données

L'exploration et la visualisation des données sont des étapes cruciales dans tout projet d'analyse de données. Elles permettent de saisir les caractéristiques, les tendances et les relations au sein des données, offrant ainsi des insights précieux pour orienter les analyses ultérieures et la prise de décision. Cette section présente une exploration approfondie des données, mettant en avant les principales observations et les tendances marquantes.

V.1.4.1 Statistiques descriptives

Les statistiques descriptives offrent un premier aperçu des données, fournissant des mesures de centralité, de dispersion et de distribution. Voici quelques points saillants :

- Le taux d'occupation moyen est estimé à 5.90, avec une variation significative allant jusqu'à 98.40, illustrant la diversité des situations observées.
- Les données, telles que la date et l'heure de comptage, montrent une concentration autour d'une période spécifique, indiquant potentiellement une période d'échantillonnage spécifique.

V.1.4.2 Distribution du risque de congestion

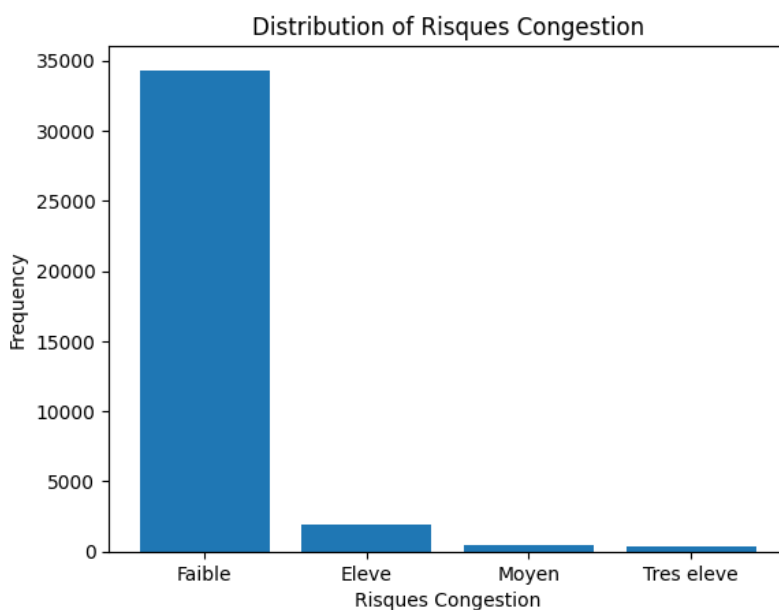


FIG. V.5 : Distribution du risque de congestion

On remarque un déséquilibre entre les différentes classes de congestion, chose qu'il faut considérer lors de l'entraînement de notre modèle.

V.1.4.3 Distribution du Taux d'Occupation

Une analyse plus approfondie de la distribution du taux d'occupation révèle :

- Une distribution légèrement inclinée vers la droite, avec une médiane de 3.21 et une moyenne de 5.90, suggérant une prévalence de valeurs plus faibles avec une dispersion significative.

- Des variations substantielles dans les taux d'occupation sont observées entre différents états de trafic, allant de fluide à bloqué.

V.1.4.4 Taux d'occupation par état de trafic

Les données sont réparties en quatre états de trafic : Bloqué, Fluide, Pré-saturé et Saturé. Les taux d'occupation moyens varient considérablement entre ces états.

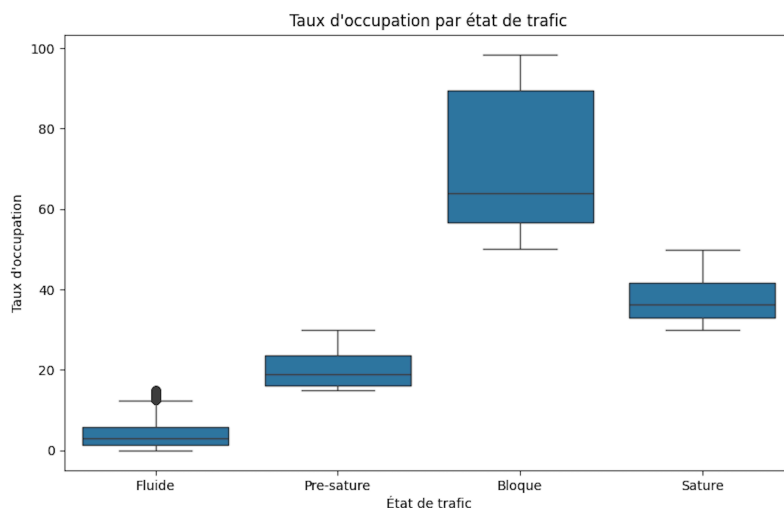


FIG. V.6 : Taux d'occupation par état de trafic

V.1.4.5 Taux d'Occupation par Heure

- Le taux d'occupation moyen est plus élevé pendant la nuit (23h47-01h53) et diminue progressivement jusqu'à 05:00, avant d'augmenter à nouveau.

V.1.4.6 Matrice de Corrélation

Identification des relations entre les différentes variables pour informer la sélection des caractéristiques pertinentes pour les modèles.

Ces analyses initiales fournissent un cadre solide pour une exploration plus approfondie, mettant en évidence les domaines clés qui méritent une attention particulière dans le processus d'analyse des données.

V.2 Nettoyage et préparation des données

Le tableau suivant montre la sélection des attributs (caractéristiques qui nous aideront à prédire la congestion du trafic), leur type et les transformations qu'ils vont subir.

Attribut	Type initial	Transformation appliquée
Date_Heure	object	Converti en datetime et suppression du fuseau horaire. Séparé en date et heure.
taux_d_occupation	float64	Aucune transformation nécessaire.
etat_trafic	object	Variable cible, encodage en utilisant LabelEncoder
etat_barre	object	Encodage en variables binaires avec OneHotEncoder.
jour_accident	int64	Aucune transformation nécessaire.
mois_accident	int64	Aucune transformation nécessaire.
annee_accident	int64	Aucune transformation nécessaire.
Heure_minute_accident	object	Converti en datetime.time. Extraction des heures, minutes et secondes.
date_debut_chantier	object	Converti en datetime. Extraction des composants de date (année, mois, jour).
date_fin_chantier	object	Converti en datetime. Extraction des composants de date (année, mois, jour).
impact_circ_chantier	object	Encodage en variables binaires avec OneHotEncoder.
accidents	int64	Aucune transformation nécessaire.
chantiers	int64	Aucune transformation nécessaire.
weekday	object	Extrait du jour de la semaine à partir de 'date_heure_comptage'.
Route_id	int64	Aucune transformation nécessaire.

TAB. V.2 : Sélection des attributs et transformations appliquées

V.3 Choix et Entrainement des modèles d'apprentissage automatique

Dans le cadre de la prédiction de la congestion du trafic et pour capturer les relations non linéaires et complexes présentes dans ces données, nous expérimentons avec différents modèles : les Machines à Vecteurs de Support (SVM), la Forêt d'arbres décisionnels (Random Forest) et les Gated Recurrent Unit (GRU), pour voir lequel est le plus adapté à notre cas.

V.3.1 Machines à Vecteurs de Support

Les Machines à Vecteurs de Support (SVM) sont des modèles d'apprentissage supervisé efficaces pour la classification et la régression. Elles sont particulièrement

adaptées aux problèmes où les données sont complexes et non linéaires, en utilisant des noyaux pour projeter les données dans un espace de plus haute dimension où elles peuvent être séparées linéairement. Nous avons choisi d'utiliser les SVM dans notre cas pour la prédiction de la congestion du trafic en raison de leur robustesse et de leur capacité à gérer des espaces de grande dimension.

V.3.1.1 Architecture

Les SVM fonctionnent en trouvant l'hyperplan optimal qui sépare les différentes classes dans les données d'entraînement. En utilisant des noyaux, les SVM peuvent créer des frontières de décision non linéaires.

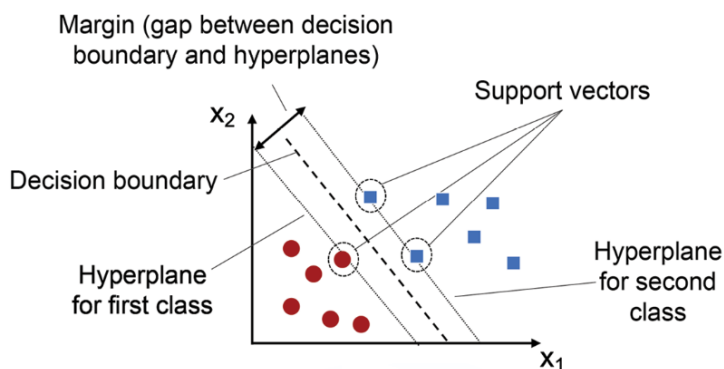


FIG. V.7 : Architecture de Model SVM

V.3.1.2 Hyperparamètres

Hyperparamètre	Valeur
Kernel	RBF
C (Regularization)	1.0
Gamma	Scale (default)
Random State	42
Test Size	0.2
Validation Split	0.2
Data Shuffling Seed	42

TAB. V.3 : Hyperparamètres du modèle SVM

V.3.1.3 Prédiction du modèle SVM

Metric	Faible	Élevé	Moyen	Très Élevé	Overall
Precision	0.90	0.99	0.98	1.00	-
Recall	0.78	1.00	0.78	0.97	-
F1-Score	0.84	0.99	0.86	0.99	-
Accuracy	-	-	-	-	0.98

TAB. V.4 : Prédiction du modèle SVM

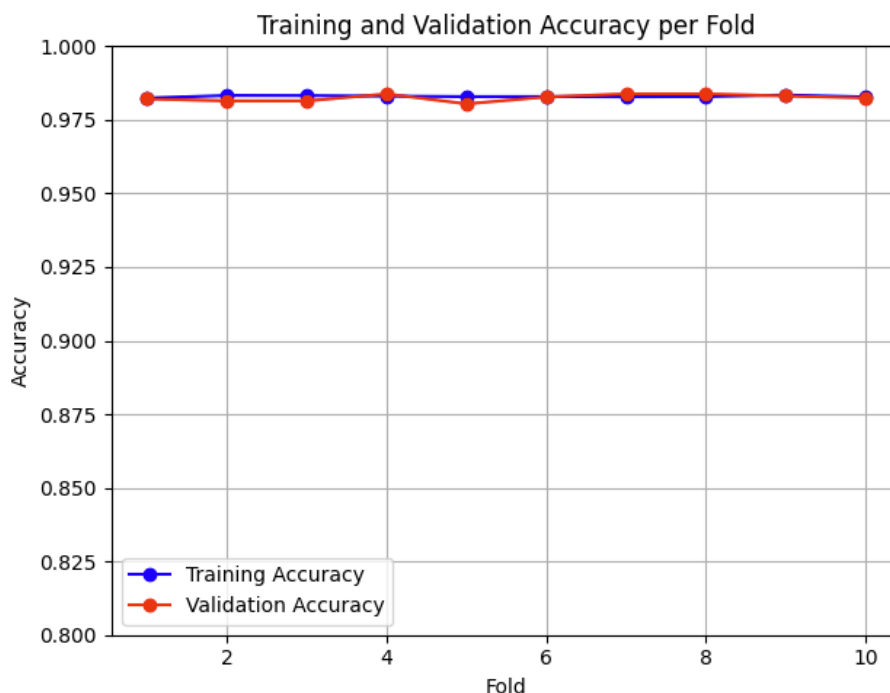


FIG. V.8 : Resultat de model SVM

V.3.2 K-Nearest Neighbors (KNN)

Les algorithmes de K-Nearest Neighbors (KNN) sont des modèles d'apprentissage supervisé utilisés pour les tâches de classification et de régression. Ils sont particulièrement adaptés aux problèmes où la proximité des échantillons dans l'espace des caractéristiques est indicative de leur classe. Le KNN est choisi pour la prédiction de la congestion du trafic en raison de sa simplicité et de son efficacité pour des ensembles de données de taille modérée.

V.3.2.1 Architecture KNN

L'algorithme KNN fonctionne en trouvant les k voisins les plus proches d'un point de données donné dans l'espace des caractéristiques et en attribuant la classe majoritaire parmi ces voisins. Pour les tâches de classification, la distance euclidienne est souvent utilisée pour mesurer la proximité entre les points de données. Voici une

représentation de l'architecture d'un KNN

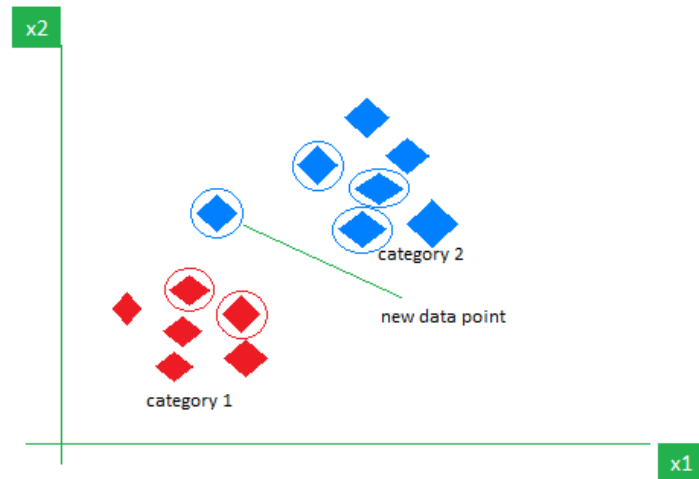


FIG. V.9 : l'architecture de model KNN

V.3.2.2 Hyperparamètres

Hyperparamètre	Valeur
k (Number of Neighbors)	5
Metric	Euclidean
Weight	Uniform
Algorithm	Auto
Test Size	0.2
Validation Split	0.2
Data Shuffling Seed	42

TAB. V.5 : Hyperparamètres du modèle KNN

V.3.2.3 Prédiction du modèle KNN

Metric / risque de congestion	Faible	Élevé	Moyen	Très élevé	Overall
Precision	0.89	0.99	0.91	1.00	-
Recall	0.79	1.00	0.82	0.88	-
F1-Score	0.84	0.99	0.86	0.94	-
Test Accuracy	-	-	-	-	0.9834

TAB. V.6 : Prédiction du modèle KNN

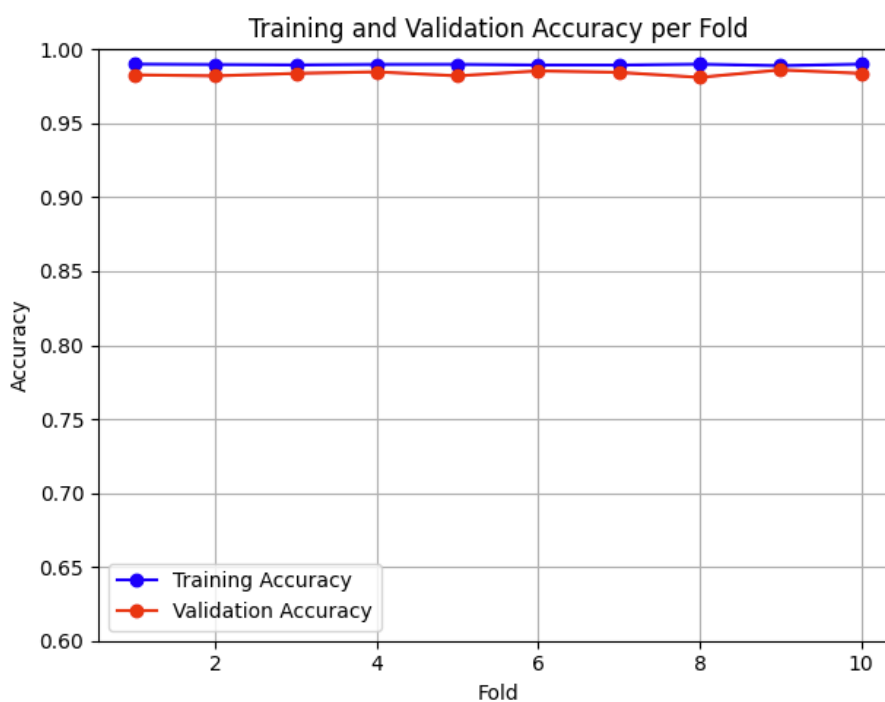


FIG. V.10 : Resultat de KNN

V.3.3 Gated Recurrent Unit (GRU)

Le choix d'un modèle GRU (Gated Recurrent Unit) pour prédire le flux de trafic d'une route particulière est justifié par plusieurs facteurs clés, en particulier lorsqu'il s'agit de prendre en compte à la fois les données de trafic actuelles et historiques de cette route spécifique, ainsi que les données de flux de trafic historiques et actuelles des segments de routes voisines qui peuvent être interdépendants.

V.3.3.1 Architecture GRU

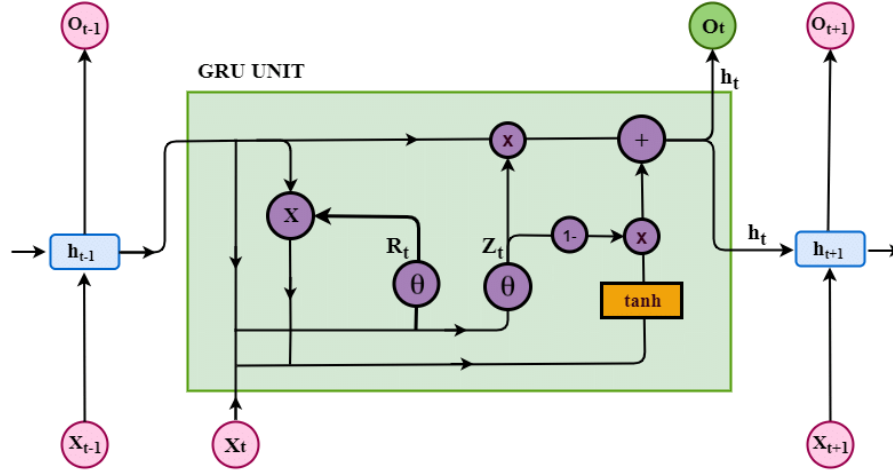


FIG. V.11 : l'architecture de model GRU

Une cellule GRU utilise des portes pour contrôler précisément quelles informations doivent être retenues ou oubliées au fil du temps. Cela permet d'apprendre des relations complexes dans les données, telles que la prédiction de la congestion du trafic en se basant sur des données historiques et actuelles comprenant le taux d'occupation, l'état du trafic, ainsi que des événements tels que les accidents et les travaux.

V.3.3.2 Hyperparamètres

Pour assurer un apprentissage équilibré et améliorer la robustesse du modèle, nous avons utilisé SMOTE (Synthetic Minority Over-sampling Technique) pour traiter les données de congestion du trafic déséquilibrées. SMOTE génère des échantillons synthétiques pour les classes minoritaires en interpolant entre les exemples existants, ce qui permet d'équilibrer la distribution des classes dans le jeu de données. Ce prétraitement est crucial pour éviter que le modèle ne soit biaisé vers les classes majoritaires et garantir une performance plus uniforme sur toutes les classes de risque de congestion.

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \underbrace{\lambda \sum_{j=1}^n \theta_j^2}_{\text{Regularization Term}}$$

↑ **Regularization Parameter**
← **start at θ_1**

FIG. V.12 : Fonction de coût régularisée avec L2

Hyperparameter	Value
Sequence Length	10
GRU Units	100 (for each GRU layer)
Dropout Rate	0.2
Dense Units	4
Activation (Dense)	Softmax
Learning Rate	0.001
Optimizer	Adam
Regularization	L2
Loss Function	Sparse Categorical Crossentropy
Metrics	Accuracy
Epochs	50
Batch Size	32
Validation Split	0.2
Test Size	0.2
SMOTE Random State	42
Data Shuffling Seed	42

TAB. V.7 : Hyperparamètres du modèle GRU

Cette table résume les hyperparamètres clés utilisés dans votre architecture de modèle GRU pour la prédiction de la congestion du trafic.

V.3.3.3 Prédiction du modèle GRU

Metric / Risque de Congestion	Faible	Élevé	Moyen	Très Élevé	Overall
Precision	0.93	0.98	0.98	0.99	-
Recall	0.98	0.90	1.00	1.00	-
F1-Score	0.95	0.94	0.99	0.99	-
Accuracy	-	-	-	-	0.97

TAB. V.8 : Prédiction du modèle GRU

Ce tableau présente la précision, le rappel, le score F1 et le support pour chaque classe de risque de congestion.

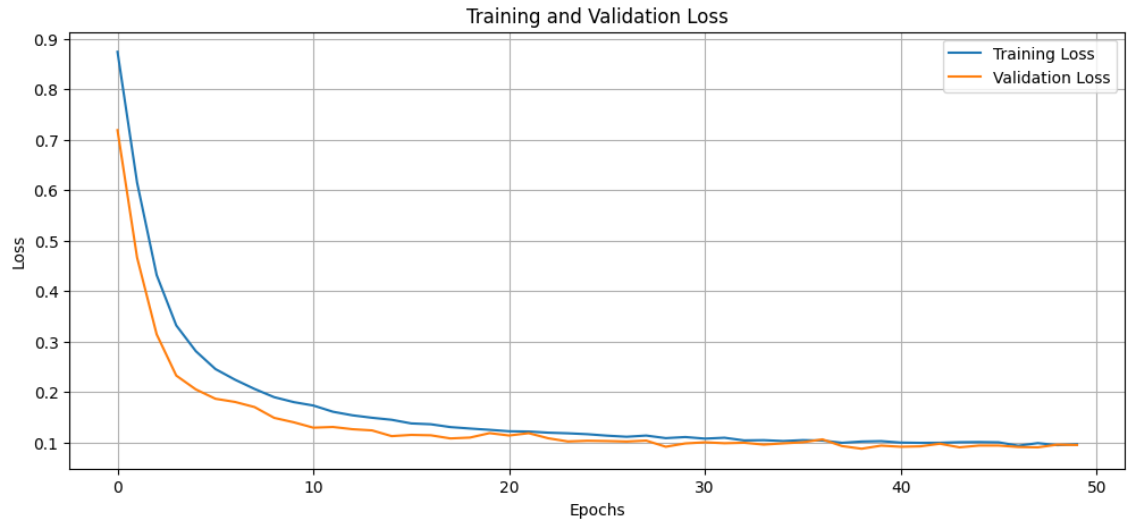


FIG. V.13 : Evolution de la fonction d'erreur en fonction d'epochs (GRU)

V.4 Interprétation des métriques de performance

V.4.1 Métriques d'évaluation

Vu que c'est un problème de classification, nous allons utiliser les métriques suivantes :

Métrique	Définition	Formule
Précision	La proportion de prédictions positives correctes parmi l'ensemble des prédictions positives.	$\frac{TP}{TP+FP}$
Rappel	La proportion des vrais positifs détectés parmi l'ensemble des échantillons positifs.	$\frac{TP}{TP+FN}$
F1 Score	La moyenne harmonique de la précision et du rappel.	$2 \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$
Exactitude	La proportion de prédictions correctes parmi l'ensemble des prédictions.	$\frac{TP+TN}{TP+TN+FP+FN}$

TAB. V.9 : Métriques d'évaluation

V.4.2 Comparaison entre les modèles

Metric / risque de congestion	Faible	Élevé	Moyen	Très élevé	Overall
Precision					
SVM	0.90	0.99	0.98	1.00	-
GRU	0.93	0.98	0.98	0.99	-
KNN	0.89	0.99	0.91	1.00	-
Recall					
SVM	0.78	1.00	0.78	0.97	-
GRU	0.98	0.90	1.00	1.00	-
KNN	0.79	1.00	0.82	0.88	-
F1-Score					
SVM	0.84	0.99	0.86	0.99	-
GRU	0.95	0.94	0.99	0.99	-
KNN	0.84	0.99	0.86	0.94	-
Accuracy					
SVM	-	-	-	-	0.98
GRU	-	-	-	-	0.97
KNN	-	-	-	-	0.98

TAB. V.10 : Comparaison entre les modèles

Dans l'ensemble, les modèles SVM, GRU et KNN montrent de bonnes performances pour la prédiction du risque de congestion. Cependant, le modèle GRU semble surpasser les autres dans la plupart des métriques, notamment pour les niveaux de risque faible et moyen, ce qui en fait un candidat prometteur pour la prédiction de la congestion du trafic. Les résultats suggèrent que, bien que SVM et KNN soient compétitifs, GRU offre une meilleure balance entre précision et rappel, particulièrement pour les niveaux de risque moyen et très élevé. De plus, le modèle GRU est capable de mieux gérer de grandes quantités de données, ce qui est essentiel pour traiter les vastes ensembles de données générés en temps réel par les dispositifs IoT et les capteurs de trafic.

V.5 Intégration du modèle dans l'écosystème existant

Après avoir entraîné le modèle de prédiction de la congestion du trafic avec les données historiques, nous l'intégrerons dans notre écosystème de big data pour traiter les nouvelles données en temps réel. Voici comment cela sera réalisé :

- **Collecte des données en temps réel** : Les edgeboxes placés dans les véhicules et les capteurs installés sur les routes collectent des données en temps réel sur le trafic, les conditions routières, les accidents, et plus encore.
- **Ingestion des données avec Kafka** : Les données collectées sont envoyées vers Apache Kafka, qui agit comme un système de messagerie pour gérer le flux continu de données en temps réel.
- **Traitement des données avec Flink** : Apache Flink consomme les données provenant des topics Kafka. Flink effectue le traitement nécessaire sur les données et les dirige vers les différents sinks configurés. Ces sinks peuvent envoyer les données transformées vers MongoDB, Elasticsearch, ou directement vers le modèle de prédiction.
- **Déploiement du modèle avec Flask** : Le modèle de prédiction est déployé via une application Flask. Le modèle consomme les données provenant d'un des sinks de Flink et effectue des prédictions basées sur ces données.
- **Stockage des résultats avec MongoDB** : Le modèle prédit la congestion du trafic et renvoie un output JSON. Cet output JSON est ensuite stocké dans MongoDB pour une consultation et une analyse ultérieures.

Ce flux de travail intégré assure que les nouvelles données sont collectées, traitées et analysées de manière continue, permettant des prédictions de congestion précises et en temps réel pour une gestion optimale du trafic urbain.

V.6 Conclusion

L'intégration de l'apprentissage automatique dans notre écosystème Big Data permet non seulement de traiter et d'analyser de grandes quantités de données en temps réel, mais aussi de fournir des prévisions précises pour la gestion proactive du trafic. Cela souligne l'importance d'un écosystème bien conçu pour soutenir les flux de données et les applications de machine learning, permettant ainsi d'améliorer la gestion de la congestion urbaine de manière significative.

CONCLUSION ET PERSPECTIVES

Dans le domaine du Big Data, où les données sont une ressource extrêmement précieuse, il est crucial de disposer d'un écosystème capable de gérer différents types de données et de répondre à diverses exigences de traitement. Notre projet de mise en place d'un écosystème Big Data répond parfaitement à ces besoins, offrant une solution complète et robuste adaptée aux défis contemporains de la gestion des données.

L'écosystème mis en place permet de collecter, d'ingérer, de stocker et de traiter efficacement les données provenant de multiples sources, qu'il s'agisse de flux de données en temps réel ou de volumes importants de données historiques. Grâce à des capacités de traitement en temps réel et par lots, il assure une gestion fluide et cohérente des données, garantissant ainsi leur disponibilité et leur intégrité à chaque étape du processus. Cette flexibilité permet d'utiliser les données pour des analyses avancées, des prédictions de machine learning et des applications variées, répondant ainsi aux besoins diversifiés des entreprises modernes.

En intégrant des composants clés tels qu'Apache Kafka pour la gestion des flux de données en temps réel, Apache Flink pour le traitement en temps réel, et Apache Spark pour le traitement par lots à grande échelle, notre écosystème assure un flux de données continu et structuré. Ces composants interagissent harmonieusement pour offrir une plateforme capable de traiter des volumes massifs de données avec une latence minimale, permettant ainsi une prise de décision rapide et basée sur des données actualisées. De plus, l'utilisation de systèmes de stockage robustes comme HDFS pour le stockage distribué et de bases de données NoSQL telles que MongoDB pour la gestion des données semi-structurées garantit une accessibilité et une fiabilité maximales des données.

Notre écosystème Big Data n'est pas seulement techniquement avancé, mais il est également conçu pour être évolutif et adaptable. Il peut facilement intégrer de nouveaux composants et technologies au fur et à mesure que les besoins de l'entreprise évoluent, assurant ainsi une pérennité et une adaptabilité à long terme. Cette capacité d'évolution est cruciale dans un environnement technologique en constante mutation, où les exigences en matière de traitement et d'analyse des données peuvent changer rapidement.

En outre, cet écosystème permet une exploitation optimale des données pour des applications d'intelligence artificielle et de machine learning. En offrant une plateforme solide pour l'entraînement et le déploiement de modèles de machine learning,

ning, il ouvre la voie à des innovations significatives dans divers domaines tels que la détection de fraudes, la personnalisation de l'expérience client, et l'optimisation des chaînes d'approvisionnement. Les capacités d'analyse avancée permettent également de dériver des insights précieux à partir des données, facilitant ainsi la prise de décisions stratégiques et informées.

En somme, notre écosystème Big Data est une solution complète et robuste qui répond aux défis actuels de la gestion des données, tout en offrant des perspectives prometteuses pour l'avenir. Il permet aux entreprises de tirer pleinement parti de leurs données, en les transformant en un atout stratégique qui peut booster la compétitivité et l'efficacité opérationnelle. Grâce à cet écosystème, les entreprises peuvent non seulement répondre aux exigences actuelles en matière de traitement des données, mais également se préparer aux défis futurs, assurant ainsi une croissance durable et soutenue dans l'ère numérique.

BIBLIOGRAPHIE

- [1] Yu ZHENG. “Methodologies for cross-domain data fusion : An overview”. In : *IEEE transactions on Big Data* 1.1 (2015), p. 16-34.
- [2] Mitra BARATCHI, Nirvana MERATNIA et Paul JM HAVINGA. “On the use of mobility data for discovery and description of social ties”. In : *Proceedings of the 2013 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. ACM, 2013, p. 5.
- [3] Minho JO et al. “A survey of converging solutions for heterogeneous mobile networks”. In : *IEEE Wireless Communications* 21.6 (2014), p. 54-62.
- [4] Han GUO et al. “An event-driven dynamic updating method for 3D geodatabases”. In : *Geo-spatial Information Science* (2016), p. 1-8.
- [5] Xuejun DING, Yong TIAN et Yan YU. “A real-time Big data gathering algorithm based on indoor wireless sensor networks for risk analysis of industrial operations”. In : *IEEE Transactions on Industrial Informatics* 12.3 (2016), p. 1232-1242.
- [6] R. M. SIMMONDS et al. “A Platform for Analysing Stream and Historic Data with Efficient and Scalable Design Patterns”. In : *2014 IEEE World Congress on Services*. IEEE, 2014, p. 174-181.
- [7] Nir KSHETRI. “Big Data’s Impact on Privacy, Security and Consumer Welfare”. In : *Telecommunications Policy* 38.11 (2014), p. 1134-1145.
- [8] H. SONG, G. FINK et S. JESCHKE. *Security and Privacy in Cyber-Physical Systems : Foundations and Applications*. England, UK : Wiley, 2017.
- [9] Christoph STRAUCH. “NoSQL Databases”. In : *Lecture Notes on Data Engineering and Communications Technologies* (2011).
- [10] Katarina GROLINGER, Miriam A M CAPRETZ et Subash MAZUMDAR. “Security Issues in Big Data”. In : *Proceedings of the 2013 IEEE International Congress on Big Data*. IEEE, 2013, p. 791-795.
- [12] Giuseppe ATENIESE et al. “Improved Proxy Re-Encryption Schemes with Applications to Secure Distributed Storage”. In : *ACM Transactions on Information and System Security (TISSEC)*. T. 9. 1. ACM, 2006, p. 1-30.
- [13] Matthew GREEN et Giuseppe ATENIESE. “Identity-Based Proxy Re-Encryption”. In : *Applied Cryptography and Network Security*. Springer, 2007, p. 288-306.

- [14] Vipul GOYAL et al. "Attribute-Based Encryption for Fine-Grained Access Control of Encrypted Data". In : *Proceedings of the 13th ACM Conference on Computer and Communications Security*. ACM, 2006, p. 89-98.
- [15] Shucheng YU et al. "Defending Against Key Abuse Attacks in KP-ABE Enabled Broadcast Systems". In : *IEEE Transactions on Information Forensics and Security* 5.3 (2010), p. 402-415.
- [16] Kaitai LIANG et al. "An Efficient Cloud-Based Revocable Identity-Based Proxy Re-Encryption Scheme for Public Clouds Data Sharing". In : *Future Generation Computer Systems* 30 (2013), p. 120-132.
- [17] L. CHEUNG et C. NEWPORT. "Provably Secure Ciphertext Policy ABE". In : *Proceedings of the 14th ACM Conference on Computer and Communications Security*. ACM, 2007, p. 456-465.
- [18] B. Clifford NEUMAN et Theodore Ts'o. "Kerberos : An Authentication Service for Computer Networks". In : *IEEE Communications Magazine* 32.9 (1994), p. 33-38.
- [19] Ragib HASAN, Radu SION et Marianne WINSLETT. "The Case of the Fake Picasso : Preventing History Forgery with Secure Provenance". In : *Proceedings of the 7th USENIX Conference on File and Storage Technologies*. USENIX Association, 2009, p. 1-14.
- [20] Kaiming HE et al. "Delving Deep into Rectifiers : Surpassing Human-Level Performance on ImageNet Classification". In : *Proceedings of the IEEE International Conference on Computer Vision*. IEEE, 2015, p. 1026-1034.
- [33] Jay KREPS, Neha NARKHEDE et Jun RAO. "Kafka : a distributed messaging system for log processing". In : (2011).
- [34] Patrick HUNT et al. "Zookeeper : Wait-free coordination for internet-scale systems". In : *USENIX annual technical conference*. T. 8. 2010.
- [35] Martin KLEPPMANN. *Designing Data-Intensive Applications : The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, Inc., 2017.
- [38] Paris CARBONE et al. "Apache Flink : Stream and batch processing in a single engine". In : *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*. T. 36. 4. 2015, p. 28-38.
- [39] Matei ZAHARIA et al. "Apache Spark : a unified engine for big data processing". In : *Communications of the ACM* 59.11 (2016), p. 56-65.
- [40] Konstantin SHVACHKO et al. "The Hadoop Distributed File System". In : *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*. Ieee. 2010, p. 1-10.
- [41] Ashish THUSOO et al. "Hive : a petabyte scale data warehouse using Hadoop". In : *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*. Ieee. 2010, p. 996-1005.

Bibliographie

- [43] Clinton GORMLEY et Zachary TONG. *Elasticsearch : The Definitive Guide : A Distributed Real-Time Search and Analytics Engine*. O'Reilly Media, Inc., 2015.
- [44] Scott FARRIS. *Learning Kibana 5.0*. Packt Publishing Ltd, 2016.
- [45] Kyle BANKER. *MongoDB in action*. Manning Publications Co., 2011.
- [46] Bruce MOMJIAN. *PostgreSQL : introduction and concepts*. Addison-Wesley Longman Publishing Co., Inc., 2001.
- [47] James TURNBULL. *The Prometheus monitoring system and time series database*. Turnbull Press, 2018.

WEBGRAPHIE

- [11] Cloud Security ALLIANCE. *Expanded Top Ten Big Data Security and Privacy Challenges*. <https://cloudsecurityalliance.org/artifacts/expanded-top-ten-big-data-security-and-privacy-challenges/>. 2012.
- [21] CONFLUENT. *Confluent Kafka Connect*. <https://docs.confluent.io/platform/current/connect/index.html>. 2024.
- [22] Apache Software FOUNDATION. *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>. 2024.
- [23] Apache Software FOUNDATION. *Apache Flink Documentation*. <https://flink.apache.org/docs/>. 2024.
- [24] Elastic NV. *Elasticsearch Documentation*. <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>. 2024.
- [25] Elastic NV. *Kibana Documentation*. <https://www.elastic.co/guide/en/kibana/current/index.html>. 2024.
- [26] Grafana LABS. *Grafana Documentation*. <https://grafana.com/docs/>. 2024.
- [27] MongoDB INC.. *MongoDB Documentation*. <https://docs.mongodb.com/>. 2024.
- [28] PostgreSQL Global Development GROUP. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/current/>. 2024.
- [29] Apache Software FOUNDATION. *Apache Hadoop HDFS Documentation*. <https://hadoop.apache.org/docs/r3.3.1/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html>. 2024.
- [30] Apache Software FOUNDATION. *Apache Hive Documentation*. <https://hive.apache.org/documentation/>. 2024.
- [31] Cloud Native Computing FOUNDATION. *Prometheus Documentation*. <https://prometheus.io/docs/>. 2024.
- [32] CONFLUENT. *ksqlDB Documentation*. <https://docs.ksqldb.io/en/latest/>. 2024.
- [36] REDPANDA. *Redpanda*. <https://redpanda.com/>. 2021.
- [37] CONFLUENT. *Schema Registry*. <https://docs.confluent.io/platform/current/schema-registry/index.html>. 2021.
- [42] Grafana LABS. *Grafana*. <https://grafana.com/>. 2021.

Annexes

A Définitions

A.1 Technologies

A.1.1 Apache Kafka

Apache Kafka est une plateforme de streaming d'événements distribuée capable de gérer des milliards d'événements par jour. Elle est conçue pour le traitement de données à haut débit et à faible latence, offrant une plateforme unifiée pour les pipelines de données en temps réel et les applications de streaming [33].

A.1.2 Apache Zookeeper

Apache Zookeeper est un service centralisé pour maintenir des informations de configuration, des noms, fournir une synchronisation distribuée et des services de groupe. Il est conçu pour être facile à programmer et à utiliser, et permet de construire des systèmes distribués fiables [34].

A.1.3 ksqlDB

ksqlDB est un moteur SQL de streaming pour Apache Kafka qui permet le traitement et la requête de données en temps réel sur des topics Kafka en utilisant une syntaxe de type SQL. Il simplifie le développement d'applications de traitement de flux en abstraissant les complexités de l'API Kafka Streams [35].

A.1.4 Redpanda

Redpanda est une plateforme de streaming moderne compatible avec l'API Kafka, conçue pour être une alternative plus simple, plus rapide et plus efficace. Elle offre des fonctionnalités telles que la surveillance en temps réel, la scalabilité et la fiabilité avec une charge opérationnelle réduite [36].

A.1.5 Schema Registry

Le Schema Registry fournit un référentiel central pour la gestion et la versioning des schémas utilisés dans les applications de streaming de données. Il garantit la compatibilité des données et la gouvernance en permettant l'évolution et l'application des schémas entre les producteurs et les consommateurs [37].

A.1.6 Kafka Connect

Kafka Connect est un outil pour le streaming de données évolutif et fiable entre Apache Kafka et d'autres systèmes de données. Il fournit un cadre pour connecter Kafka à des systèmes externes tels que des bases de données, des magasins clé-valeur, des index de recherche et des systèmes de fichiers [33].

A.1.7 Apache Flink

Apache Flink est un cadre de traitement de flux pour les applications de streaming de données distribuées, performantes, toujours disponibles et précises. Il offre une API unifiée pour le traitement des flux et des lots, permettant l'analyse en temps réel et l'apprentissage automatique [38].

A.1.8 Apache Spark

Apache Spark est un moteur analytique unifié pour le traitement de données à grande échelle, avec des modules intégrés pour SQL, le streaming, l'apprentissage automatique et le traitement de graphes. Il permet une analyse rapide, flexible et évolutive des grandes données [39].

A.1.9 Hadoop Distributed File System (HDFS)

HDFS est le système de stockage principal pour les applications Hadoop, fournissant un stockage de données distribué, évolutif et fiable conçu pour gérer de grands ensembles de données. Il assure une tolérance élevée aux pannes et l'intégrité des données à travers des clusters de matériel bon marché [40].

A.1.10 Apache Hive

Apache Hive est une solution d'entreposage de données construite sur Hadoop, permettant aux utilisateurs de requêter et d'analyser de grands ensembles de données stockés dans le HDFS de Hadoop en utilisant un langage de type SQL appelé HiveQL. Il facilite la synthèse des données et les requêtes ad hoc [41].

A.1.11 Grafana

Grafana est une plateforme open-source pour la surveillance et l'observabilité, permettant aux utilisateurs de créer et de partager des tableaux de bord dynamiques et de visualiser des métriques provenant de multiples sources de données en temps réel. Elle est hautement extensible avec une large gamme de plugins et d'intégrations [42].

A.1.12 Elasticsearch

Elasticsearch est un moteur de recherche et d'analyse distribué et RESTful capable de gérer une grande variété de types de données, y compris textuelles, numériques, géospatiales et structurées. Il offre des capacités puissantes de recherche en texte intégral et d'indexation des données en temps réel [43].

A.1.13 Kibana

Kibana est un outil open-source de visualisation et d'exploration de données pour Elasticsearch, offrant une interface graphique pour requêter, analyser et visualiser les données indexées dans Elasticsearch. Il prend en charge l'analyse en temps réel et les tableaux de bord interactifs [44].

A.1.14 MongoDB

MongoDB est une base de données NoSQL connue pour ses performances élevées, sa scalabilité et sa flexibilité. Elle stocke les données dans des documents de type JSON, facilitant le travail avec des structures de données hiérarchiques et permettant des requêtes et des indexations riches [45].

A.1.15 PostgreSQL

PostgreSQL est un système de gestion de bases de données relationnelles (SGBDR) avancé et open-source, connu pour sa robustesse, son extensibilité et sa conformité aux normes. Il prend en charge une large gamme de types de données, des requêtes complexes et des techniques d'indexation [46].

A.1.16 Prometheus

Prometheus est un kit d'outils open-source de surveillance et d'alerte conçu pour la fiabilité et la scalabilité. Il collecte et stocke des métriques sous forme de séries

temporelles, fournit un langage de requête puissant (PromQL) et s'intègre à divers outils de visualisation comme Grafana [47].

B Performance Benchmarks

Composant	Description
Kafka	Throughput : 1 million messages/seconde Latency : 2 ms Replication Latency : 5 ms Disk I/O Utilization : 70% Mesuré avec 3 brokers, chacun gérant 333 000 messages par seconde. Latence moyenne de bout en bout pour la livraison des messages sous charge maximale. Temps nécessaire pour répliquer les messages entre tous les brokers. Utilisation des entrées/sorties disque pendant le throughput maximal.
Flink	Event Processing Time : 500 ms Throughput : 500 000 événements/seconde CPU Utilization : 80% Memory Utilization : 60% Temps moyen pour traiter un événement de l'ingestion à la sortie. Nombre d'événements traités par seconde avec un cluster de 10 TaskManagers. Utilisation moyenne du CPU pendant le traitement maximal. Utilisation moyenne de la mémoire pendant le traitement maximal.
Spark	Batch Processing Time : 30 minutes Shuffle Read/Write Time : 15 minutes CPU Utilization : 85% Memory Utilization : 75% Temps nécessaire pour traiter un lot de 1 To de données avec 20 nœuds. Temps passé à lire et écrire des données pendant la phase de shuffle. Utilisation moyenne du CPU pendant le traitement par lots. Utilisation moyenne de la mémoire pendant le traitement par lots.

HDFS	Data Write Speed : 100 Mo/seconde/nœud Data Read Speed : 120 Mo/seconde/nœud Disk Space Utilization : 60% Replication Factor : 3 Vitesse à laquelle les données sont écrites dans HDFS à travers 10 nœuds. Vitesse à laquelle les données sont lues depuis HDFS à travers 10 nœuds. Pourcentage de l'espace disque utilisé pendant les opérations maximales. Nombre de répliques de données maintenues à travers le cluster pour la tolérance aux pannes.
Elasticsearch	Indexing Throughput : 50 000 documents/seconde Search Latency : 100 ms CPU Utilization : 70% Memory Utilization : 65% Nombre de documents indexés par seconde avec un cluster de 5 nœuds. Temps moyen pour retourner les résultats de recherche pour une requête. Utilisation moyenne du CPU pendant les opérations maximales d'indexation et de recherche. Utilisation moyenne de la mémoire pendant les opérations maximales d'indexation et de recherche.
MongoDB	Insert Latency : 2 ms Query Latency : 10 ms Write Throughput : 30 000 écritures/seconde Read Throughput : 40 000 lectures/seconde Temps moyen pour insérer un document dans la base de données. Temps moyen pour retourner les résultats d'une requête. Nombre d'écritures par seconde avec un ensemble de répliques de 3 nœuds. Nombre de lectures par seconde avec un ensemble de répliques de 3 nœuds.
PostgreSQL	Transaction Throughput : 20 000 transactions/seconde Query Latency : 5 ms CPU Utilization : 75% Memory Utilization : 70% Nombre de transactions traitées par seconde. Temps moyen pour retourner les résultats d'une requête. Utilisation moyenne du CPU pendant le traitement maximal des transactions. Utilisation moyenne de la mémoire pendant le traitement maximal des transactions.

Grafana	Dashboard Load Time : 1 seconde Concurrent Users : 100 Data Source Query Latency : 200 ms CPU Utilization : 60% Memory Utilization : 50% Temps moyen pour charger un tableau de bord avec des données en temps réel. Nombre d'utilisateurs accédant simultanément aux tableaux de bord sans dégradation des performances. Temps moyen pour récupérer des données à partir de sources intégrées (par ex., Prometheus, Elasticsearch). Utilisation moyenne du CPU pendant l'utilisation maximale. Utilisation moyenne de la mémoire pendant l'utilisation maximale.
Prometheus	Metrics Ingestion Rate : 1 million métriques/seconde Query Latency : 50 ms Storage Utilization : 70% CPU Utilization : 65% Nombre de métriques ingérées par seconde. Temps moyen pour retourner les résultats des requêtes PromQL. Pourcentage de stockage utilisé pour les données de métriques. Utilisation moyenne du CPU pendant les charges maximales d'ingestion et de requête.

TAB. B.1 : Performances de Référence des Composants de l'Écosystème Big Data

B.1 Code et Scripts de l'Écosystème Big Data

L'intégralité du code et des scripts utilisés pour la mise en place de notre écosystème Big Data est disponible sur notre dépôt GitHub. Les différents composants de l'écosystème, tels que Kafka, Zookeeper, ksqldb, Schema Registry, Kafka Connect, MongoDB, SQL Client, Flink, Elasticsearch, Kibana, PostgreSQL, cAdvisor, et Redpanda Console, HDFS, spark, Hive sont chacun configurés avec des Dockerfiles et des scripts d'entrypoint spécifiques pour assurer une configuration et un déploiement homogènes.

La structure du répertoire **docker/** est organisée comme suit :

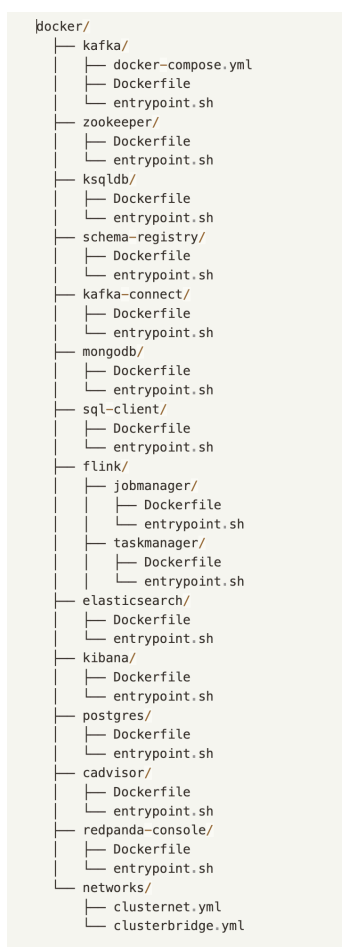


FIG. B.1 : Structure du répertoire contenant les Dockerfiles et les scripts de déploiement

Composant	Fichier	Description
kafka	docker-compose.yml, Dockerfile, entrypoint.sh	Configuration et démarrage de Kafka et ses dépendances.
zookeeper	Dockerfile, entrypoint.sh	Configuration de Zookeeper pour la coordination des nœuds Kafka.
ksqldb	Dockerfile, entrypoint.sh	Configuration de ksqlDB pour les requêtes SQL en temps réel sur Kafka.
schema-registry	Dockerfile, entrypoint.sh	Gestion et versionnement des schémas utilisés dans les applications de streaming.

Annexe B. Performance Benchmarks

kafka-connect	Dockerfile, entrypoint.sh	Intégration de Kafka avec d'autres systèmes de données.
mongodb	Dockerfile, entrypoint.sh	Configuration de MongoDB pour le stockage des données semi-structurées.
sql-client	Dockerfile, entrypoint.sh	Client SQL pour interagir avec les bases de données du système.
flink	Dockerfile, entrypoint.sh	Configuration de Flink pour le traitement de flux en temps réel.
elasticsearch	Dockerfile, entrypoint.sh	Configuration d'Elasticsearch pour l'indexation et la recherche de données.
kibana	Dockerfile, entrypoint.sh	Configuration de Kibana pour la visualisation des données indexées dans Elasticsearch.
postgres	Dockerfile, entrypoint.sh	Configuration de PostgreSQL pour le stockage des données structurées.
cadvisor	Dockerfile, entrypoint.sh	Surveillance des performances des conteneurs Docker.
redpanda-console	Dockerfile, entrypoint.sh	Interface de gestion et de surveillance pour Redpanda.
networks	clusternet.yml, cluster-bridge.yml	Configuration des réseaux Docker pour la communication entre les services.

Pour accéder au code et aux scripts, veuillez scanner le QR code ci-dessous ou cliquer sur le lien suivant :



FIG. B.2 : QR Code vers le dépôt GitHub

https://github.com/Haitam-B/BigData_infrastructure_multi-usecase/tree/main

